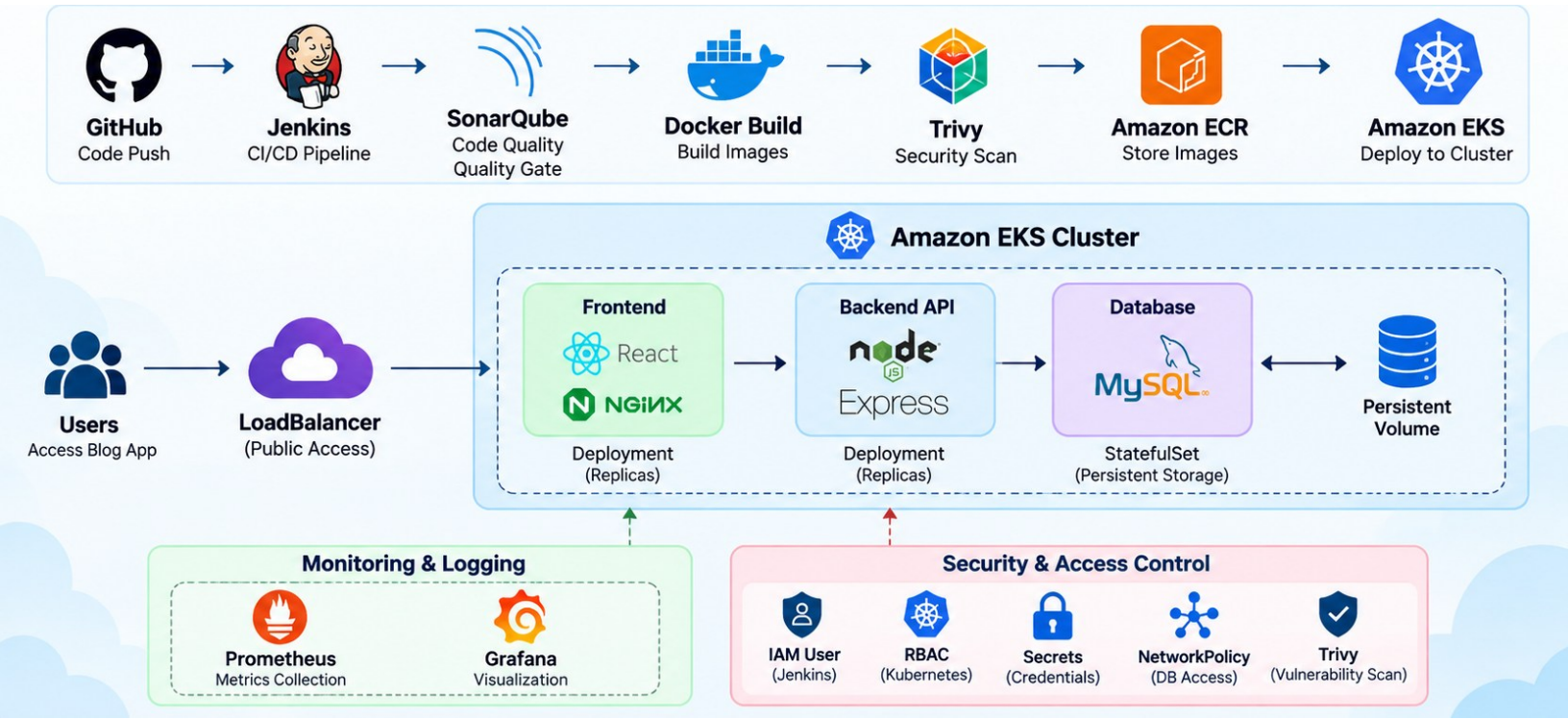
PROJECT - 4

Production-Level Three-Tier Blog Application CI/CD Deployment on Amazon EKS



Task 1 : Application

Overview

The Blog Application was cloned from the provided GitHub repository and successfully set up and verified locally as a three-tier application consisting of Frontend, Backend API, and MySQL Database. Communication between all three tiers was successfully verified.

Application Structure

- Frontend: React + Vite — Port 3000
- Backend: Node.js + Express — Port 5000
- Database: MySQL 8.0 — Port 3306
- Database Initialization: database/init.sql
- Backend Configuration: .env
- Git Ignore: .gitignore configured to exclude .env and node_modules/

Execution

The repository was cloned from the provided GitHub repository.
The MySQL database was started using Docker with the blogdb database and bloguser configured for application access.
The database was initialized using database/init.sql and verified with the users, posts, and comments tables.
Frontend and Backend dependencies were installed using npm install.
The Backend was configured using .env and started using npm start, followed by the Frontend using npm run dev.

Verification

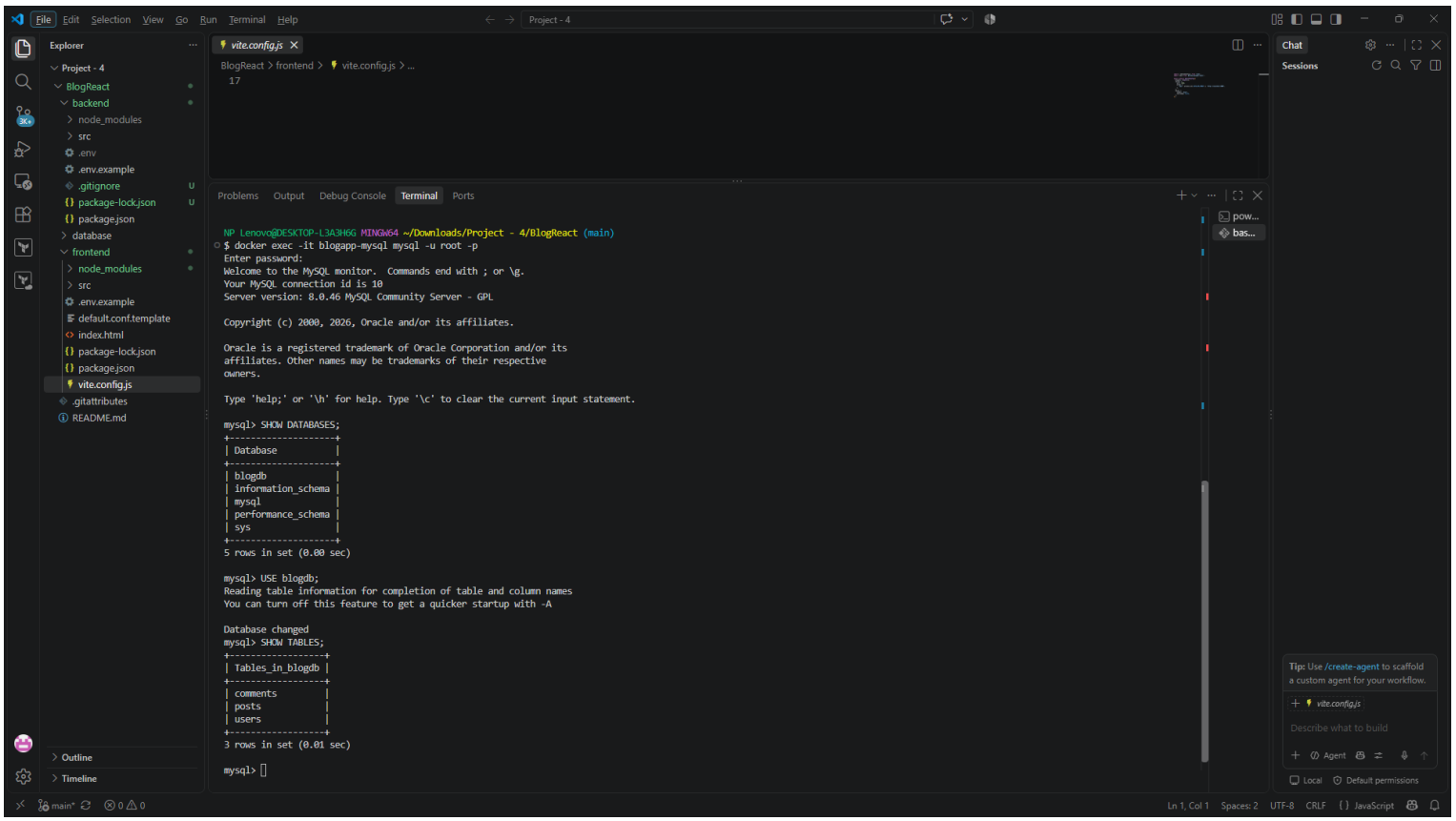
The Backend /healthz endpoint returned {"status":"ok"} and /readyz returned {"status":"ready"}.
The Backend /api/posts endpoint successfully returned blog post data from the database.
The Frontend successfully communicated with the Backend through /api/posts and blog posts were displayed successfully in the browser.
Backend logs also confirmed successful API requests from the Frontend.

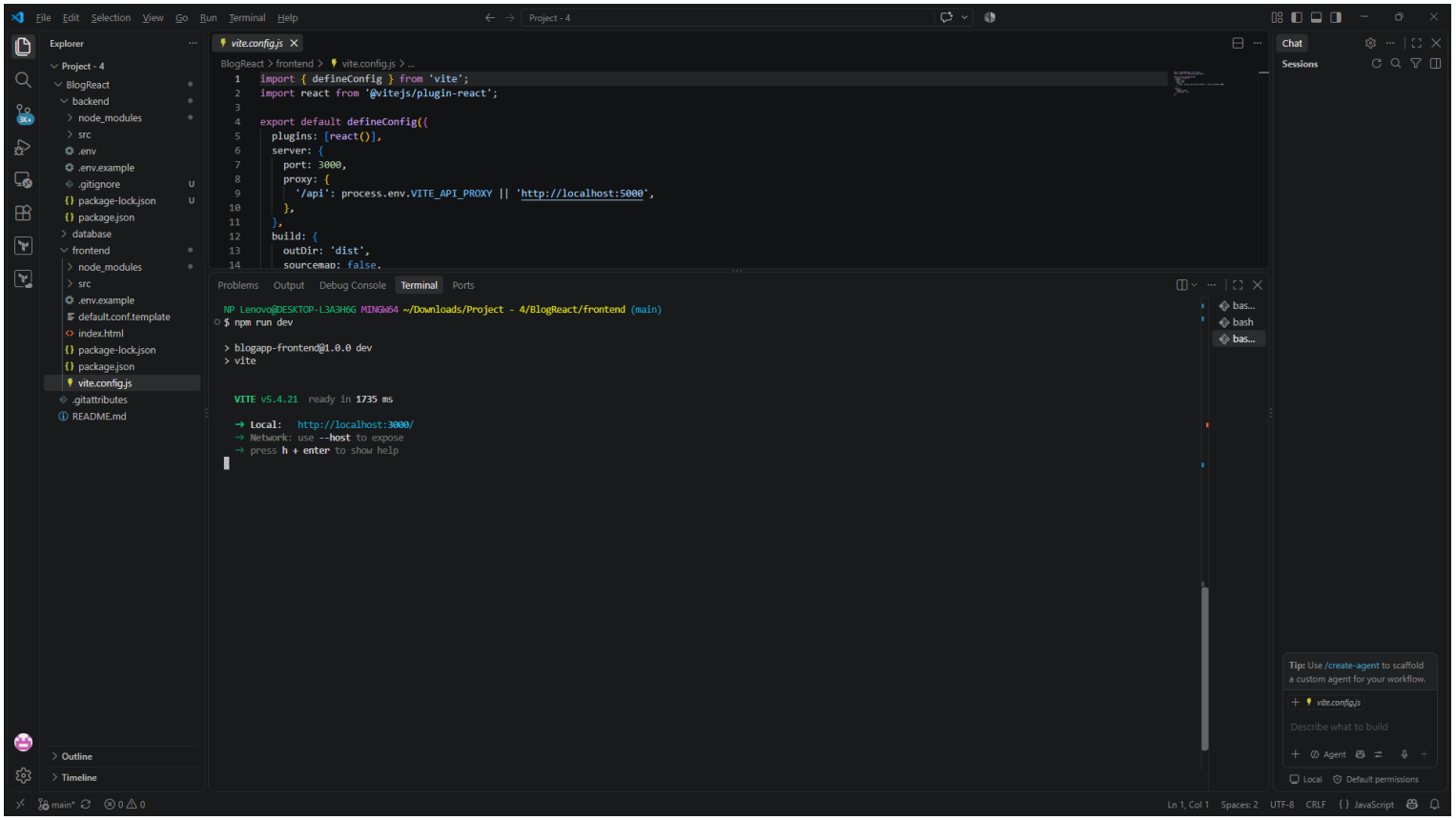
Result

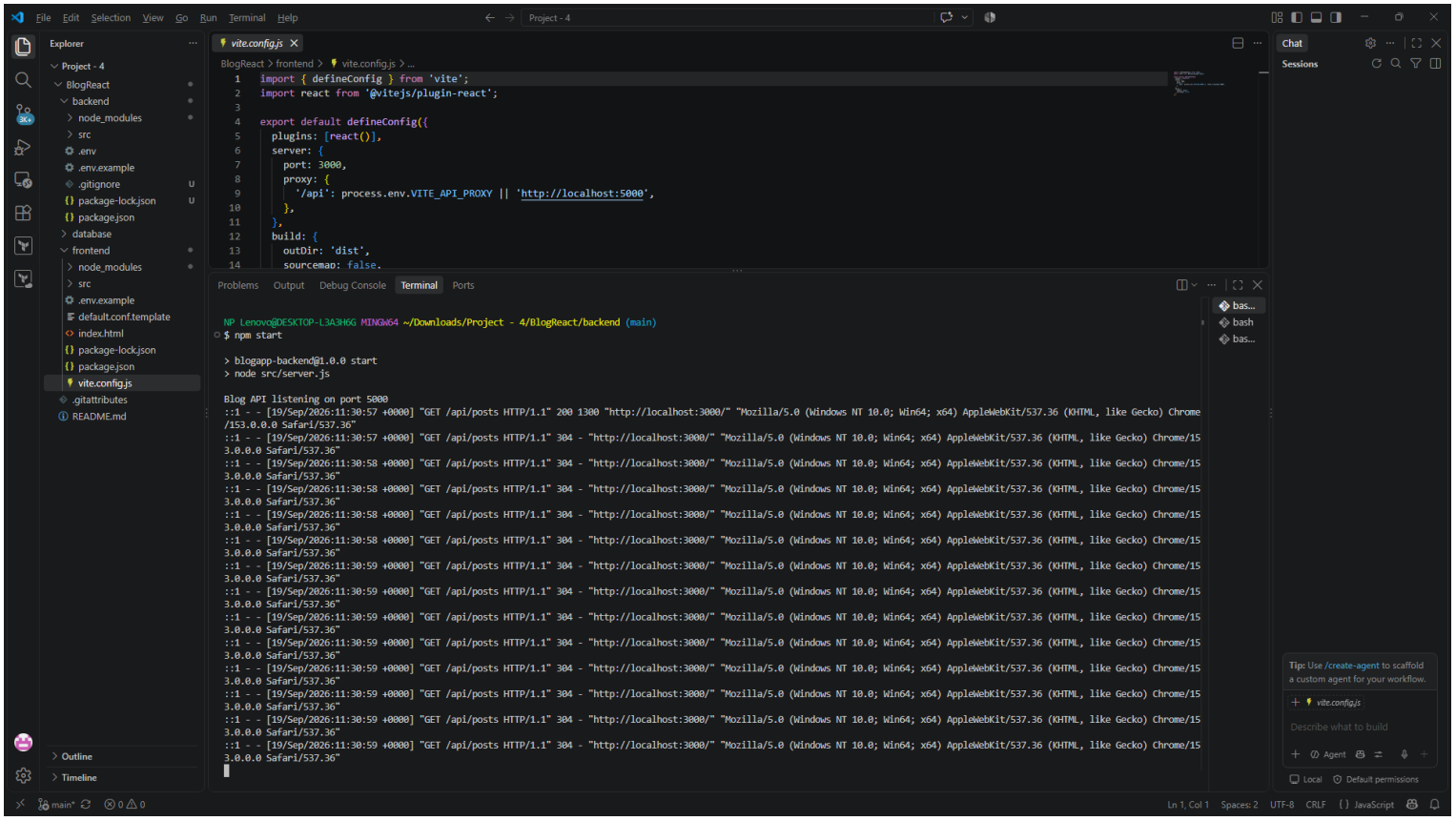
The three-tier application was successfully integrated and verified with the following flow:

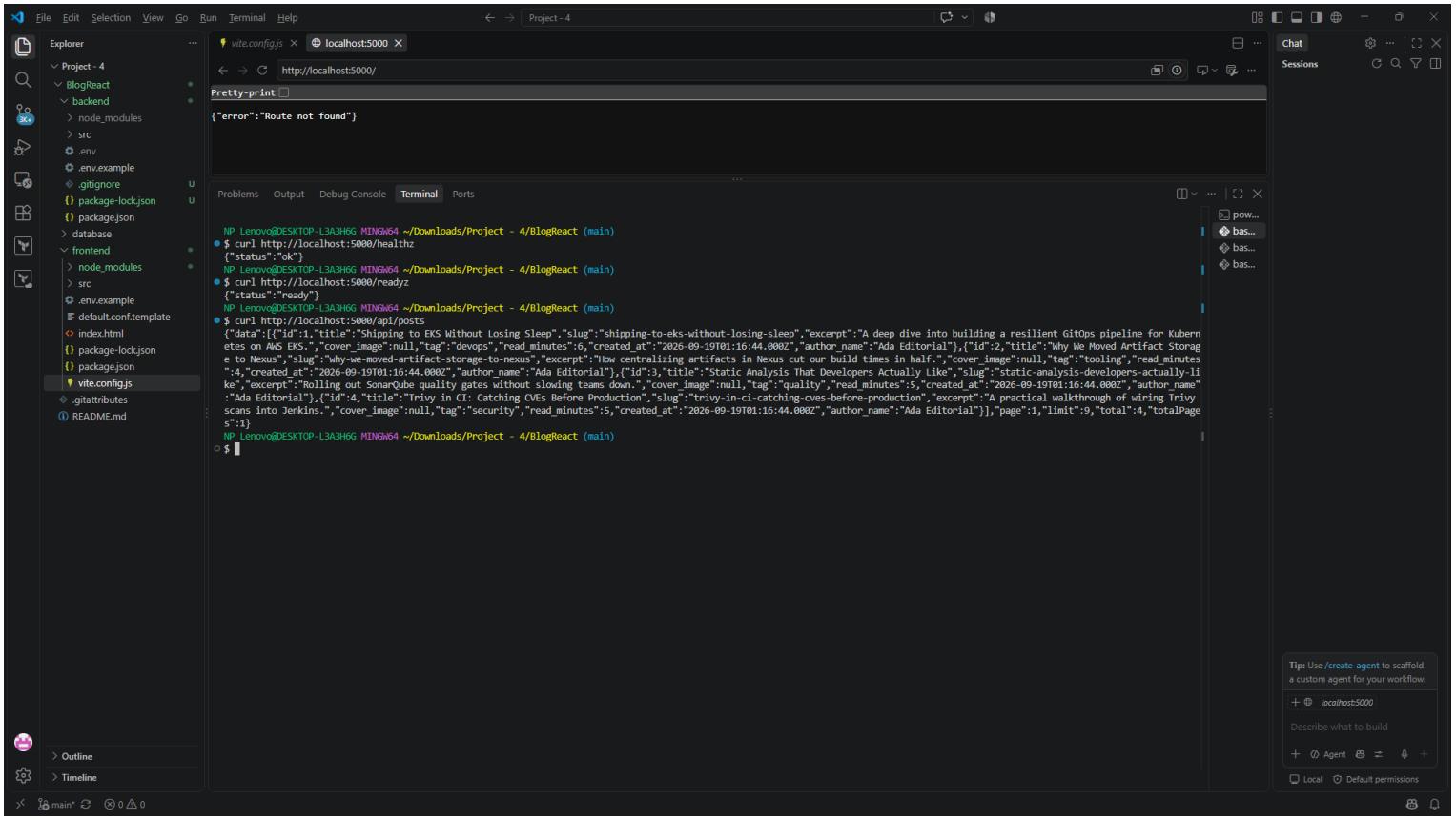
Frontend → Backend API → MySQL Database

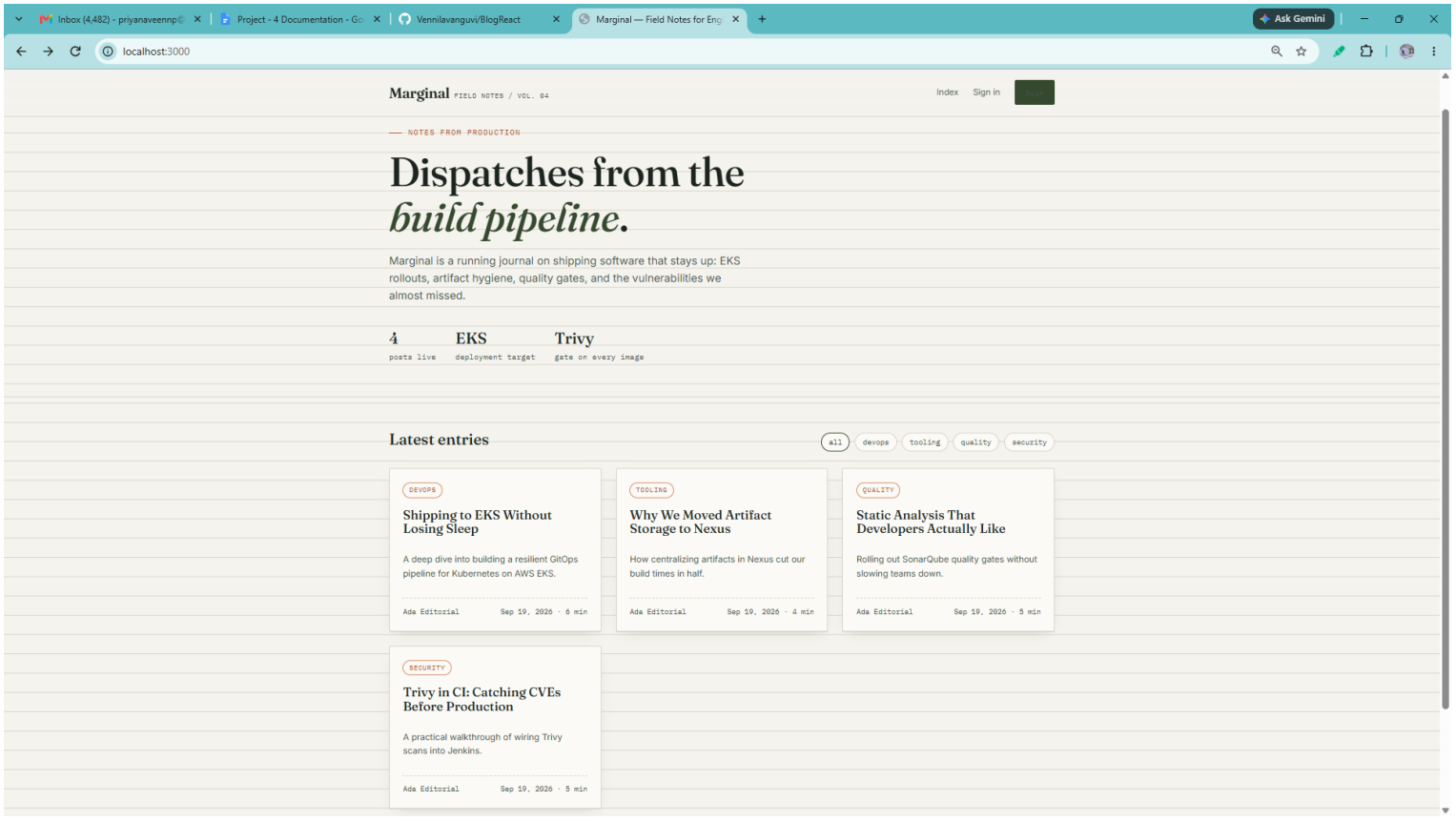
All three application components are working together successfully.




vite.config.js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
export default defineConfig({
plugins: [react()],
server: {
port: 3000,
proxy: {
'/api': process.env.VITE_API_PROXY || 'http://localhost:5000',
},
},
build: {
outDir: 'dist',
sourcemap: false,
Terminal
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact/frontend (main)
$ npm run dev
> blogapp-frontend@1.0.0 dev
> vite
VITE v5.4.21 ready in 1735 ms
→ Local: http://localhost:3000/
→ Network: use --host to expose
→ press h + enter to show help


vite.config.js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
export default defineConfig({
plugins: [react()],
server: {
port: 3000,
proxy: {
'/api': process.env.VITE_API_PROXY || 'http://localhost:5000',
},
},
build: {
outDir: 'dist',
sourcemap: false,
Terminal
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact/backend (main)
$ npm start
> blogapp-backend@1.0.0 start
> node src/server.js
Blog API listening on port 5000
::1 - - [19/Sep/2026:11:30:57 +0000] "GET /api/posts HTTP/1.1" 200 1300 "http://localhost:3000/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36"
::1 - - [19/Sep/2026:11:30:57 +0000] "GET /api/posts HTTP/1.1" 304 - "http://localhost:3000/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36"
::1 - - [19/Sep/2026:11:30:58 +0000] "GET /api/posts HTTP/1.1" 304 - "http://localhost:3000/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36"
::1 - - [19/Sep/2026:11:30:58 +0000] "GET /api/posts HTTP/1.1" 304 - "http://localhost:3000/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36"
::1 - - [19/Sep/2026:11:30:58 +0000] "GET /api/posts HTTP/1.1" 304 - "http://localhost:3000/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36"
::1 - - [19/Sep/2026:11:30:58 +0000] "GET /api/posts HTTP/1.1" 304 - "http://localhost:3000/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36"
::1 - - [19/Sep/2026:11:30:59 +0000] "GET /api/posts HTTP/1.1" 304 - "http://localhost:3000/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36"
::1 - - [19/Sep/2026:11:30:59 +0000] "GET /api/posts HTTP/1.1" 304 - "http://localhost:3000/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36"
::1 - - [19/Sep/2026:11:30:59 +0000] "GET /api/posts HTTP/1.1" 304 - "http://localhost:3000/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36"
::1 - - [19/Sep/2026:11:30:59 +0000] "GET /api/posts HTTP/1.1" 304 - "http://localhost:3000/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36"
::1 - - [19/Sep/2026:11:30:59 +0000] "GET /api/posts HTTP/1.1" 304 - "http://localhost:3000/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36"
::1 - - [19/Sep/2026:11:30:59 +0000] "GET /api/posts HTTP/1.1" 304 - "http://localhost:3000/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36"
::1 - - [19/Sep/2026:11:30:59 +0000] "GET /api/posts HTTP/1.1" 304 - "http://localhost:3000/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36"
::1 - - [19/Sep/2026:11:30:59 +0000] "GET /api/posts HTTP/1.1" 304 - "http://localhost:3000/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36"


Project - 4
BlogReact
backend
node_modules
src
.env
.env.example
.gitignore
package-lock.json
package.json
database
frontend
node_modules
src
.env.example
default.conf.template
index.html
package-lock.json
package.json
vite.config.js
.gitattributes
README.md
vite.config.js
localhost:5000
http://localhost:5000/
Pretty-print
{"error":"Route not found"}
Problems
Output
Debug Console
Terminal
Ports
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$ curl http://localhost:5000/healthz
{"status":"ok"}
$ curl http://localhost:5000/readyz
{"status":"ready"}
$ curl http://localhost:5000/api/posts
{"data":[{"id":1,"title":"Shipping to EKS Without Losing Sleep","slug":"shipping-to-eks-without-losing-sleep","excerpt":"A deep dive into building a resilient GitOps pipeline for Kubernetes on AWS EKS.","cover_image":null,"tag":"devops","read_minutes":6,"created_at":"2026-09-19T01:16:44.000Z","author_name":"Ada Editorial"},{"id":2,"title":"Why We Moved Artifact Storage to Nexus","slug":"why-we-moved-artifact-storage-to-nexus","excerpt":"How centralizing artifacts in Nexus cut our build times in half.","cover_image":null,"tag":"tooling","read_minutes":4,"created_at":"2026-09-19T01:16:44.000Z","author_name":"Ada Editorial"},{"id":3,"title":"Static Analysis That Developers Actually Like","slug":"static-analysis-developers-actually-like","excerpt":"Rolling out SonarQube quality gates without slowing teams down.","cover_image":null,"tag":"quality","read_minutes":5,"created_at":"2026-09-19T01:16:44.000Z","author_name":"Ada Editorial"},{"id":4,"title":"Trivy in CI: Catching CVEs Before Production","slug":"trivy-in-ci-catching-cves-before-production","excerpt":"A practical walkthrough of wiring Trivy scans into Jenkins.","cover_image":null,"tag":"security","read_minutes":5,"created_at":"2026-09-19T01:16:44.000Z","author_name":"Ada Editorial"}],"page":1,"limit":9,"total":4,"totalPages":1}
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Describe what to build
Agent
Local
Default permissions
Outline
Timeline
main*


Inbox (4,482) - priyanaveennp@
Project - 4 Documentation - Go
Vennilavanguvi/BlogReact
Marginal — Field Notes for Eng
Ask Gemini
localhost:3000
Marginal FIELD NOTES / VOL. 04
Index
Sign in
Subscribe
NOTES FROM PRODUCTION
Dispatches from the build pipeline.
Marginal is a running journal on shipping software that stays up: EKS rollouts, artifact hygiene, quality gates, and the vulnerabilities we almost missed.
4
posts live
EKS
deployment target
Trivy
gate on every image
Latest entries
all
devops
tooling
quality
security
DEVOPS
Shipping to EKS Without Losing Sleep
A deep dive into building a resilient GitOps pipeline for Kubernetes on AWS EKS.
Ada Editorial
Sep 19, 2026 · 6 min
TOOLING
Why We Moved Artifact Storage to Nexus
How centralizing artifacts in Nexus cut our build times in half.
Ada Editorial
Sep 19, 2026 · 4 min
QUALITY
Static Analysis That Developers Actually Like
Rolling out SonarQube quality gates without slowing teams down.
Ada Editorial
Sep 19, 2026 · 5 min
SECURITY
Trivy in CI: Catching CVEs Before Production
A practical walkthrough of wiring Trivy scans into Jenkins.
Ada Editorial
Sep 19, 2026 · 5 min

Task 2 : Docker

Overview

The Frontend and Backend applications were successfully containerized using Docker with multi-stage builds. Both applications were configured with non-root users, .dockerignore files, health checks, and environment-based configuration.

Docker Configuration

- Backend: blog-backend:tag1
- Frontend: blog-frontend:tag1
- Multi-stage Dockerfiles were created for both applications.
- Alpine-based production images were used.
- Backend runs as nodeuser and Frontend runs as nginxuser.
- .dockerignore files were configured for both applications.
- .env.example and environment-based configuration were used for application settings.
- Health checks were configured for both containers.
- Application credentials were supplied through environment variables instead of being hardcoded in the Dockerfiles.

Execution

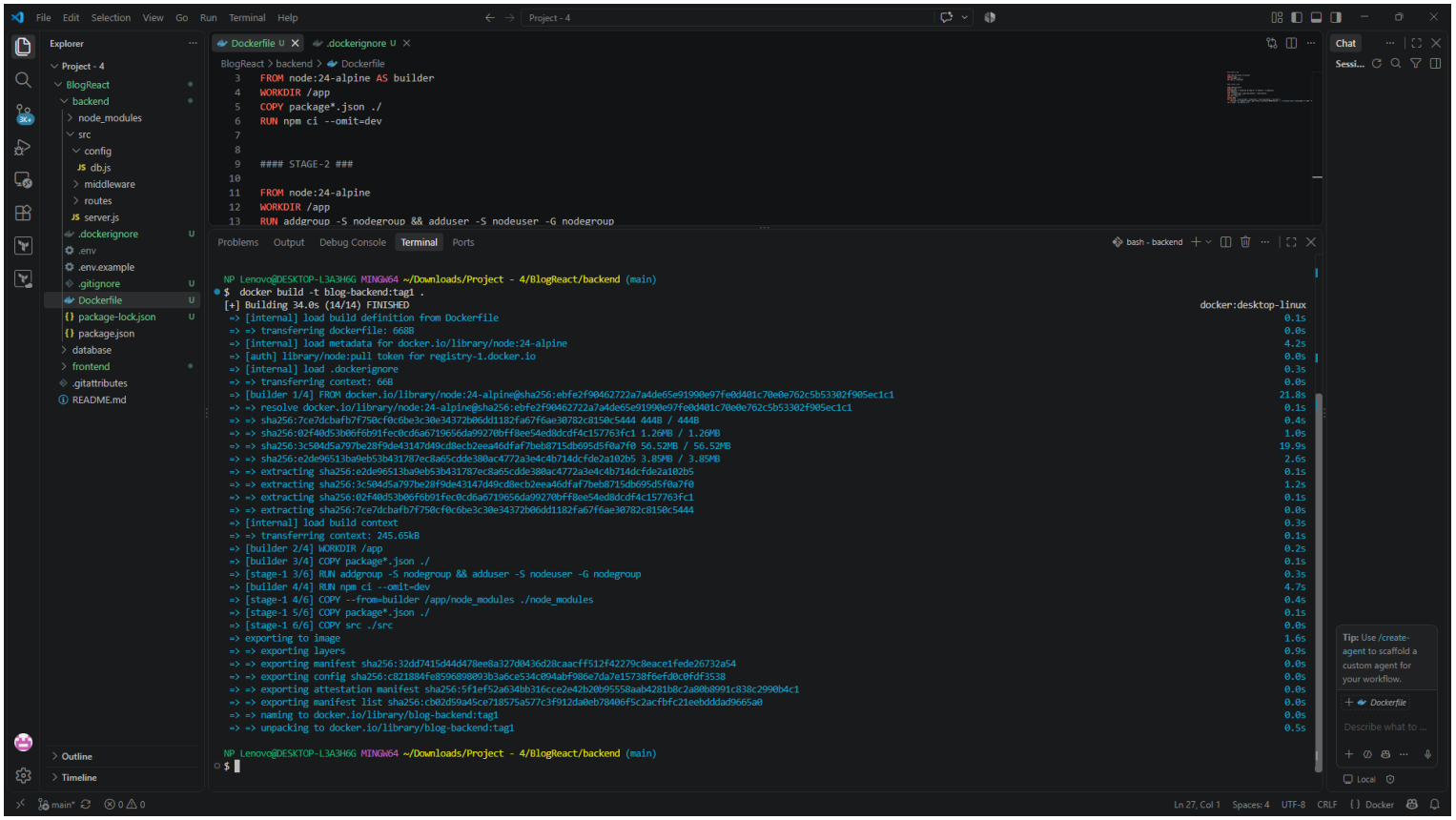
The Frontend and Backend Docker images were built successfully and started as containers.
The Frontend container was configured to communicate with the Backend using host.docker.internal, while the Backend was configured to connect to the local MySQL database using environment variables.

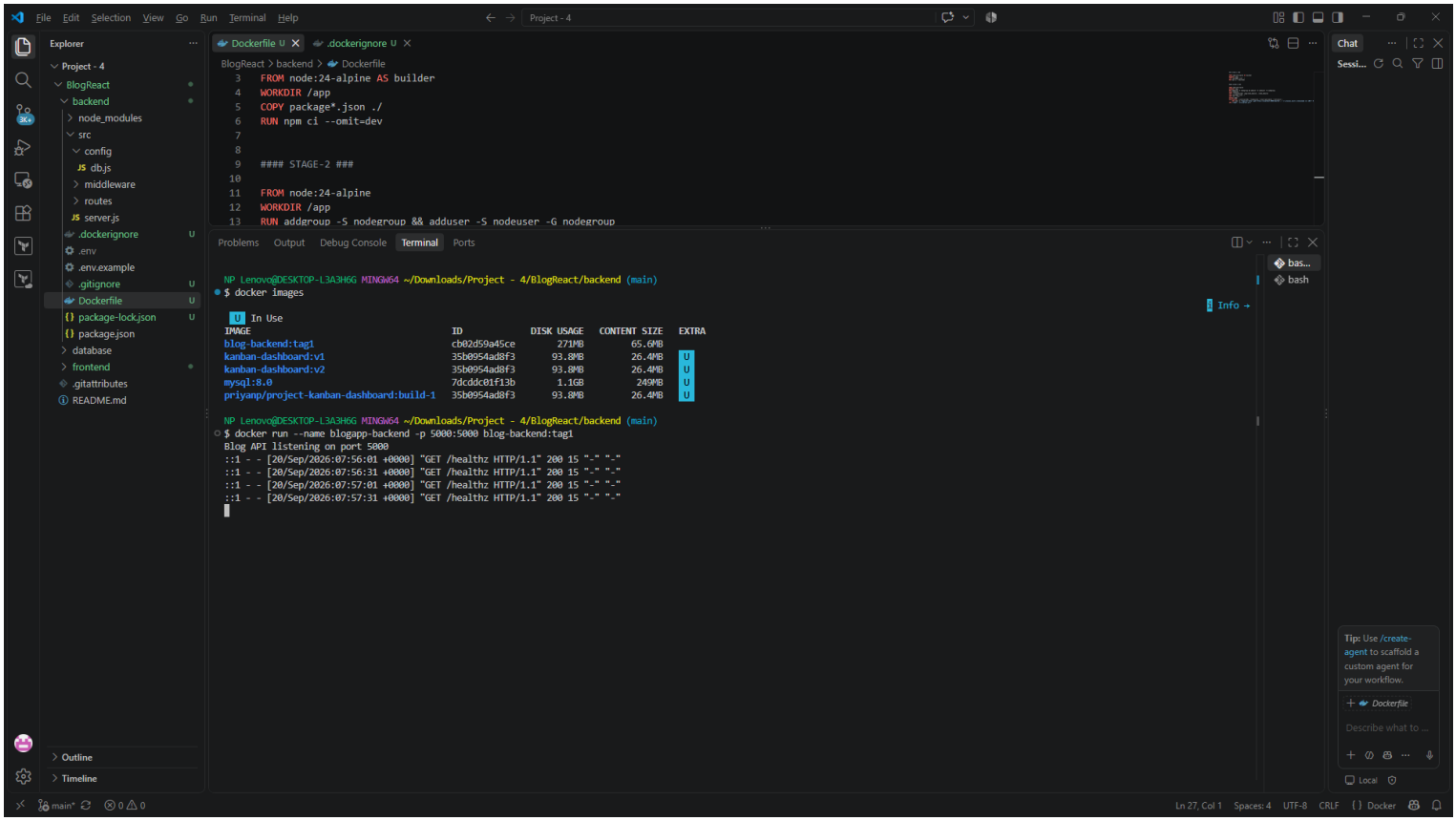
Verification

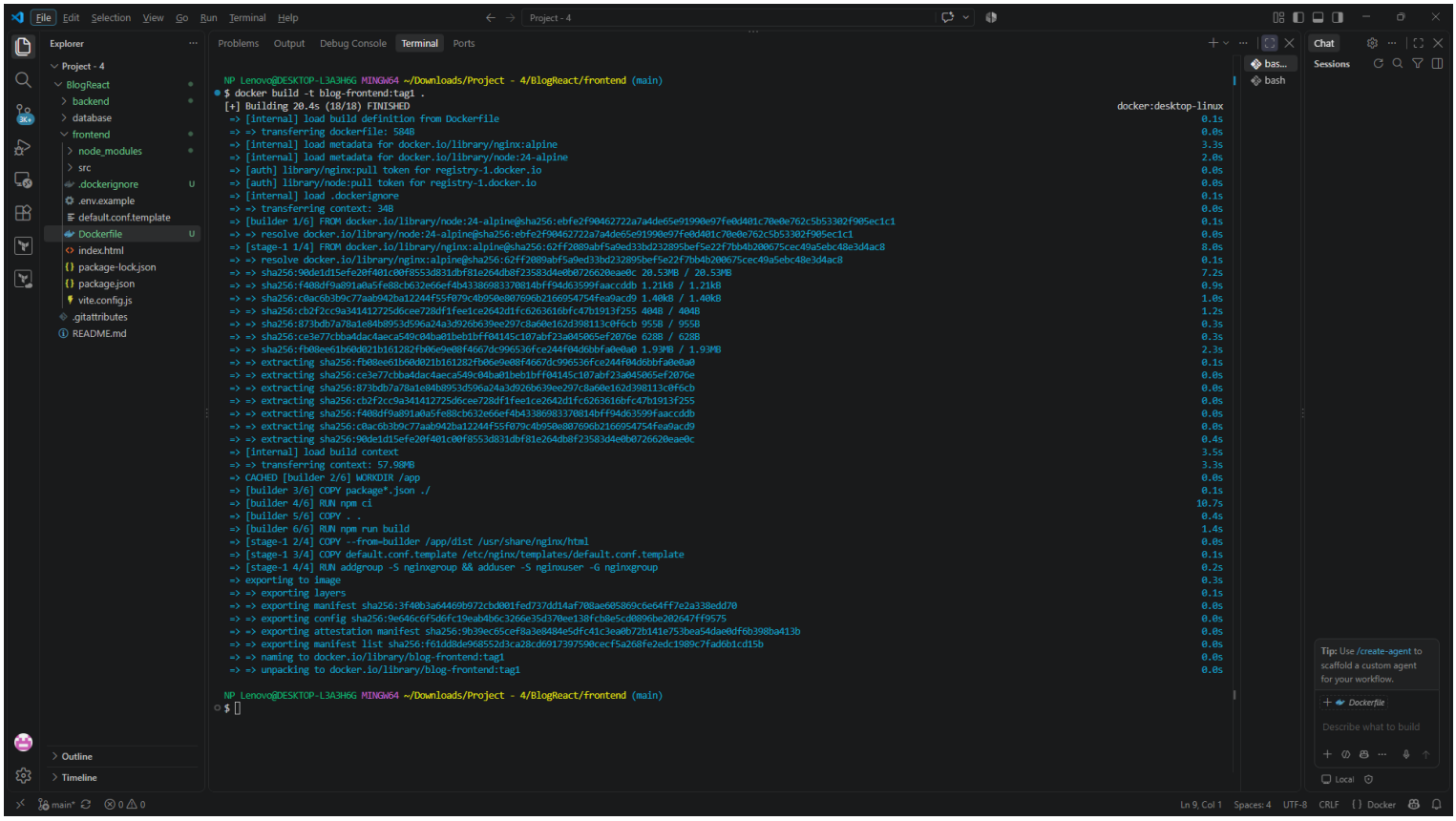
Both blog-frontend:tag1 and blog-backend:tag1 images were successfully created.
The Frontend and Backend containers were running with **healthy** status, and their non-root users were verified as nginxuser and nodeuser respectively.
The Frontend successfully communicated with the Backend, the Backend retrieved blog post data from MySQL, and the posts were successfully displayed in the browser.

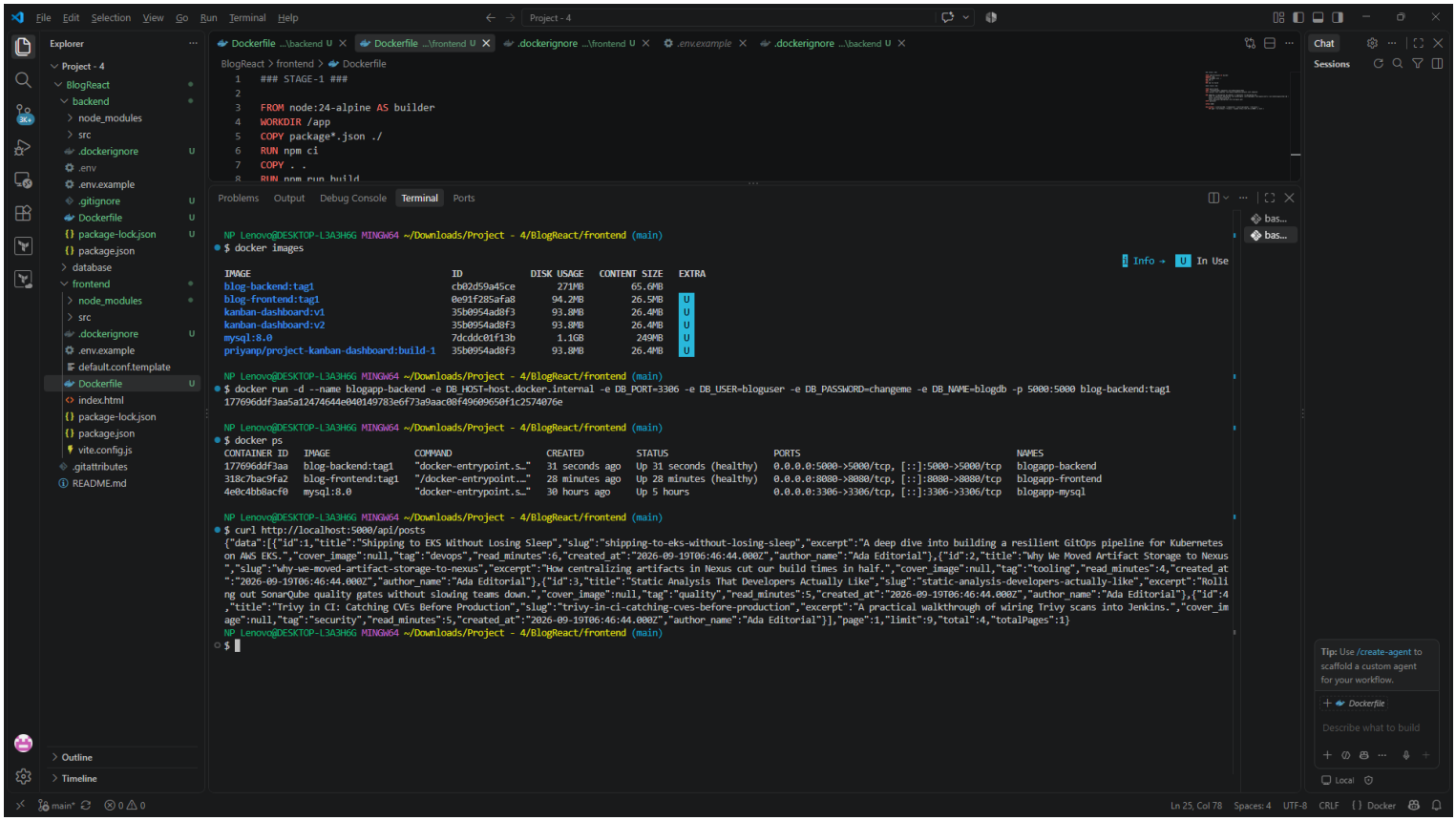
Result

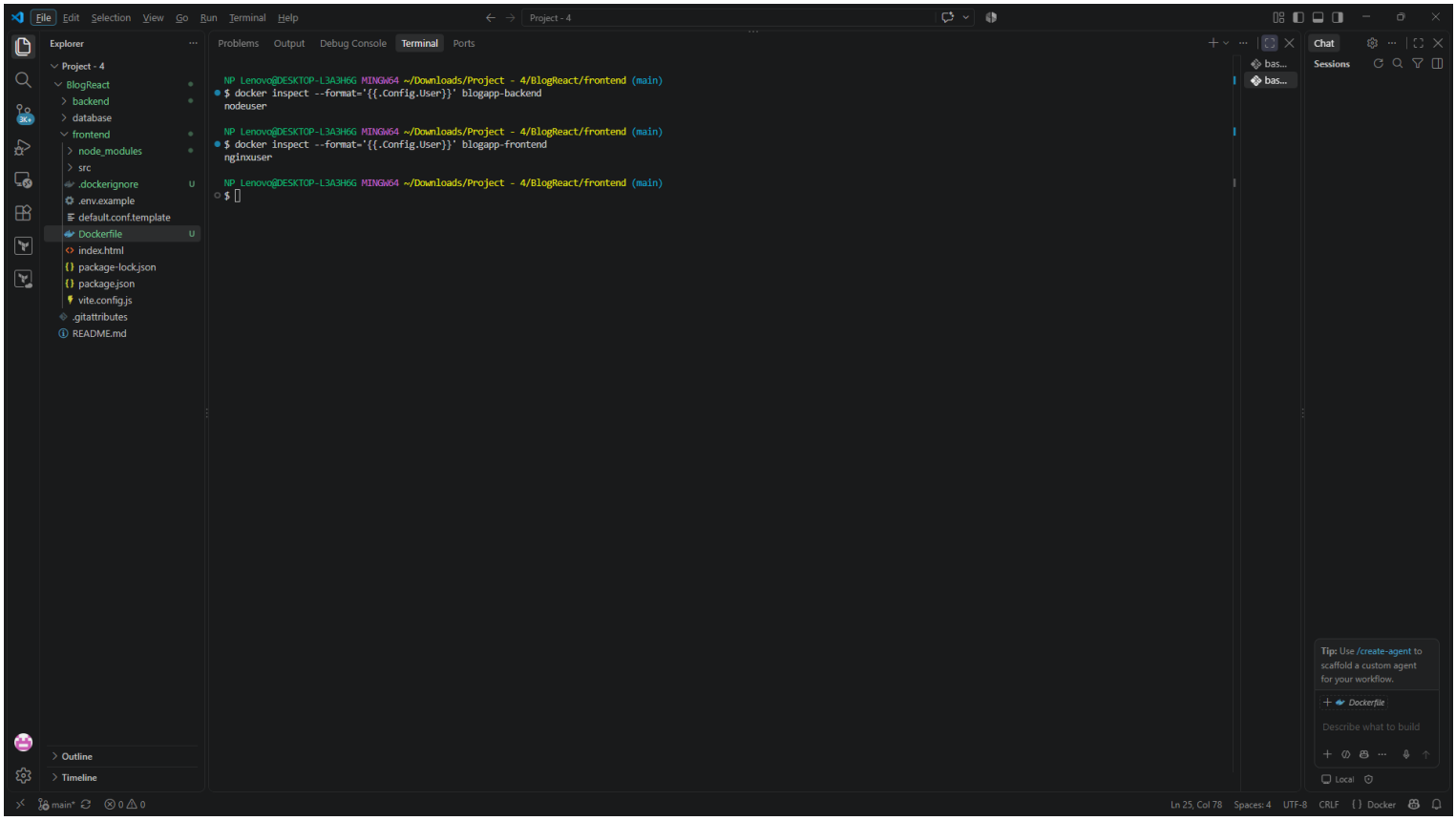
The Frontend and Backend applications were successfully containerized and verified according to the Task 2 requirements.

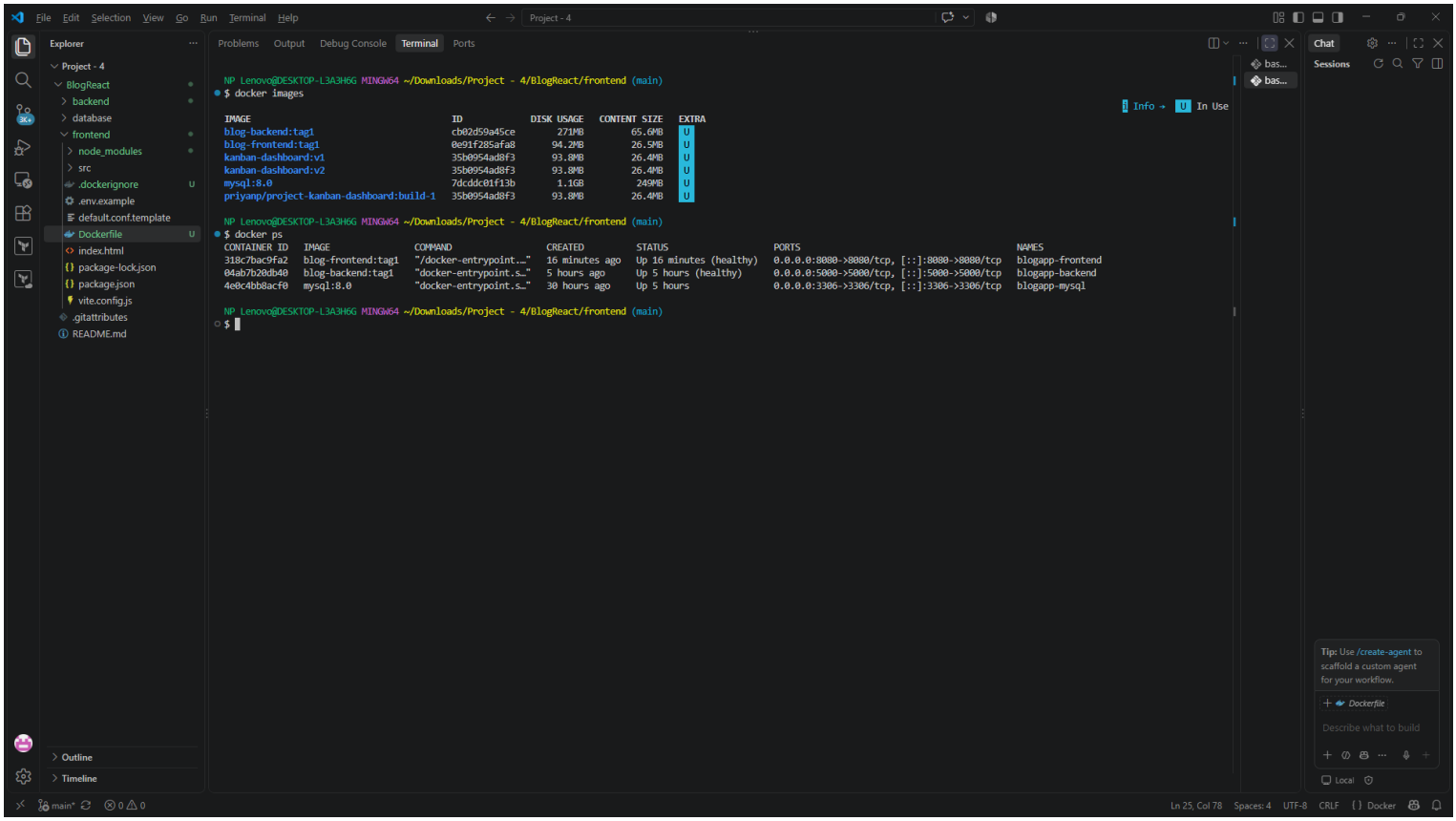



Dockerfile
.dockerignore
BlogReact > backend > Dockerfile
FROM node:24-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
STAGE-2
FROM node:24-alpine
WORKDIR /app
RUN addgroup -S nodegroup && adduser -S nodeuser -G nodegroup
Problems Output Debug Console Terminal Ports
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact/backend (main)
$ docker images
U In Use
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
blog-backend:tag1 cb02d59a45ce 271MB 65.6MB
kanban-dashboard:v1 35b0954ad8f3 93.8MB 26.4MB U
kanban-dashboard:v2 35b0954ad8f3 93.8MB 26.4MB U
mysql:8.0 7dcddc01f13b 1.1GB 249MB U
priyanp/project-kanban-dashboard:build-1 35b0954ad8f3 93.8MB 26.4MB U
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact/backend (main)
$ docker run --name blogapp-backend -p 5000:5000 blog-backend:tag1
Blog API listening on port 5000
::1 - - [20/Sep/2026:07:56:01 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "-"
::1 - - [20/Sep/2026:07:56:31 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "-"
::1 - - [20/Sep/2026:07:57:01 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "-"
::1 - - [20/Sep/2026:07:57:31 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "-"
Chat
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Describe what to ...


NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact/frontend (main)
$ docker build -t blog-frontend:tag1 .
[+] Building 20.4s (18/18) FINISHED docker:desktop-linux
=> [internal] load build definition from Dockerfile 0.1s
=> [internal] load metadata for docker.io/library/nginx:alpine 3.3s
=> [internal] load metadata for docker.io/library/node:24-alpine 2.0s
=> [auth] library/nginx:pull token for registry-1.docker.io 0.0s
=> [auth] library/node:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore 0.1s
=> => transferring context: 34B 0.0s
=> [builder 1/6] FROM docker.io/library/node:24-alpine@sha256:ebfe2f90462722a7a4de65e91990e97fe0d401c70e0e762c5b53302f905ec1c1 0.1s
=> => resolve docker.io/library/node:24-alpine@sha256:ebfe2f90462722a7a4de65e91990e97fe0d401c70e0e762c5b53302f905ec1c1 0.0s
=> [stage-1 1/4] FROM docker.io/library/nginx:alpine@sha256:62ff2089abf5a9ed33bd232895bef5e22f7bb4b200675cec49a5ebc48e3d4ac8 8.0s
=> => resolve docker.io/library/nginx:alpine@sha256:62ff2089abf5a9ed33bd232895bef5e22f7bb4b200675cec49a5ebc48e3d4ac8 0.1s
=> => sha256:90de1d15efe20f401c00f8553d831dbf81e264db8f23583d4e0b0726620eae0c 20.53MB / 20.53MB 7.2s
=> => sha256:f408df9a891a0a5fe88cb632e66ef4b43386983370814bff94d63599faaccddb 1.21kB / 1.21kB 0.9s
=> => sha256:c0ac6b3b9c77aab942ba12244f55f879c4b950e807696b2166954754fea9acd9 1.40kB / 1.40kB 1.0s
=> => sha256:cb2f2cc9a341412725d6cee728df1fee1ce2642d1fc6263616bfc47b1913f255 404B / 404B 1.2s
=> => sha256:873bdb7a78a1e84b8953d596a24a3d926b639ee297c8a60e162d398113c0f6cb 955B / 955B 0.3s
=> => sha256:ce3e77cbba4dac4aeca549c04ba01beb1bff04145c107abf23a045065ef2076e 628B / 628B 0.3s
=> => sha256:fb08ee61b60d021b161282fb06e9e08f4667dc996536fce244f04d6bbfa0e0a0 1.93MB / 1.93MB 2.3s
=> => extracting sha256:fb08ee61b60d021b161282fb06e9e08f4667dc996536fce244f04d6bbfa0e0a0 0.1s
=> => extracting sha256:ce3e77cbba4dac4aeca549c04ba01beb1bff04145c107abf23a045065ef2076e 0.0s
=> => extracting sha256:873bdb7a78a1e84b8953d596a24a3d926b639ee297c8a60e162d398113c0f6cb 0.0s
=> => extracting sha256:cb2f2cc9a341412725d6cee728df1fee1ce2642d1fc6263616bfc47b1913f255 0.0s
=> => extracting sha256:f408df9a891a0a5fe88cb632e66ef4b43386983370814bff94d63599faaccddb 0.0s
=> => extracting sha256:c0ac6b3b9c77aab942ba12244f55f879c4b950e807696b2166954754fea9acd9 0.0s
=> => extracting sha256:90de1d15efe20f401c00f8553d831dbf81e264db8f23583d4e0b0726620eae0c 0.4s
=> [internal] load build context 3.5s
=> => transferring context: 57.98MB 3.3s
=> CACHED [builder 2/6] WORKDIR /app 0.0s
=> [builder 3/6] COPY package*.json ./ 0.1s
=> [builder 4/6] RUN npm ci 10.7s
=> [builder 5/6] COPY . . 0.4s
=> [builder 6/6] RUN npm run build 1.4s
=> [stage-1 2/4] COPY --from=builder /app/dist /usr/share/nginx/html 0.0s
=> [stage-1 3/4] COPY default.conf.template /etc/nginx/templates/default.conf.template 0.1s
=> [stage-1 4/4] RUN addgroup -S nginxgroup && adduser -S nginxuser -G nginxgroup 0.2s
=> exporting to image 0.3s
=> => exporting layers 0.1s
=> => exporting manifest sha256:3f40b3a64469b972cbd001fed737dd14af708ae605869c6e64ff7e2a338edd70 0.0s
=> => exporting config sha256:9e646c6f5d6fc19eab4b6c3266e35d370ee138fcb8e5cd0896be202647ff9575 0.0s
=> => exporting attestation manifest sha256:9b39ec65cef8a3e8484e5dfc41c3ea0b72b141e753bea54dae0df6b398ba413b 0.0s
=> => exporting manifest list sha256:f61dd8de968552d3ca28cd6917397590cecf5a268fe2edc1989c7fad6b1cd15b 0.0s
=> => naming to docker.io/library/blog-frontend:tag1 0.0s
=> => unpacking to docker.io/library/blog-frontend:tag1 0.0s
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact/frontend (main)
$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Describe what to build


Dockerfile
STAGE-1
FROM node:24-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build
Problems Output Debug Console Terminal Ports
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact/frontend (main)
$ docker images
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
blog-backend:tag1 cb02d59a45ce 271MB 65.6MB U
blog-frontend:tag1 0e01f285afa8 94.2MB 26.5MB U
kanban-dashboard:v1 35b0954ad8f3 93.8MB 26.4MB U
kanban-dashboard:v2 35b0954ad8f3 93.8MB 26.4MB U
mysql:8.0 7dcddc01f13b 1.1GB 249MB U
priyanp/project-kanban-dashboard:build-1 35b0954ad8f3 93.8MB 26.4MB U
$ docker run -d --name blogapp-backend -e DB_HOST=host.docker.internal -e DB_PORT=3306 -e DB_USER=bloguser -e DB_PASSWORD=changeme -e DB_NAME=blogdb -p 5000:5000 blog-backend:tag1
177696ddf3aa5a12474644e040149783e6f73a9aac08f4960650f1c2574076e
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
177696ddf3aa blog-backend:tag1 "docker-entrypoint.s…" 31 seconds ago Up 31 seconds (healthy) 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp blogapp-backend
318c7bac9fa2 blog-frontend:tag1 "/docker-entrypoint.…" 28 minutes ago Up 28 minutes (healthy) 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp blogapp-frontend
4e0c4bb8acf0 mysql:8.0 "docker-entrypoint.s…" 30 hours ago Up 5 hours 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp blogapp-mysql
$ curl http://localhost:5000/api/posts
{"data":[{"id":1,"title":"Shipping to EKS Without Losing Sleep","slug":"shipping-to-eks-without-losing-sleep","excerpt":"A deep dive into building a resilient GitOps pipeline for Kubernetes on AWS EKS.","cover_image":null,"tag":"devops","read_minutes":6,"created_at":"2026-09-19T06:46:44.000Z","author_name":"Ada Editorial"},{"id":2,"title":"Why We Moved Artifact Storage to Nexus","slug":"why-we-moved-artifact-storage-to-nexus","excerpt":"How centralizing artifacts in Nexus cut our build times in half.","cover_image":null,"tag":"tooling","read_minutes":4,"created_at":"2026-09-19T06:46:44.000Z","author_name":"Ada Editorial"},{"id":3,"title":"Static Analysis That Developers Actually Like","slug":"static-analysis-developers-actually-like","excerpt":"Rolling out SonarQube quality gates without slowing teams down.","cover_image":null,"tag":"quality","read_minutes":5,"created_at":"2026-09-19T06:46:44.000Z","author_name":"Ada Editorial"},{"id":4,"title":"Trivy in CI: Catching CVEs Before Production","slug":"trivy-in-ci-catching-cves-before-production","excerpt":"A practical walkthrough of wiring Trivy scans into Jenkins.","cover_image":null,"tag":"security","read_minutes":5,"created_at":"2026-09-19T06:46:44.000Z","author_name":"Ada Editorial"}],"page":1,"limit":9,"total":4,"totalPages":1}
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact/frontend (main)
$


NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact/frontend (main)
$ docker inspect --format='{{.Config.User}}' blogapp-backend
nodeuser
$ docker inspect --format='{{.Config.User}}' blogapp-frontend
nginxuser
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact/frontend (main)
$


File Edit Selection View Go Run Terminal Help
Project - 4
Explorer
Project - 4
BlogReact
backend
database
frontend
node_modules
src
.dockerignore
.env.example
default.conf.template
Dockerfile
index.html
package-lock.json
package.json
vite.config.js
.gitattributes
README.md
Outline
Timeline
Problems Output Debug Console Terminal Ports
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact/frontend (main)
$ docker images
Info U In Use
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
blog-backend:tag1 cb02d59a45ce 271MB 65.6MB U
blog-frontend:tag1 0e91f285afa8 94.2MB 26.5MB U
kanban-dashboard:v1 35b0954ad8f3 93.8MB 26.4MB U
kanban-dashboard:v2 35b0954ad8f3 93.8MB 26.4MB U
mysql:8.0 7dcddc01f13b 1.1GB 249MB U
priyanp/project-kanban-dashboard:build-1 35b0954ad8f3 93.8MB 26.4MB U
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact/frontend (main)
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
318c7bac0fa2 blog-frontend:tag1 "/docker-entrypoint.…" 16 minutes ago Up 16 minutes (healthy) 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp blogapp-frontend
04ab7b20db40 blog-backend:tag1 "docker-entrypoint.s…" 5 hours ago Up 5 hours (healthy) 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp blogapp-backend
4e0c4bb8acf0 mysql:8.0 "docker-entrypoint.s…" 30 hours ago Up 5 hours 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp blogapp-mysql
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact/frontend (main)
$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Dockerfile
Describe what to build
Local
main*
Ln 25, Col 78 Spaces: 4 UTF-8 CRLF Docker

Task 3 : Database

Overview

MySQL was deployed inside Amazon EKS using a Kubernetes StatefulSet with persistent storage, secure credentials, database initialization, backend integration, and restricted network access.

Execution

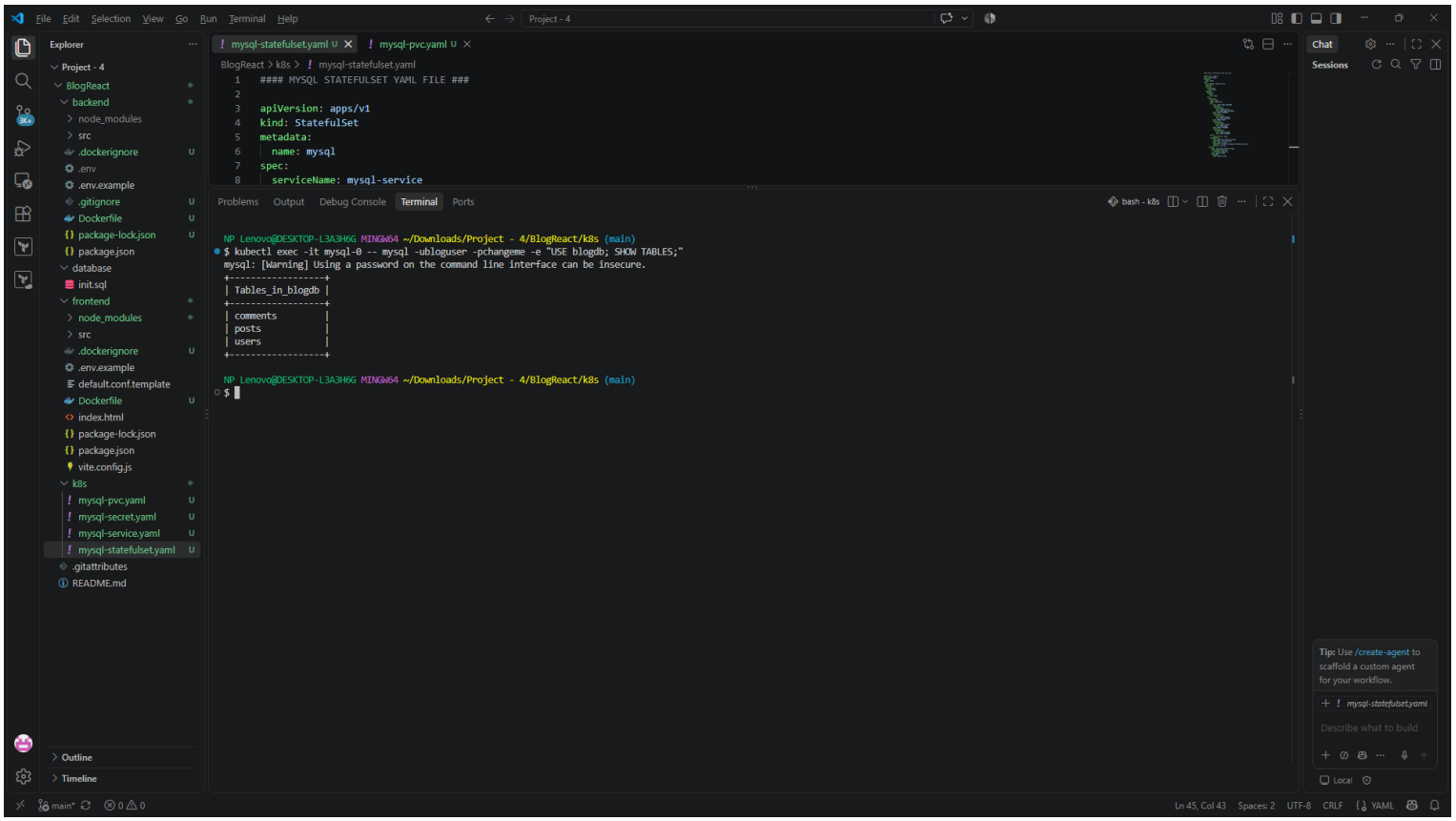
- Created a Kubernetes Secret : `mysql-secret` — to store MySQL credentials securely
- Created a 5 GiB PersistentVolumeClaim : `mysql-pvc` — using gp2 storage class backed by AWS EBS
- Loaded the provided `init.sql` file as a ConfigMap : `mysql-initdb` — and mounted it into the MySQL container to initialize the database and create the required tables
- Deployed MySQL as a StatefulSet : `mysql` — with 1 replica, connected to the PVC for persistent storage
- Exposed MySQL internally using a ClusterIP Service : `mysql-service` — on port 3306
- Deployed the Backend using the versioned ECR image and configured it to connect to MySQL via mysql-service using Secret-based credentials
- Created a NetworkPolicy : `mysql-allow-backend-only` — to allow MySQL access only from Backend pods
- Enabled EKS VPC CNI NetworkPolicy feature and Network Policy Agent to enforce the NetworkPolicy

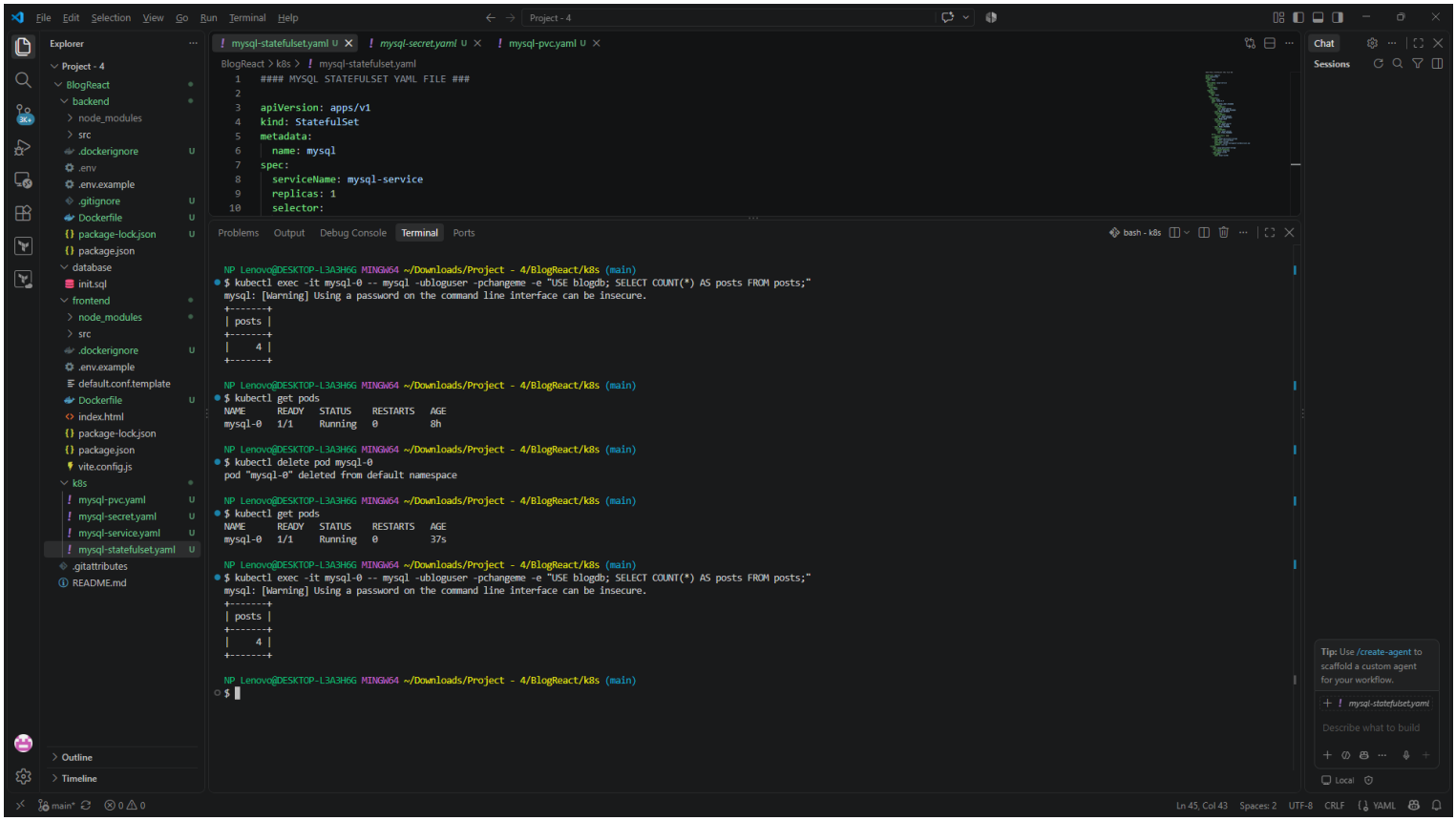
Verification Results

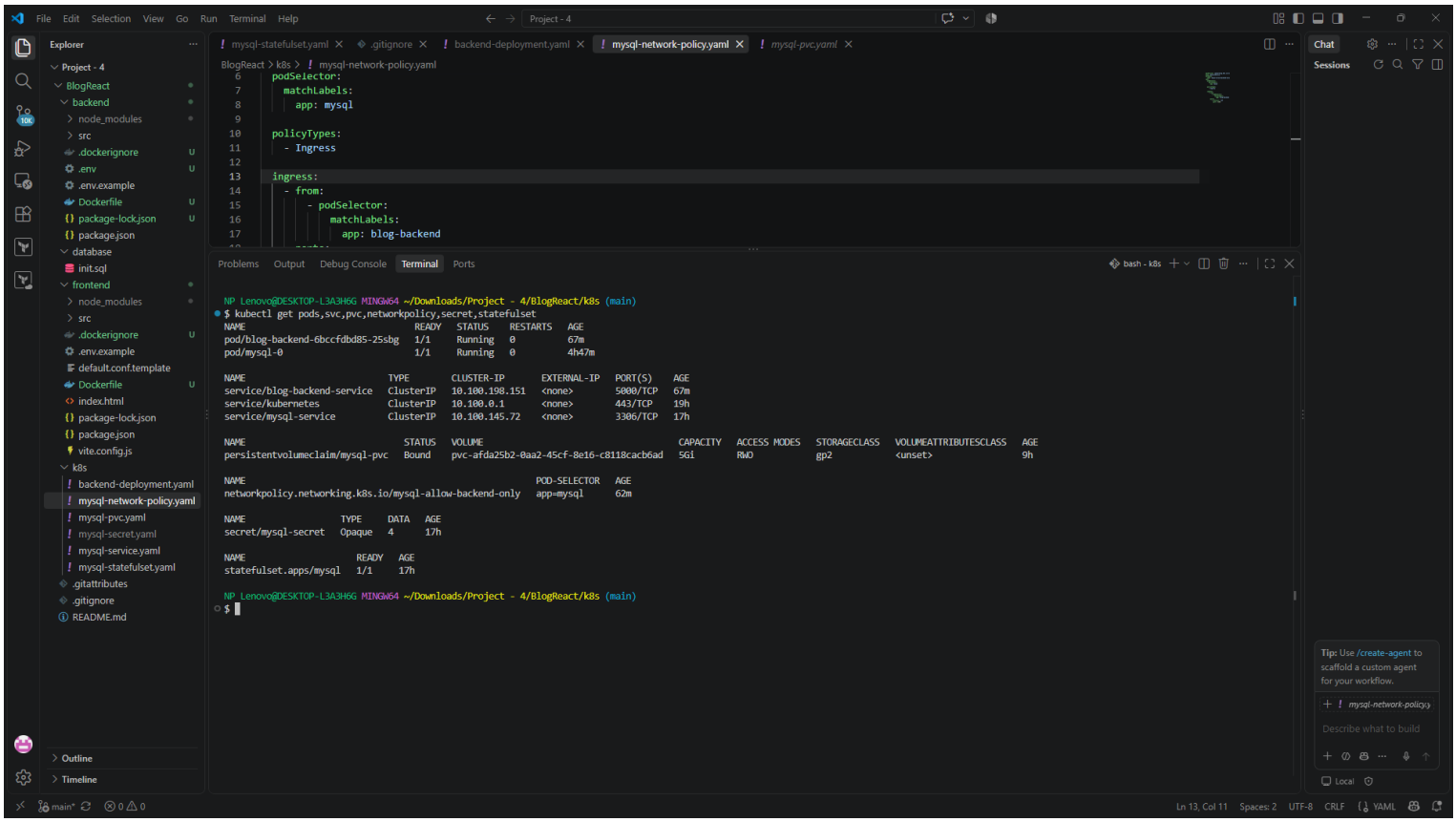
- `mysql-0 pod` → Running
- `mysql-pvc` → Bound — 5 GiB — gp2
- `mysql-service` → ClusterIP — port 3306
- Tables verified → users, posts, comments created successfully
- Backend `/readyz` → returned status ready
- Backend `/api/posts` → returned all 4 blog records from MySQL
- Pod recreation test → data remained intact after `mysql-0` was deleted and recreated — confirming persistent storage
- NetworkPolicy test → unauthorized test pod was blocked from connecting to MySQL on port 3306 — connection blocked, confirming restricted database access
- Final cluster check → both pods running, PVC bound, services available, NetworkPolicy active

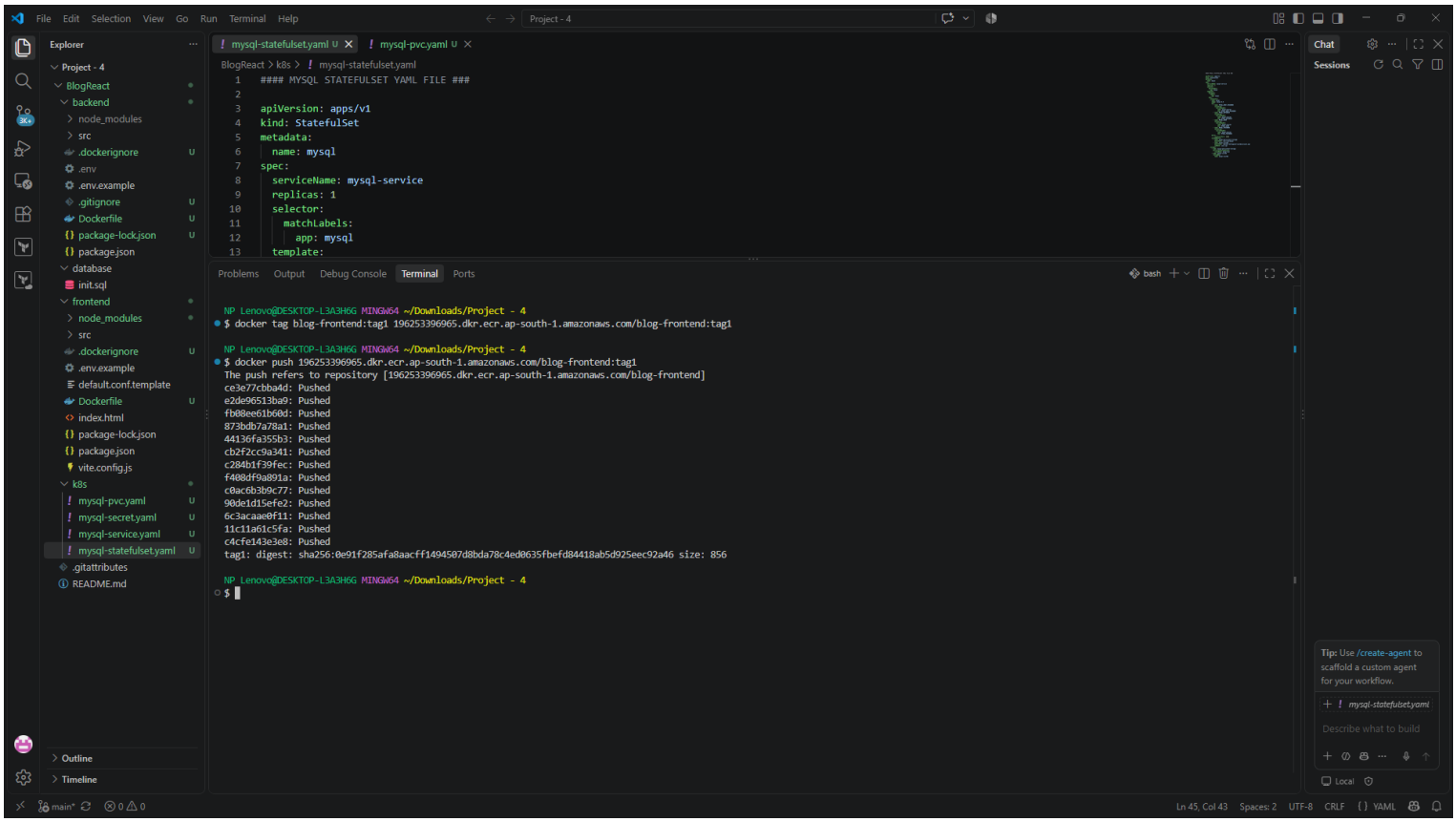
Result

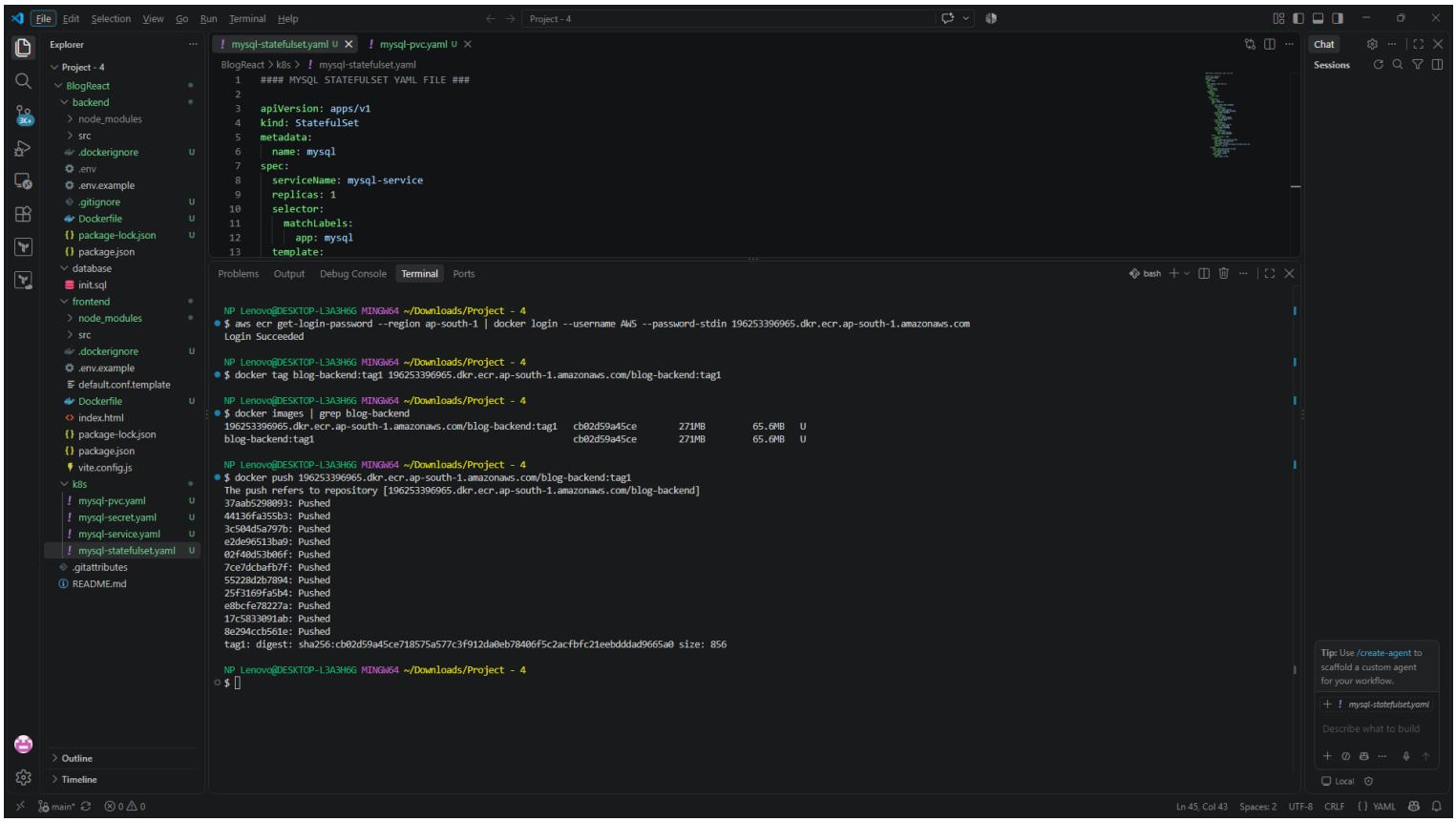
MySQL was successfully deployed inside Amazon EKS with StatefulSet-based deployment, persistent EBS storage, secure credential management, database initialization, backend integration, data persistence across pod restarts, and MySQL access restricted to Backend pods using NetworkPolicy — verified by blocking an unauthorized test pod.

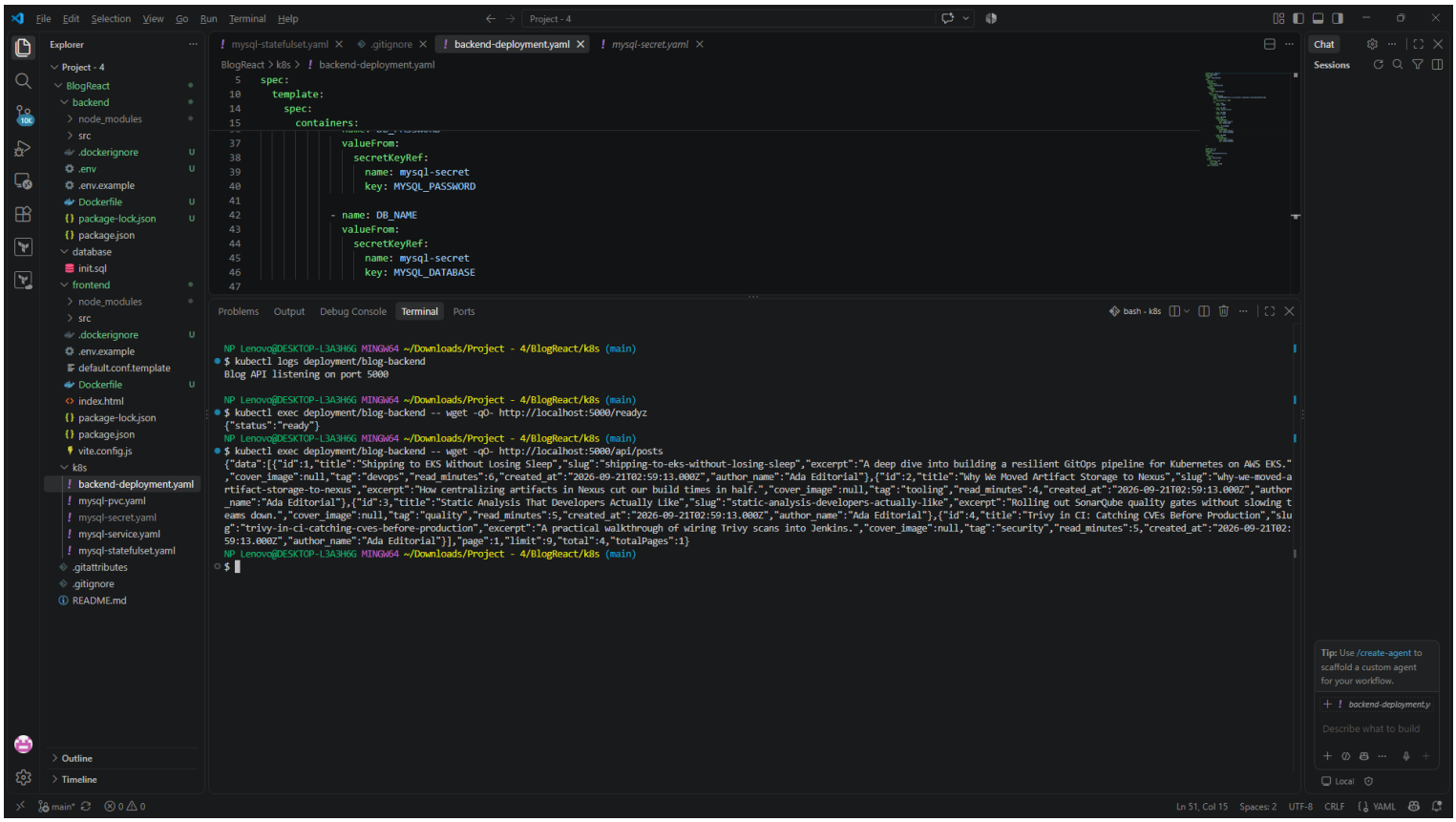


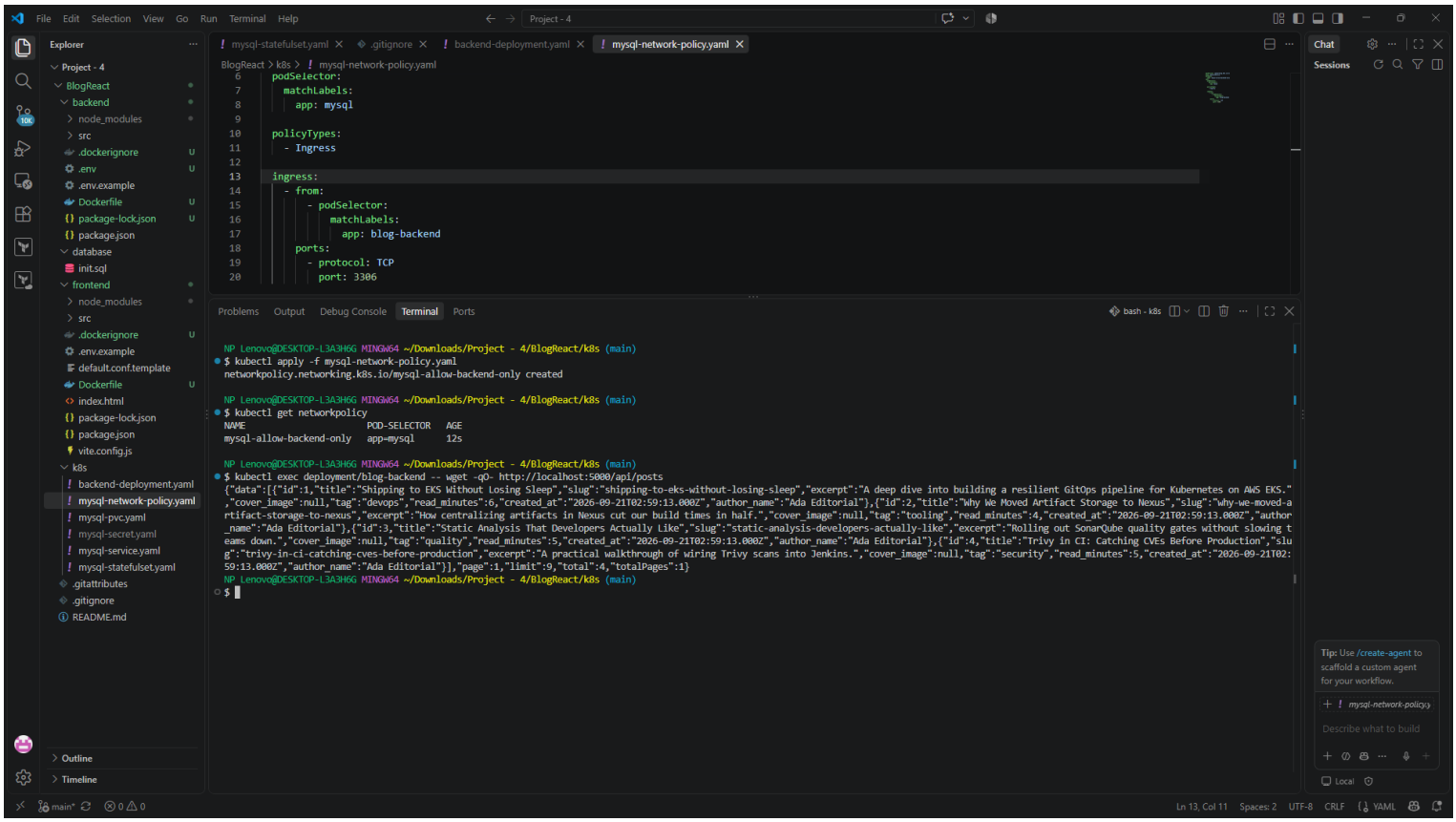

mysql-statefulset.yaml
mysql-secret.yaml
mysql-pvc.yaml
BlogReact > k8s > mysql-statefulset.yaml
MYSQL STATEFULSET YAML FILE
apiVersion: apps/v1
kind: StatefulSet
metadata:
name: mysql
spec:
serviceName: mysql-service
replicas: 1
selector:
Problems Output Debug Console Terminal Ports
bash - k8s
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact/k8s (main)
$ kubectl exec -it mysql-0 -- mysql -ubloguser -pchangeme -e "USE blogdb; SELECT COUNT(*) AS posts FROM posts;"
mysql: [Warning] Using a password on the command line interface can be insecure.
| posts |
| 4 |
$ kubectl get pods
NAME READY STATUS RESTARTS AGE
mysql-0 1/1 Running 0 8h
$ kubectl delete pod mysql-0
pod "mysql-0" deleted from default namespace
$ kubectl get pods
NAME READY STATUS RESTARTS AGE
mysql-0 1/1 Running 0 37s
$ kubectl exec -it mysql-0 -- mysql -ubloguser -pchangeme -e "USE blogdb; SELECT COUNT(*) AS posts FROM posts;"
mysql: [Warning] Using a password on the command line interface can be insecure.
| posts |
| 4 |
$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Describe what to build
Outline
Timeline
Ln 45, Col 43 Spaces: 2 UTF-8 CRLF YAML

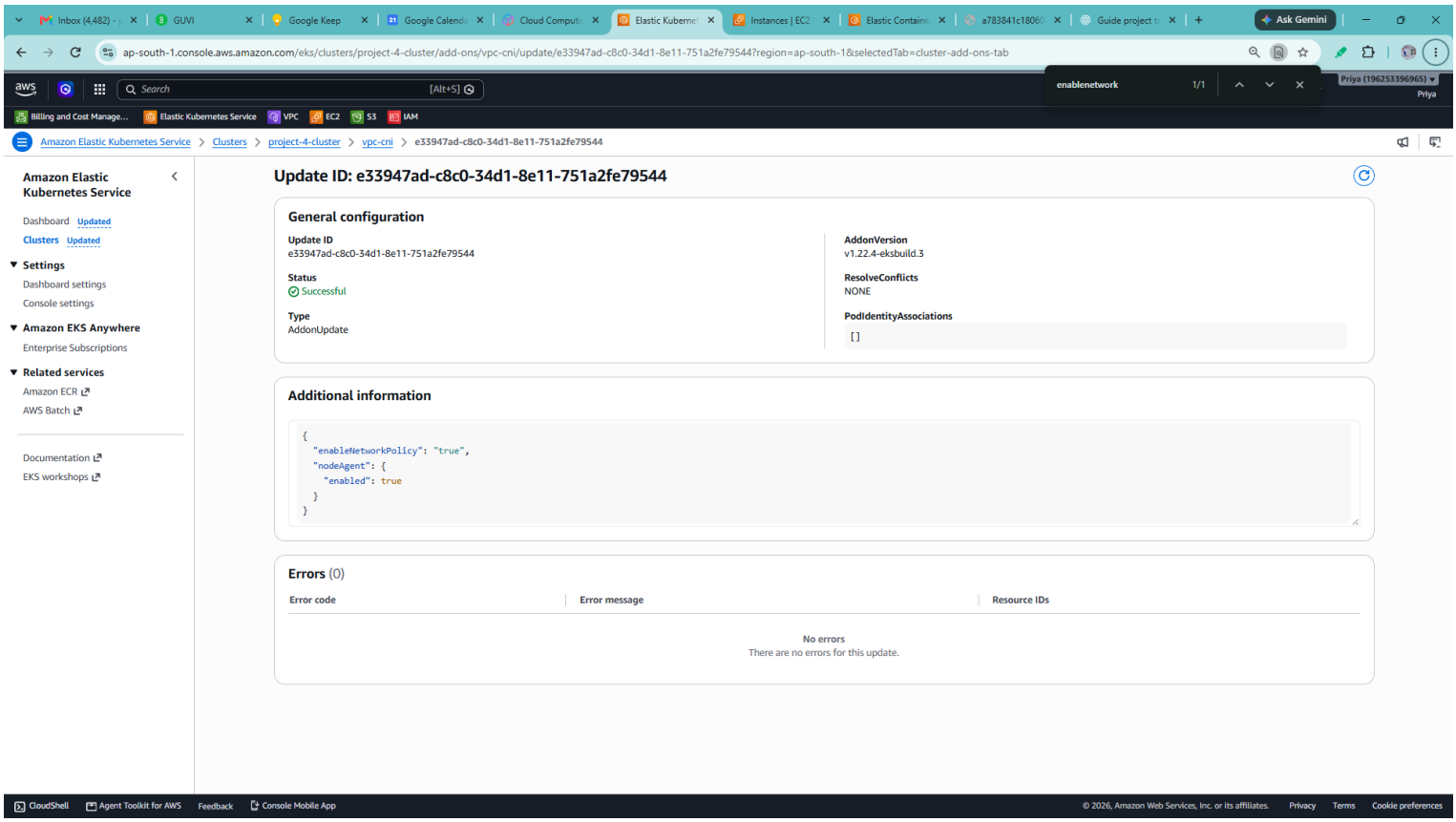

mysql-statefulset.yaml
.gitignore
backend-deployment.yaml
mysql-network-policy.yaml
mysql-pvc.yaml
BlogReact > k8s > mysql-network-policy.yaml
podSelector:
matchLabels:
app: mysql
policyTypes:
- Ingress
ingress:
- from:
- podSelector:
matchLabels:
app: blog-backend
Problems Output Debug Console Terminal Ports
bash - k8s
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact/k8s (main)
$ kubectl get pods,svc,pvc,networkpolicy,secret,statefulset
NAME READY STATUS RESTARTS AGE
pod/blog-backend-6bccfdbd85-25sbg 1/1 Running 0 67m
pod/mysql-0 1/1 Running 0 4h47m
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
service/blog-backend-service ClusterIP 10.100.198.151 <none> 5000/TCP 67m
service/kubernetes ClusterIP 10.100.0.1 <none> 443/TCP 19h
service/mysql-service ClusterIP 10.100.145.72 <none> 3306/TCP 17h
NAME STATUS VOLUME CAPACITY ACCESS MODES STORAGECLASS VOLUMEATTRIBUTESCLASS AGE
persistentvolumeclaim/mysql-pvc Bound pvc-afda25b2-0aa2-45cf-8e16-c8118cacb6ad 5Gi RWO gp2 <unset> 9h
NAME POD-SELECTOR AGE
networkpolicy.networking.k8s.io/mysql-allow-backend-only app=mysql 62m
NAME TYPE DATA AGE
secret/mysql-secret Opaque 4 17h
NAME READY AGE
statefulset.apps/mysql 1/1 17h
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Describe what to build
Outline
Timeline
Ln 13, Col 11 Spaces: 2 UTF-8 CRLF YAML

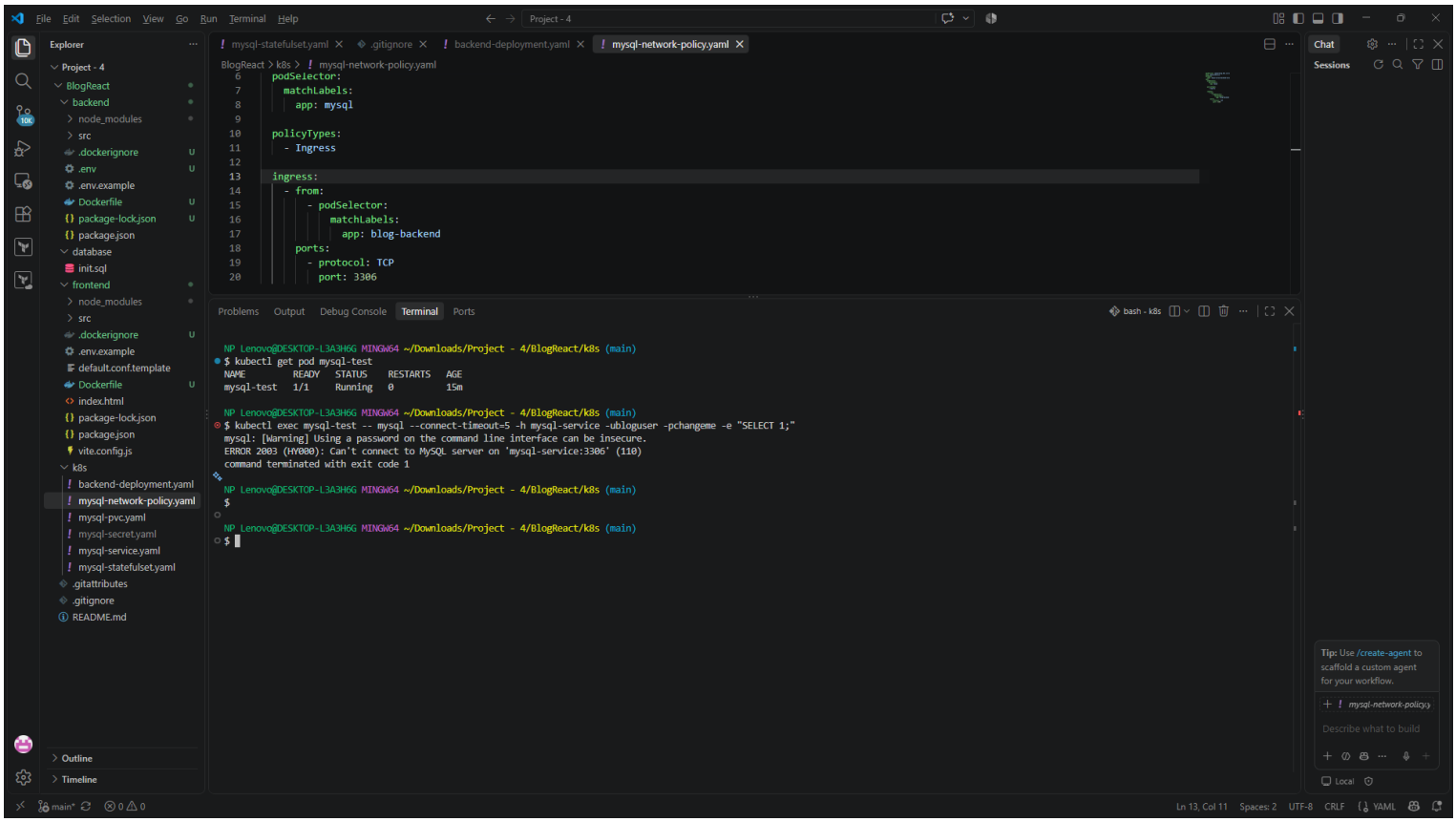

mysql-statefulset.yaml
mysql-pvc.yaml
BlogReact > k8s > mysql-statefulset.yaml
MYSQL STATEFULSET YAML FILE
apiVersion: apps/v1
kind: StatefulSet
metadata:
name: mysql
spec:
serviceName: mysql-service
replicas: 1
selector:
matchLabels:
app: mysql
template:
Problems Output Debug Console Terminal Ports
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4
$ docker tag blog-frontend:tag1 196253396965.dkr.ecr.ap-south-1.amazonaws.com/blog-frontend:tag1
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4
$ docker push 196253396965.dkr.ecr.ap-south-1.amazonaws.com/blog-frontend:tag1
The push refers to repository [196253396965.dkr.ecr.ap-south-1.amazonaws.com/blog-frontend]
ce3e77cbba4d: Pushed
e2de96513ba9: Pushed
fb08ee61b60d: Pushed
873bdb7a78a1: Pushed
44136fa355b3: Pushed
cb2f2cc9a341: Pushed
c284b1f39fec: Pushed
f408df9a891a: Pushed
c0ac6b3b9c77: Pushed
90de1d15efe2: Pushed
6c3acaae0f11: Pushed
11c11a61c5fa: Pushed
c4cfe143e3e8: Pushed
tag1: digest: sha256:0e91f285afa8aacff1494507d8bda78c4ed0635fbefd84418ab5d925eec92a46 size: 856
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4
$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Describe what to build
Outline
Timeline
Ln 45, Col 43 Spaces: 2 UTF-8 CRLF YAML


mysql-statefulset.yaml
mysql-pvc.yaml
BlogReact > k8s > mysql-statefulset.yaml
MYSQL STATEFULSET YAML FILE
apiVersion: apps/v1
kind: StatefulSet
metadata:
name: mysql
spec:
serviceName: mysql-service
replicas: 1
selector:
matchLabels:
app: mysql
template:
Problems Output Debug Console Terminal Ports
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4
$ aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin 196253396965.dkr.ecr.ap-south-1.amazonaws.com
Login Succeeded
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4
$ docker tag blog-backend:tag1 196253396965.dkr.ecr.ap-south-1.amazonaws.com/blog-backend:tag1
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4
$ docker images | grep blog-backend
196253396965.dkr.ecr.ap-south-1.amazonaws.com/blog-backend:tag1 cb02d59a45ce 271MB 65.6MB U
blog-backend:tag1 cb02d59a45ce 271MB 65.6MB U
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4
$ docker push 196253396965.dkr.ecr.ap-south-1.amazonaws.com/blog-backend:tag1
The push refers to repository [196253396965.dkr.ecr.ap-south-1.amazonaws.com/blog-backend]
37aab5298093: Pushed
44136fa355b3: Pushed
3c504d5a797b: Pushed
e2de96513ba9: Pushed
02f40d53b06f: Pushed
7ce7dcbafb7f: Pushed
55228d2b7894: Pushed
25f3169fa5b4: Pushed
e8bcfe78227a: Pushed
17c5833091ab: Pushed
8e294ccb561e: Pushed
tag1: digest: sha256:cb02d59a45ce718575a577c3f912da0eb78406f5c2acfbfc21eebdddad9665a0 size: 856
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4
$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Describe what to build
Outline
Timeline
Ln 45, Col 43 Spaces: 2 UTF-8 CRLF YAML


backend-deployment.yaml
BlogReact > k8s > backend-deployment.yaml
valueFrom:
secretKeyRef:
name: mysql-secret
key: MYSQL_PASSWORD
- name: DB_NAME
valueFrom:
secretKeyRef:
name: mysql-secret
key: MYSQL_DATABASE
Terminal
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact/k8s (main)
$ kubectl logs deployment/blog-backend
Blog API listening on port 5000
$ kubectl exec deployment/blog-backend -- wget -qO- http://localhost:5000/readyz
{"status":"ready"}
$ kubectl exec deployment/blog-backend -- wget -qO- http://localhost:5000/api/posts


mysql-network-policy.yaml
BlogReact > k8s > mysql-network-policy.yaml
podSelector:
matchLabels:
app: mysql
policyTypes:
- Ingress
ingress:
- from:
- podSelector:
matchLabels:
app: blog-backend
ports:
- protocol: TCP
port: 3306
Terminal
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact/k8s (main)
$ kubectl apply -f mysql-network-policy.yaml
networkpolicy.networking.k8s.io/mysql-allow-backend-only created
$ kubectl get networkpolicy
NAME POD-SELECTOR AGE
mysql-allow-backend-only app=mysql 12s
$ kubectl exec deployment/blog-backend -- wget -qO- http://localhost:5000/api/posts


Amazon Elastic Kubernetes Service
Update ID: e33947ad-c8c0-34d1-8e11-751a2fe79544
General configuration
Update ID
e33947ad-c8c0-34d1-8e11-751a2fe79544
Status
Successful
Type
AddonUpdate
AddonVersion
v1.22.4-eksbuild.3
ResolveConflicts
NONE
PodIdentityAssociations
[]
Additional information
{
"enableNetworkPolicy": "true",
"nodeAgent": {
"enabled": true
}
}
Errors (0)
Error code
Error message
Resource IDs
No errors
There are no errors for this update.


mysql-network-policy.yaml
podSelector:
matchLabels:
app: mysql
policyTypes:
- Ingress
ingress:
- from:
- podSelector:
matchLabels:
app: blog-backend
ports:
- protocol: TCP
port: 3306
$ kubectl get pod mysql-test
NAME READY STATUS RESTARTS AGE
mysql-test 1/1 Running 0 15m
$ kubectl exec mysql-test -- mysql --connect-timeout=5 -h mysql-service -ubloguser -pchangeme -e "SELECT 1;"
mysql: [Warning] Using a password on the command line interface can be insecure.
ERROR 2003 (HY000): Can't connect to MySQL server on 'mysql-service:3306' (110)
command terminated with exit code 1

Task 4 : ECR Repository

Overview

Amazon Elastic Container Registry (ECR) was used as the container image repository for the Frontend and Backend applications. The Docker images were versioned, pushed to ECR, and the same versioned images were deployed to Amazon EKS.

ECR Configuration

- ECR Registry: 196253396965.dkr.ecr.ap-south-1.amazonaws.com
- Frontend Repository: blog-frontend
- Backend Repository: blog-backend
- Image Version: tag1

Execution

The Frontend and Backend Docker images were built and tagged with version tag1. Docker was authenticated with Amazon ECR and both images were pushed to their respective ECR repositories.

The Backend was deployed to EKS using the versioned ECR image: `196253396965.dkr.ecr.ap-south-1.amazonaws.com/blog-backend:tag1`

The Frontend was deployed to EKS using the versioned ECR image: `196253396965.dkr.ecr.ap-south-1.amazonaws.com/blog-frontend:tag1`

A Kubernetes LoadBalancer Service was configured for the Frontend to provide external application access.

Verification

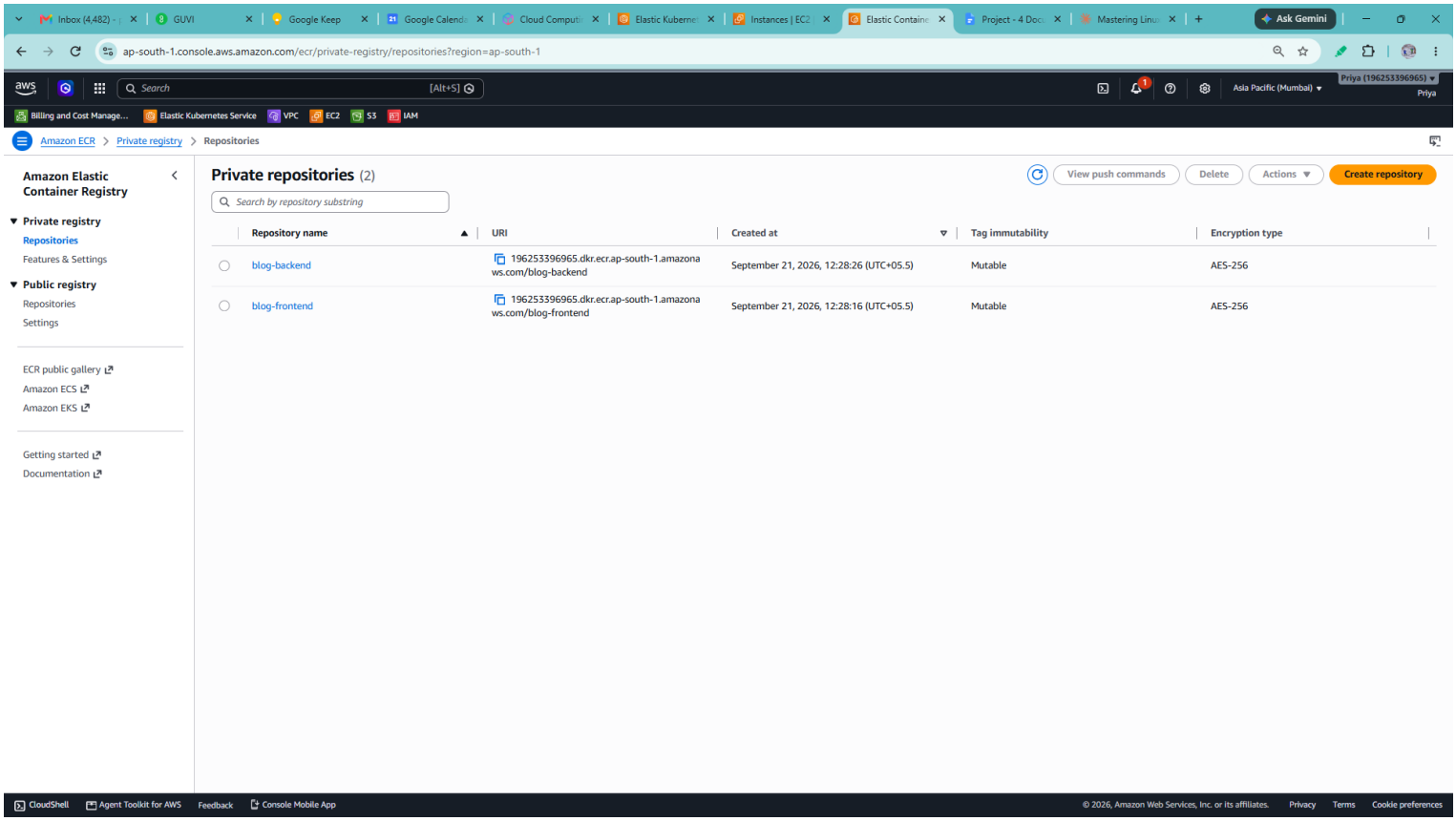
The following verification was successfully completed:

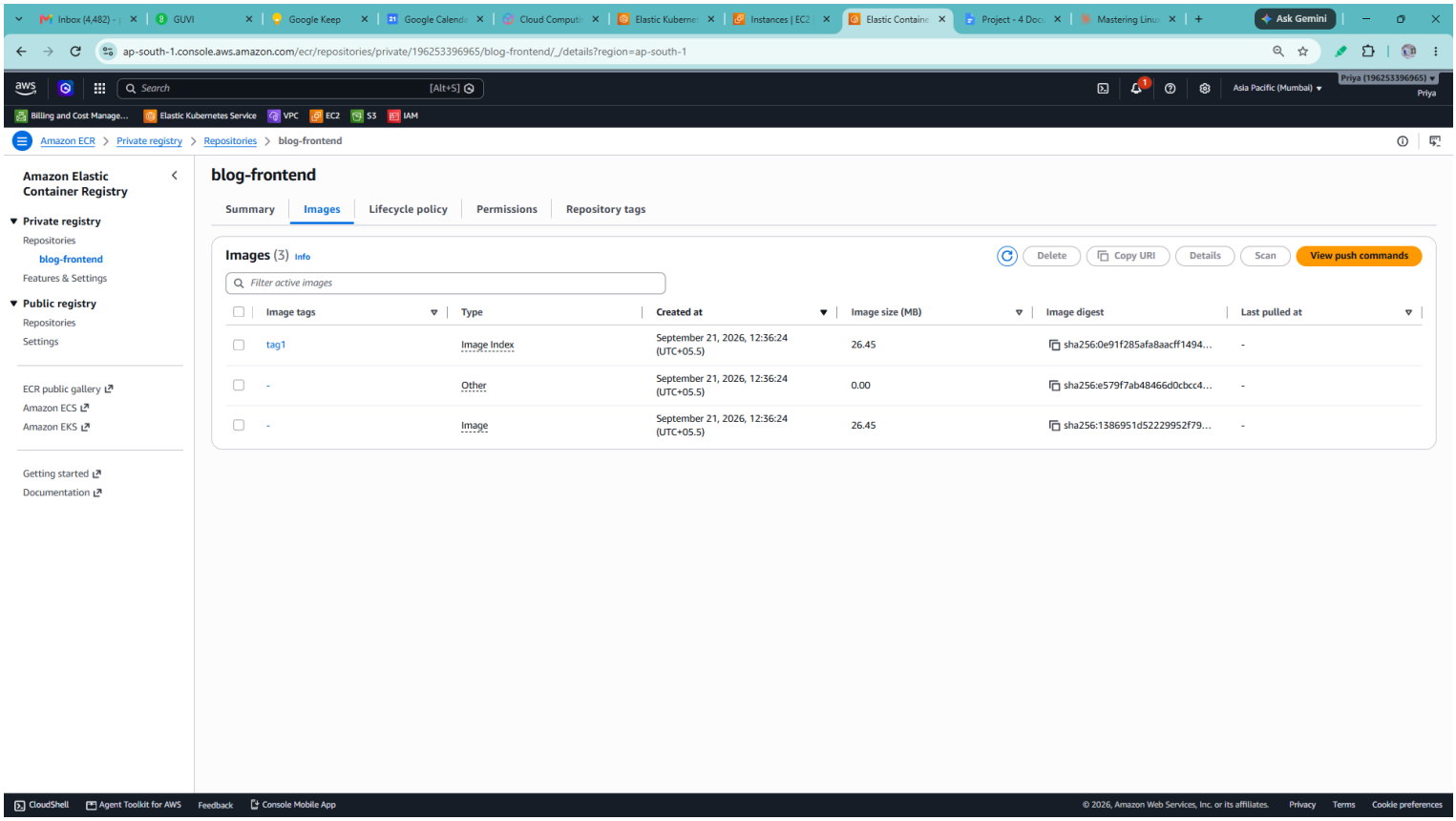
- blog-frontend:tag1 was verified in the Frontend ECR repository.
- blog-backend:tag1 was verified in the Backend ECR repository.
- Backend Deployment was running successfully in EKS.
- Frontend Deployment was running successfully in EKS.
- Backend Pod and Frontend Pod were in **Running** state.
- Frontend Service was exposed through an AWS LoadBalancer.
- The application was successfully accessed through the EKS LoadBalancer URL.

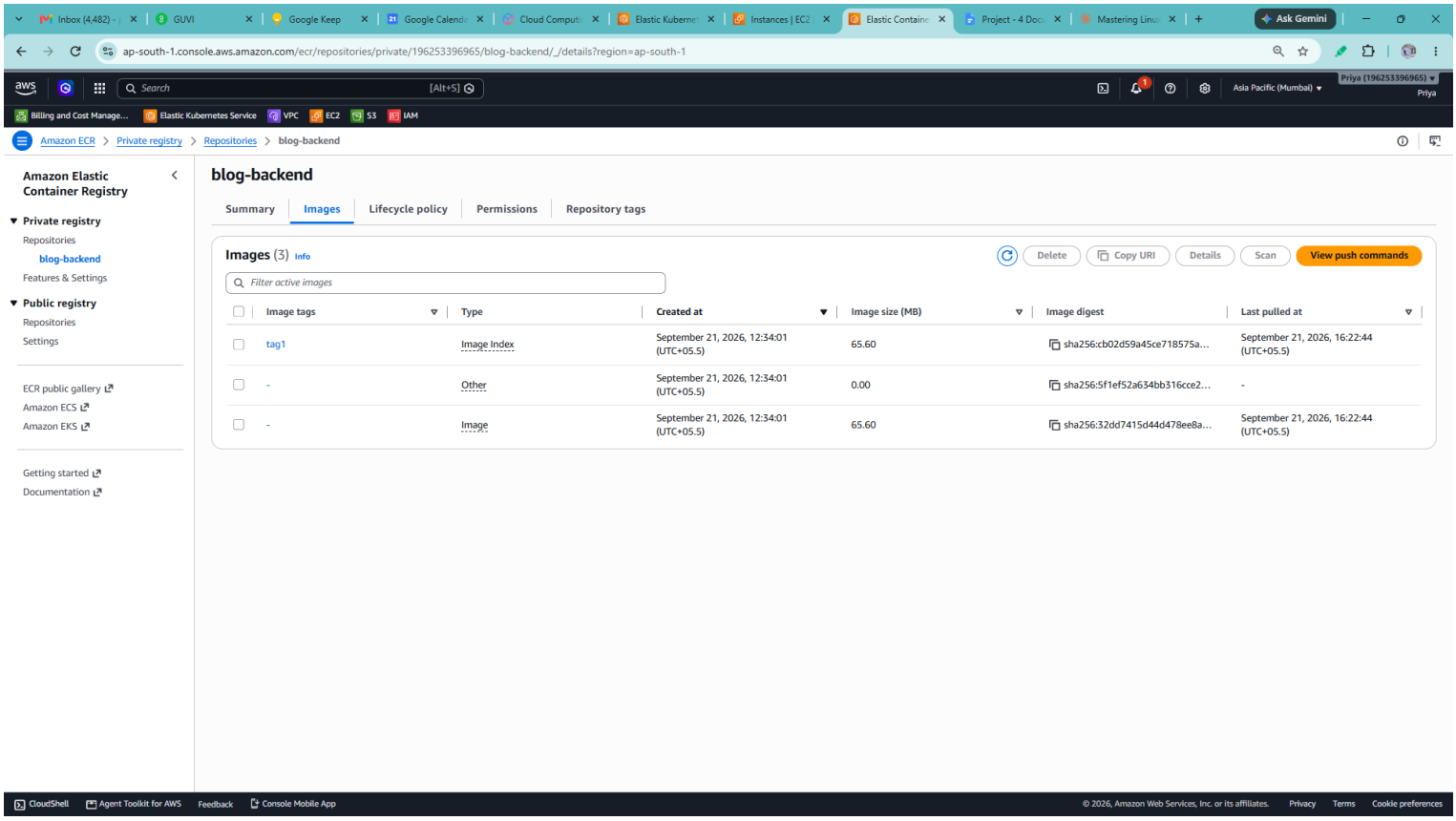
- The application displayed the four blog posts successfully, confirming communication between the Frontend, Backend, and Database.

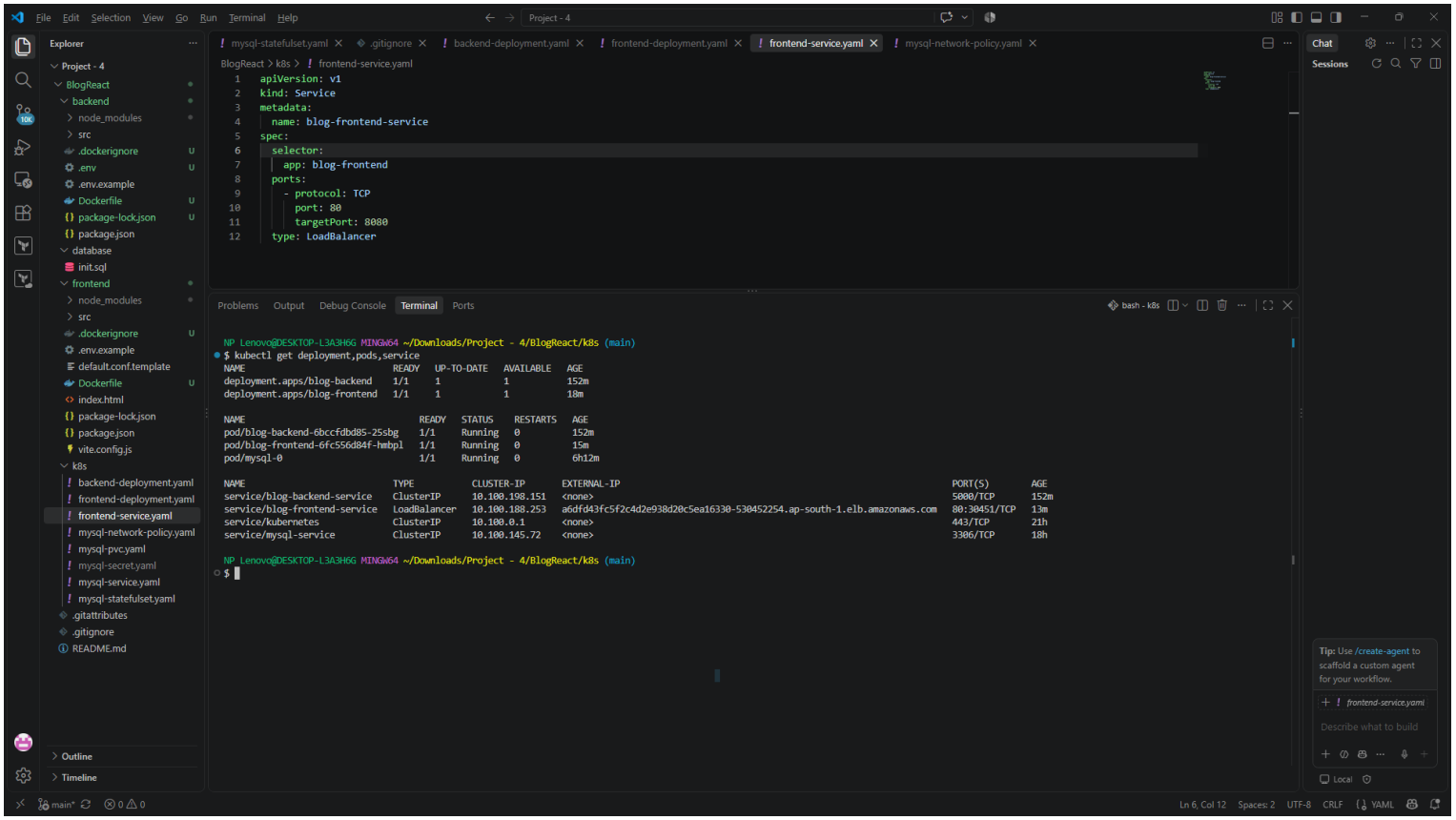
Result

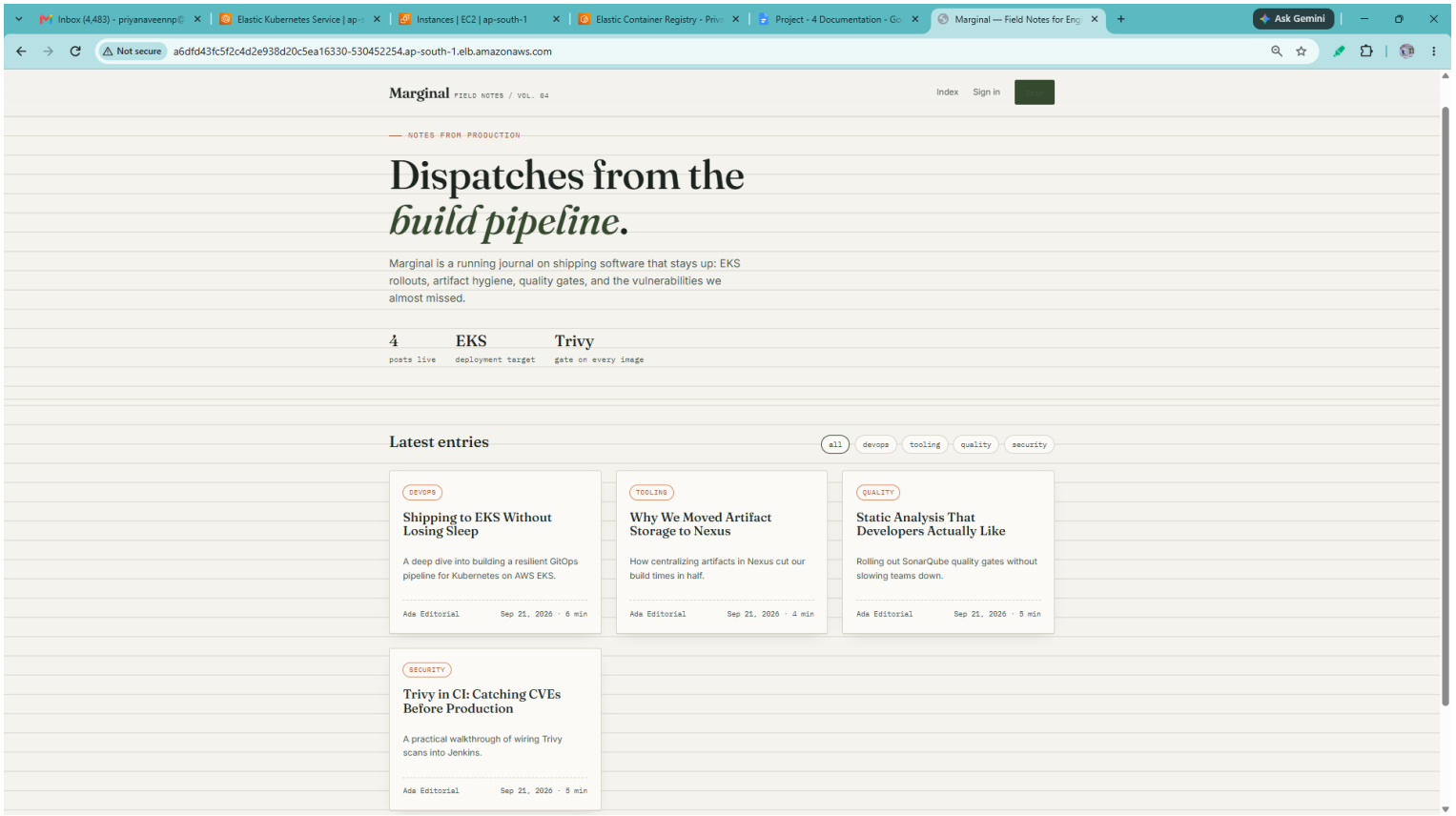
The versioned Frontend and Backend Docker images were successfully built, tagged, pushed to Amazon ECR, and deployed to Amazon EKS. The application was externally accessible through the AWS LoadBalancer and successfully retrieved and displayed database-backed application data.




ap-south-1.console.aws.amazon.com/ecr/repositories/private/196253396965/blog-frontend/_/details?region=ap-south-1
Amazon ECR > Private registry > Repositories > blog-frontend
Amazon Elastic Container Registry
Private registry
Repositories
blog-frontend
Features & Settings
Public registry
Repositories
Settings
ECR public gallery
Amazon ECS
Amazon EKS
Getting started
Documentation
blog-frontend
Summary
Images
Lifecycle policy
Permissions
Repository tags
Images (3) Info
Delete
Copy URI
Details
Scan
View push commands
Filter active images
Image tags
Type
Created at
Image size (MB)
Image digest
Last pulled at
tag1
Image Index
September 21, 2026, 12:36:24 (UTC+05.5)
26.45
sha256:0e91f285afa8aacff1494...
-
-
Other
September 21, 2026, 12:36:24 (UTC+05.5)
0.00
sha256:e579f7ab48466d0cbcc4...
-
-
Image
September 21, 2026, 12:36:24 (UTC+05.5)
26.45
sha256:1386951d52229952f79...
-
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


ap-south-1.console.aws.amazon.com/ecr/repositories/private/196253396965/blog-backend/_/details?region=ap-south-1
Amazon ECR > Private registry > Repositories > blog-backend
Amazon Elastic Container Registry
Private registry
Repositories
blog-backend
Features & Settings
Public registry
Repositories
Settings
ECR public gallery
Amazon ECS
Amazon EKS
Getting started
Documentation
blog-backend
Summary
Images
Lifecycle policy
Permissions
Repository tags
Images (3) Info
Delete
Copy URI
Details
Scan
View push commands
Filter active images
Image tags
Type
Created at
Image size (MB)
Image digest
Last pulled at
tag1
Image Index
September 21, 2026, 12:34:01 (UTC+05.5)
65.60
sha256:cb02d59a45ce718575a...
September 21, 2026, 16:22:44 (UTC+05.5)
-
Other
September 21, 2026, 12:34:01 (UTC+05.5)
0.00
sha256:5f1ef52a634bb316cce2...
-
-
Image
September 21, 2026, 12:34:01 (UTC+05.5)
65.60
sha256:32dd7415d44d478ee8a...
September 21, 2026, 16:22:44 (UTC+05.5)
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


frontend-service.yaml
apiVersion: v1
kind: Service
metadata:
name: blog-frontend-service
spec:
selector:
app: blog-frontend
ports:
- protocol: TCP
port: 80
targetPort: 8080
type: LoadBalancer
$ kubectl get deployment,pods,service
NAME READY UP-TO-DATE AVAILABLE AGE
deployment.apps/blog-backend 1/1 1 1 152m
deployment.apps/blog-frontend 1/1 1 1 18m
NAME READY STATUS RESTARTS AGE
pod/blog-backend-6bccfdbd85-25sbg 1/1 Running 0 152m
pod/blog-frontend-6fc556d84f-hmbpl 1/1 Running 0 15m
pod/mysql-0 1/1 Running 0 6h12m
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
service/blog-backend-service ClusterIP 10.100.198.151 <none> 5000/TCP 152m
service/blog-frontend-service LoadBalancer 10.100.188.253 a6dfd43fc5f2c4d2e938d20c5ea16330-530452254.ap-south-1.elb.amazonaws.com 80:30451/TCP 13m
service/kubernetes ClusterIP 10.100.0.1 <none> 443/TCP 21h
service/mysql-service ClusterIP 10.100.145.72 <none> 3306/TCP 18h


Not secure
a6dfd43fc5f2c4d2e938d20c5ea16330-530452254.ap-south-1.elb.amazonaws.com
Marginal FIELD NOTES / VOL. 04
Index
Sign in
NOTES FROM PRODUCTION
Dispatches from the build pipeline.
Marginal is a running journal on shipping software that stays up: EKS rollouts, artifact hygiene, quality gates, and the vulnerabilities we almost missed.
4 posts live
EKS deployment target
Trivy gate on every image
Latest entries
all
devops
tooling
quality
security
DEVOPS
Shipping to EKS Without Losing Sleep
A deep dive into building a resilient GitOps pipeline for Kubernetes on AWS EKS.
Ada Editorial
Sep 21, 2026 · 6 min
TOOLING
Why We Moved Artifact Storage to Nexus
How centralizing artifacts in Nexus cut our build times in half.
Ada Editorial
Sep 21, 2026 · 4 min
QUALITY
Static Analysis That Developers Actually Like
Rolling out SonarQube quality gates without slowing teams down.
Ada Editorial
Sep 21, 2026 · 5 min
SECURITY
Trivy in CI: Catching CVEs Before Production
A practical walkthrough of wiring Trivy scans into Jenkins.
Ada Editorial
Sep 21, 2026 · 5 min

Task 5 : SonarQube

Overview

SonarQube was integrated with the Jenkins CI/CD pipeline to perform automated code-quality analysis of the Frontend and Backend applications and enforce a Quality Gate before the pipeline proceeds.

SonarQube Configuration

- Project Name: Blog Application
- Project Key: Blog-Application
- Main Branch: main
- Analysis Method: Jenkins
- SonarScanner Version: 8.1.0.6389
- Source Directories: frontend/src, backend/src

Execution

The SonarQube Scanner was configured in Jenkins and a project-scoped SonarQube Analysis Token was securely stored in Jenkins Credentials.

The pipeline was configured to execute:

`Checkout → Build & Test → SonarQube Analysis → Quality Gate`

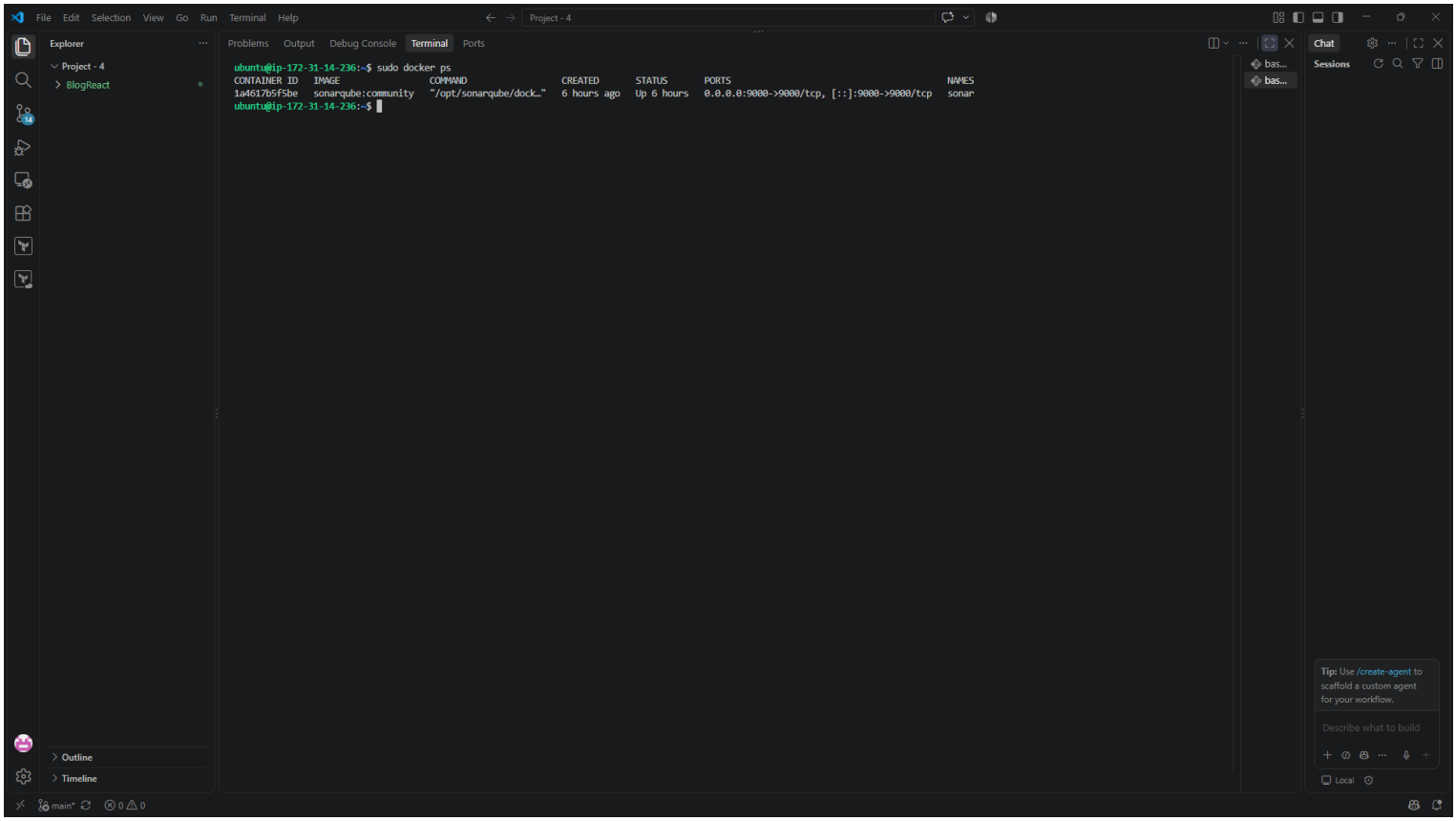
SonarQube analysis was completed successfully and the analysis report was uploaded. The Quality Gate was enforced using `abortPipeline: true` so that the pipeline stops if the configured Quality Gate fails. A SonarQube webhook was configured to return the analysis status to Jenkins.

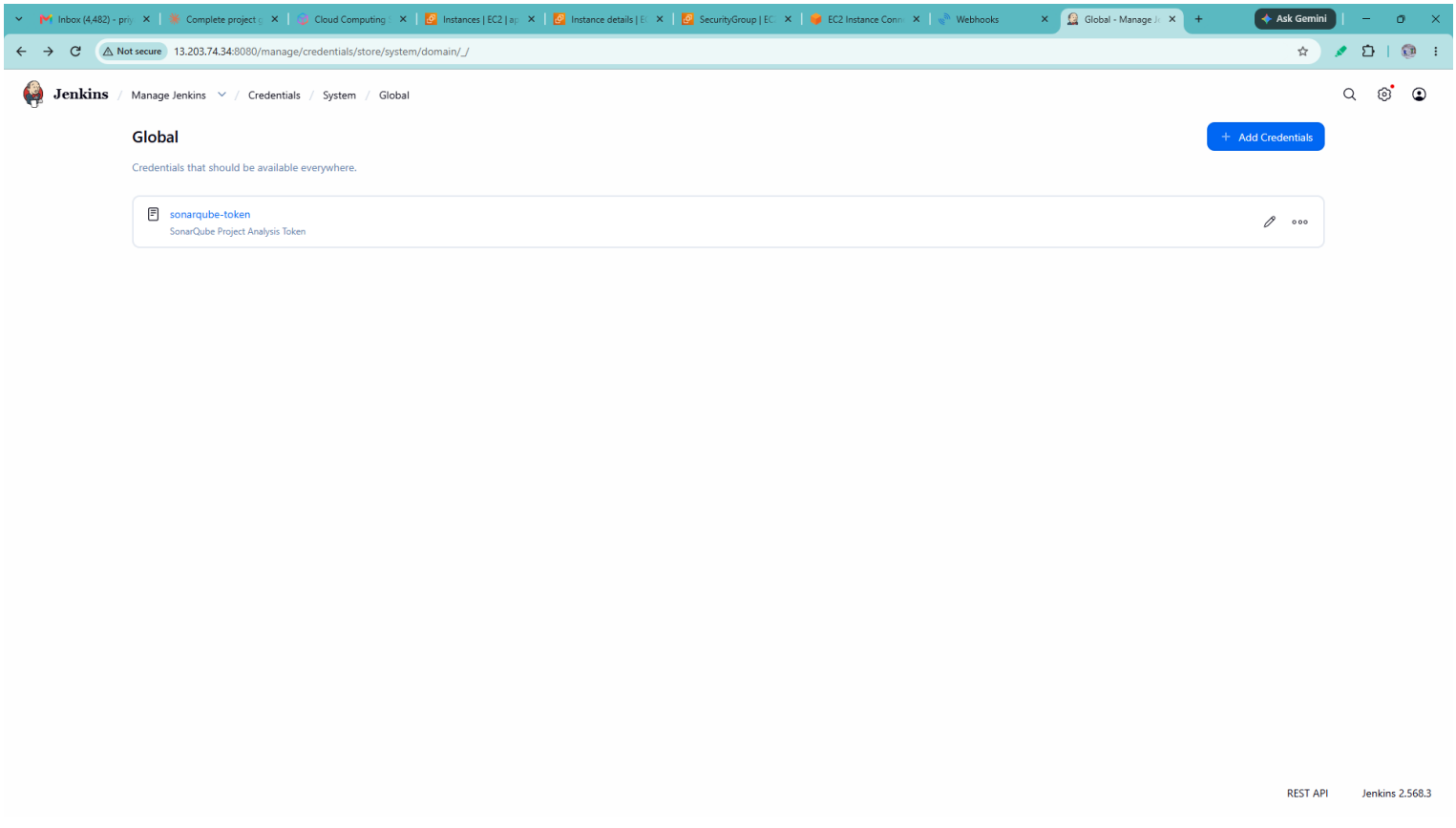
Verification

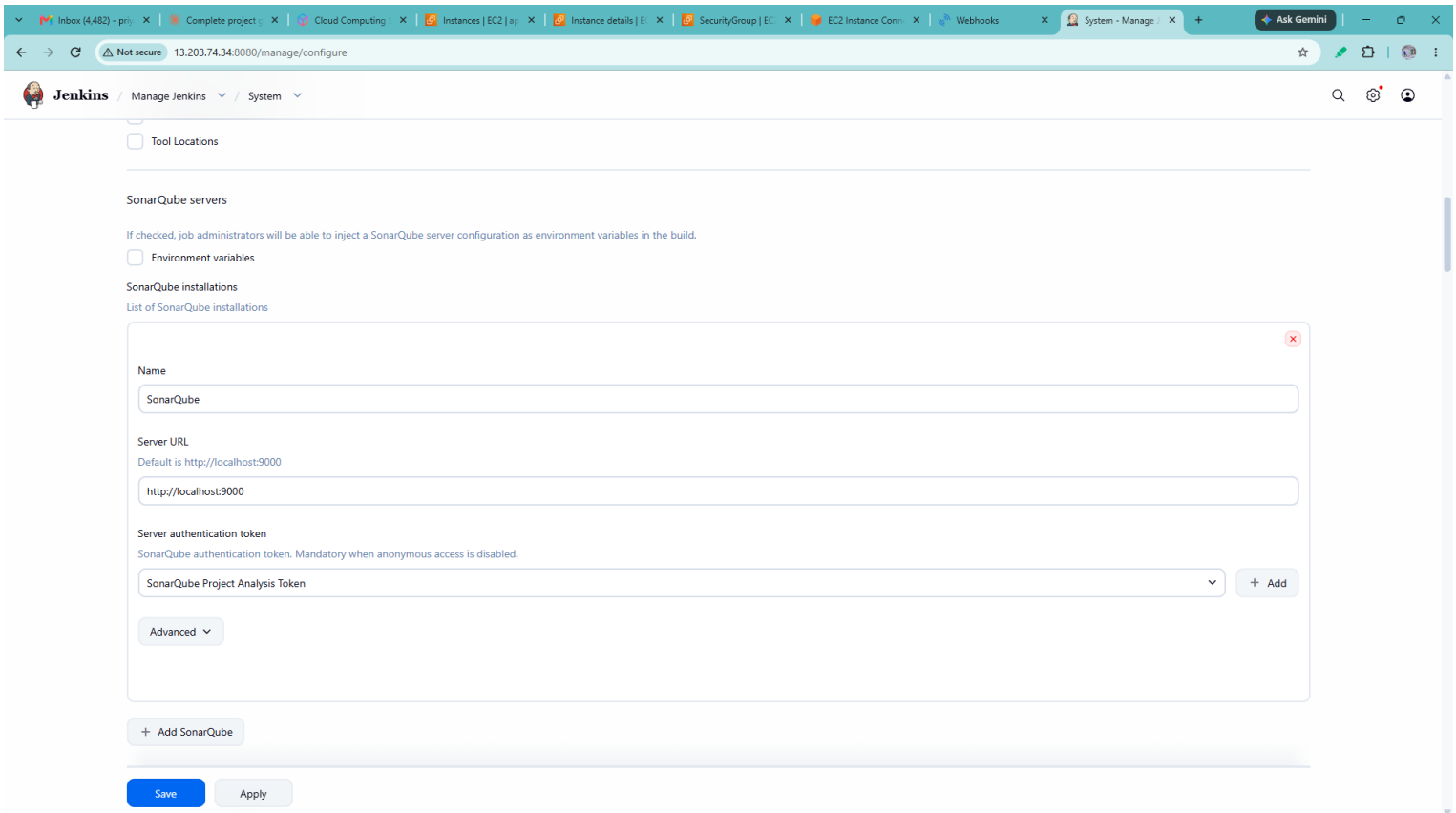
The SonarQube project displayed Quality Gate: Passed, and the Jenkins Quality Gate stage returned OK and Jenkins Build #1 completed successfully with SUCCESS.

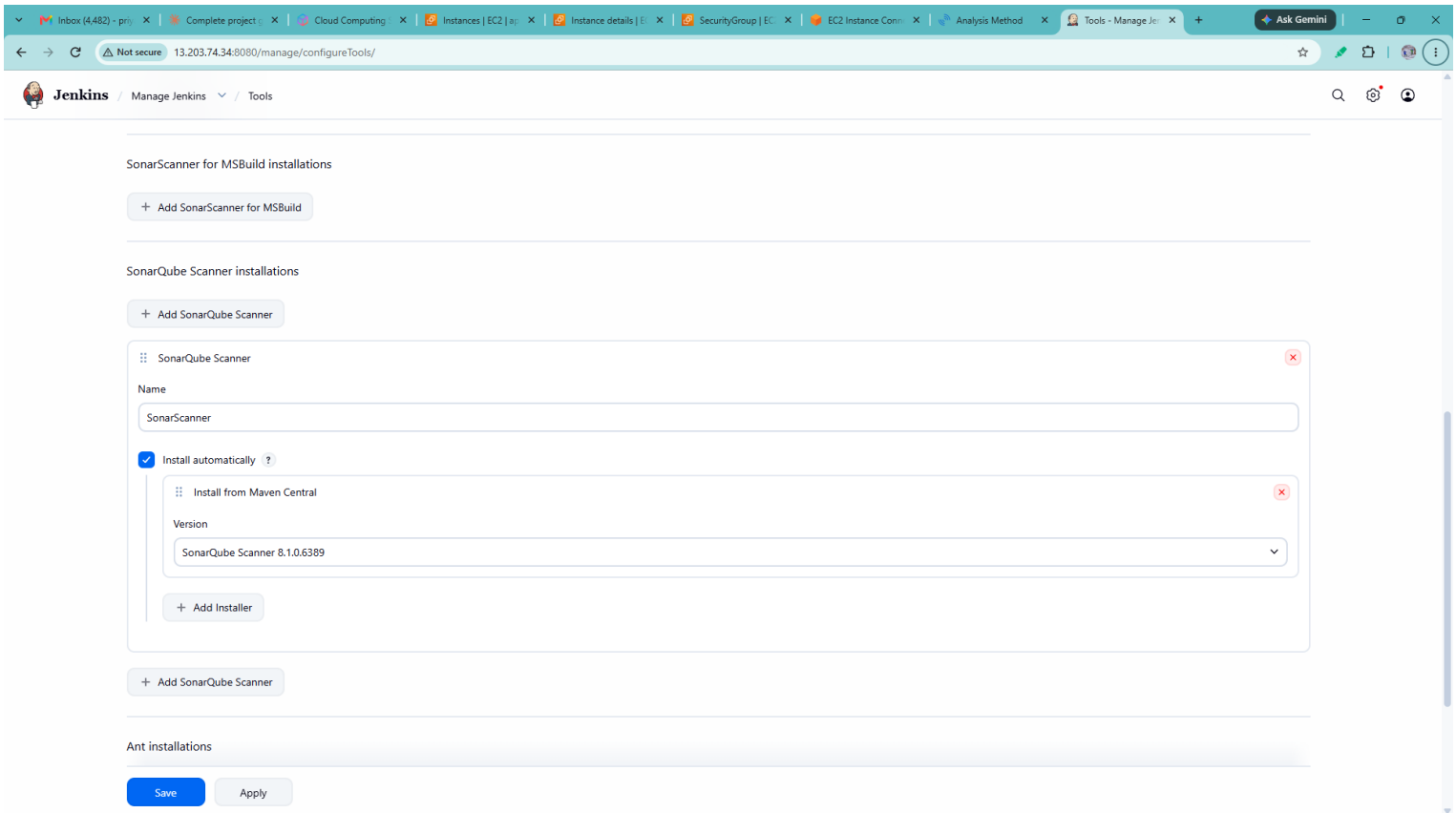
Result

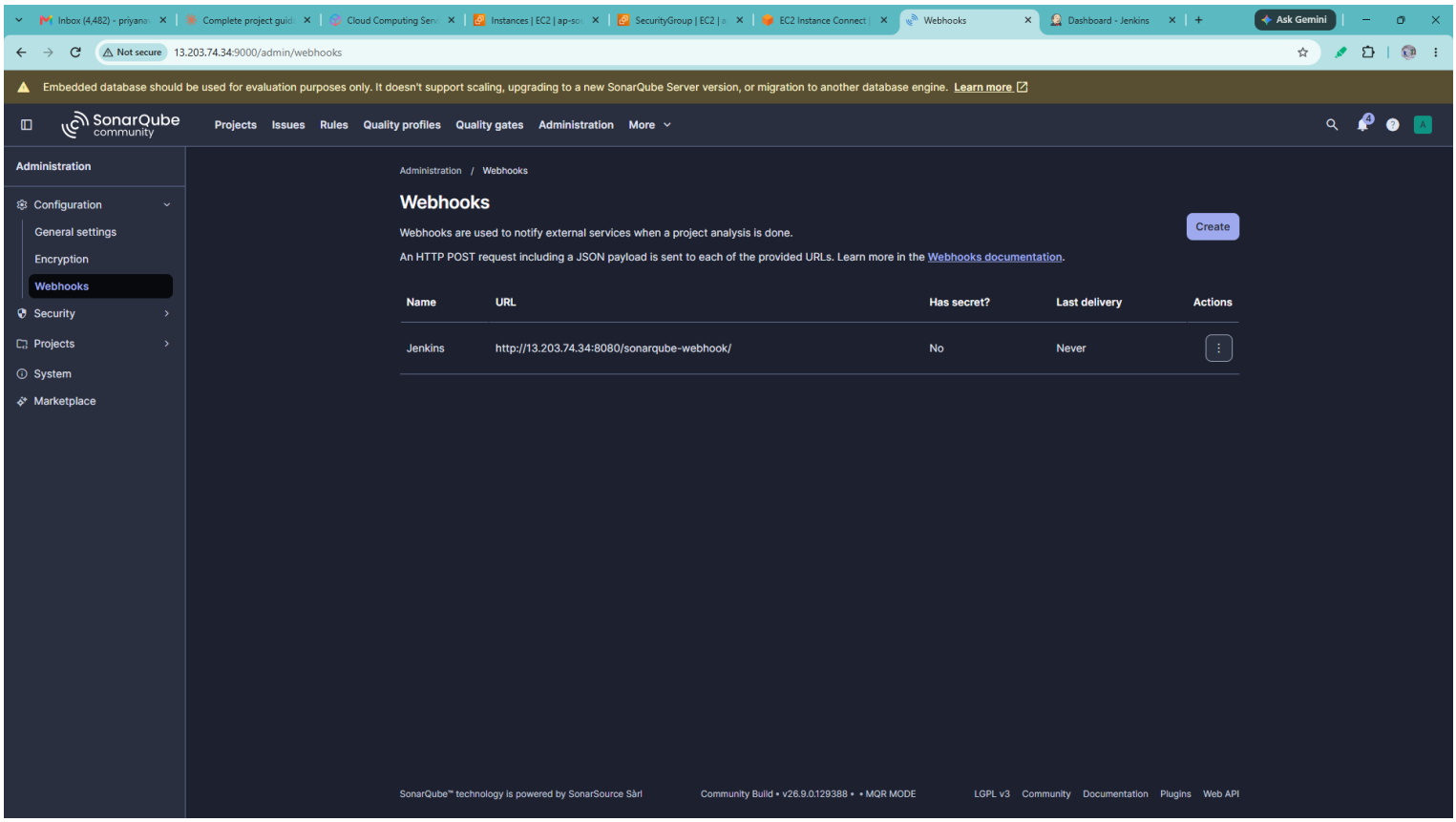
SonarQube code-quality analysis and Quality Gate enforcement were successfully integrated with Jenkins. The application passed the configured Quality Gate and the pipeline was successfully validated.

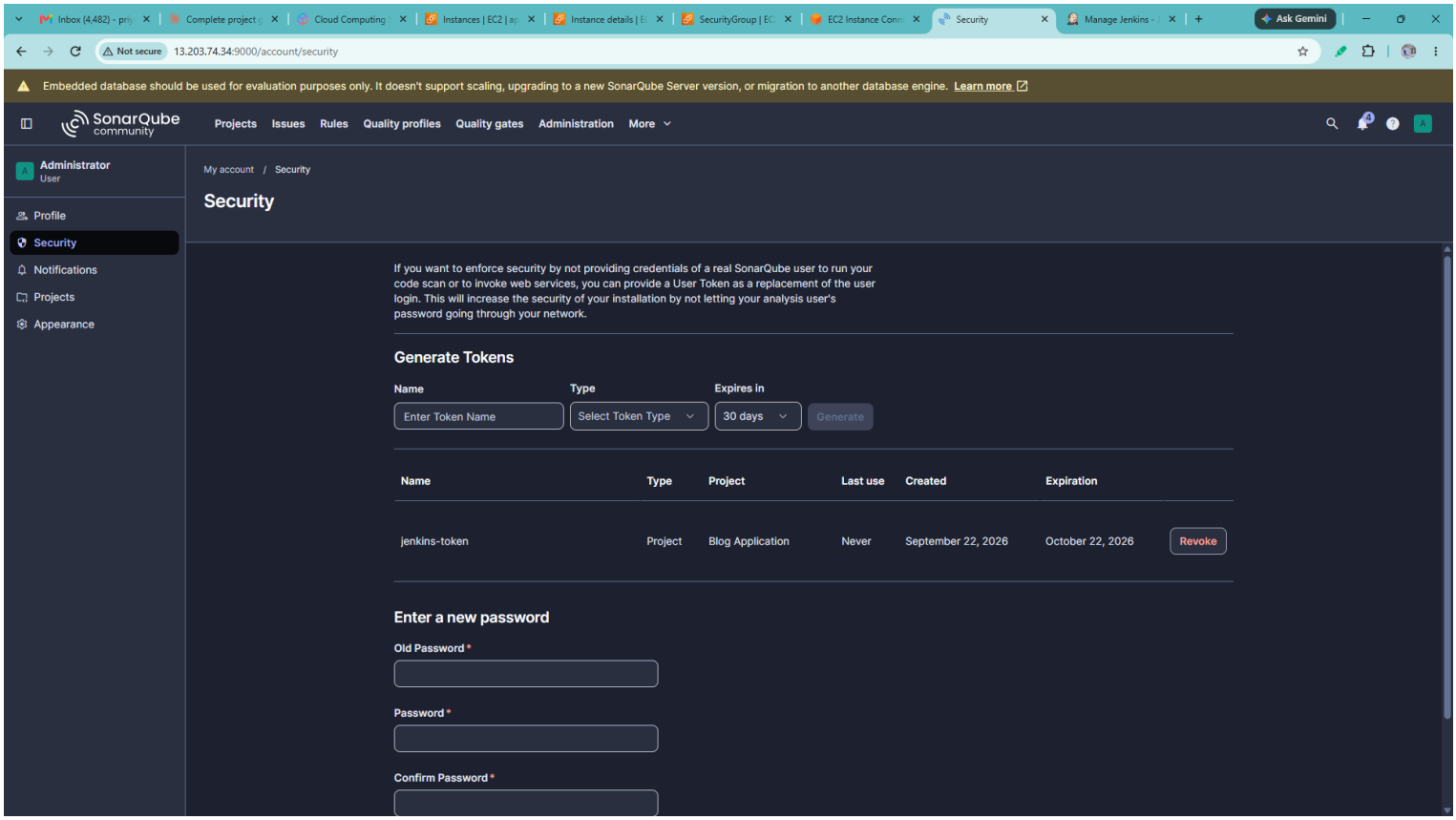

File Edit Selection View Go Run Terminal Help
Project - 4
Explorer
Project - 4
BlogReact
Problems Output Debug Console Terminal Ports
ubuntu@ip-172-31-14-236:~$ sudo docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
1a4617b5f5be sonarqube:community "/opt/sonarqube/dock…" 6 hours ago Up 6 hours 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp sonar
ubuntu@ip-172-31-14-236:~$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Describe what to build
Outline
Timeline

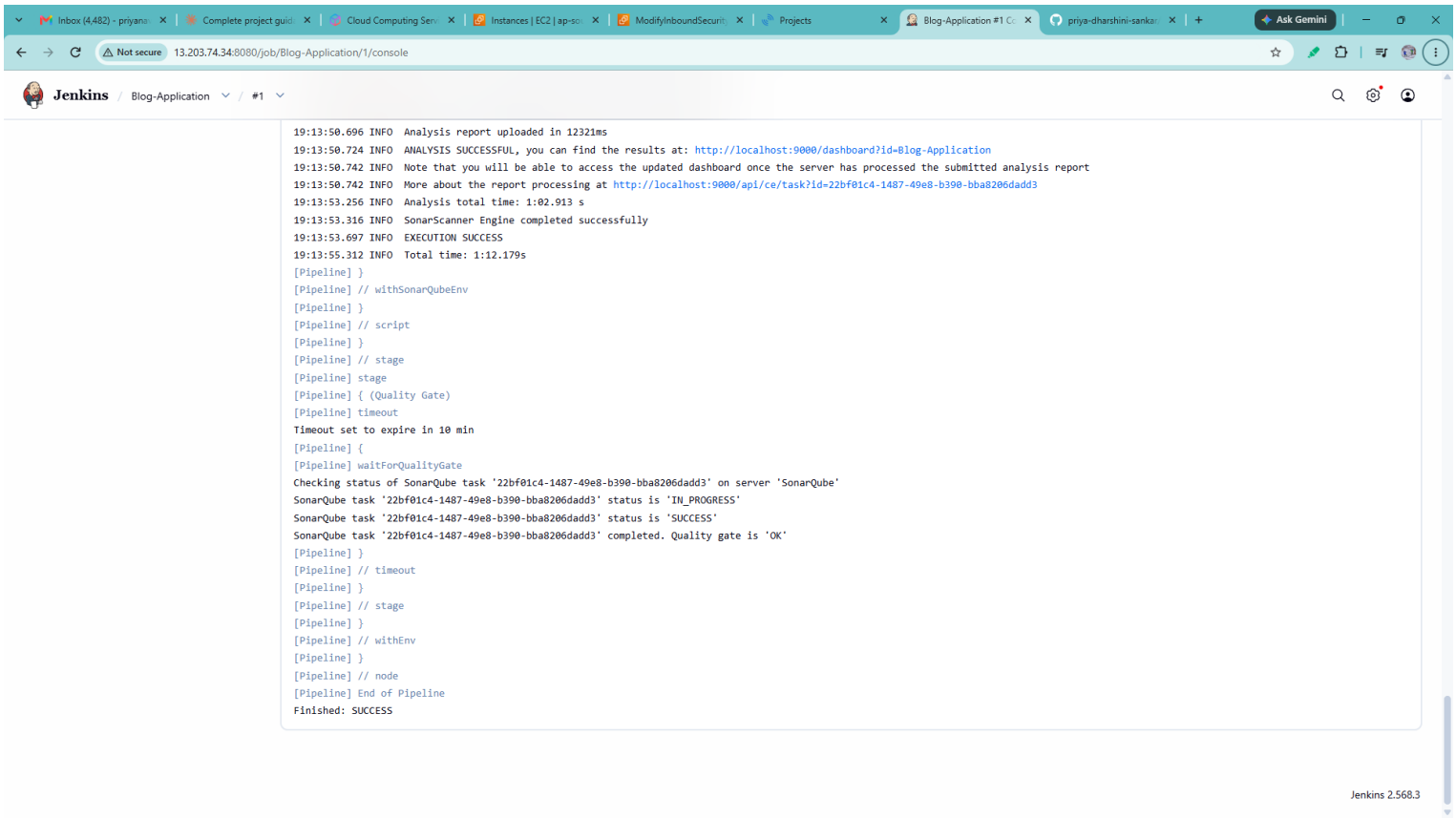

Not secure 13.203.74.34:8080/manage/credentials/store/system/domain/_/
Ask Gemini
Jenkins / Manage Jenkins / Credentials / System / Global
Global
Add Credentials
Credentials that should be available everywhere.
sonarqube-token
SonarQube Project Analysis Token
REST API
Jenkins 2.568.3

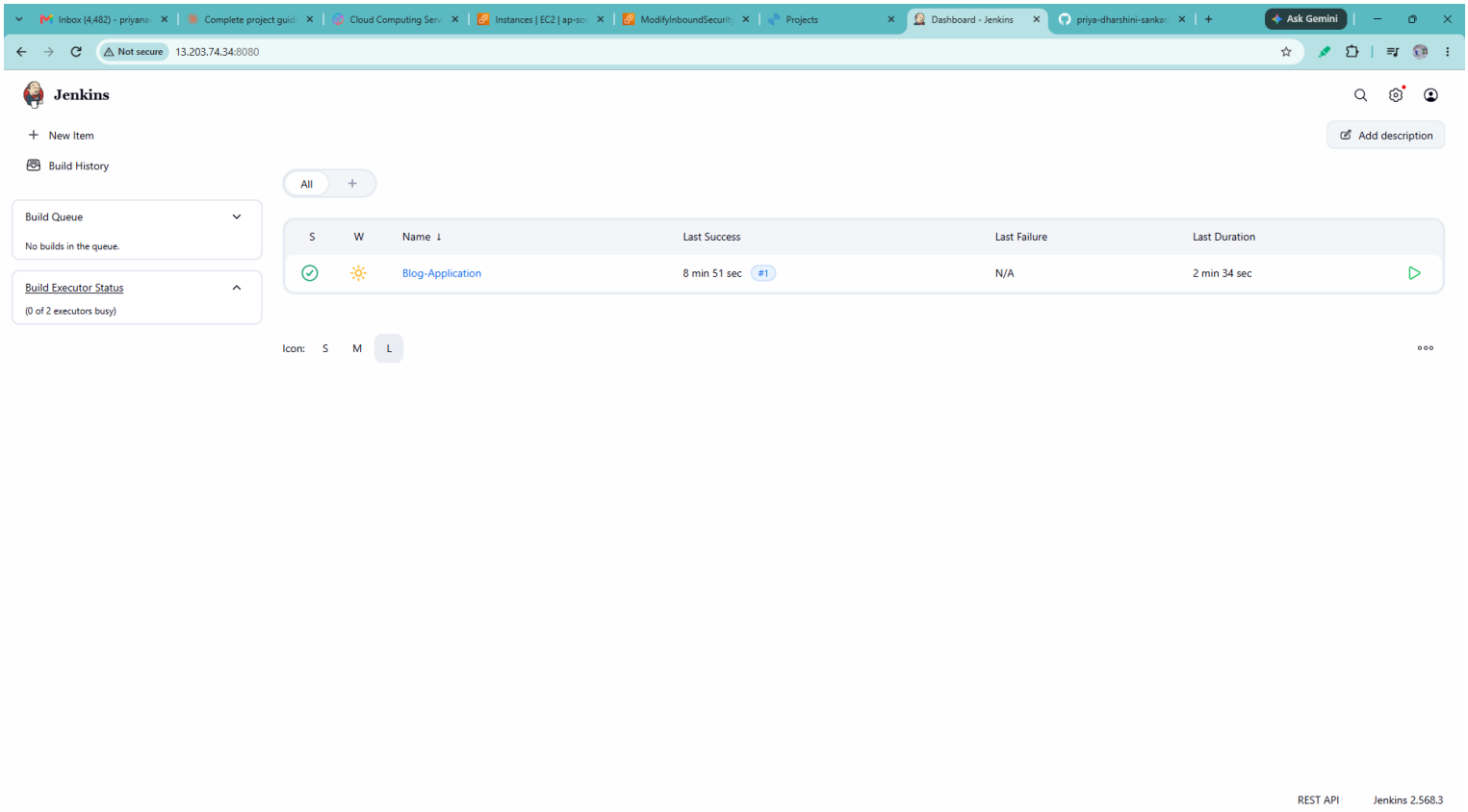

Jenkins / Manage Jenkins / System
Not secure 13.203.74.34:8080/manage/configure
Tool Locations
SonarQube servers
If checked, job administrators will be able to inject a SonarQube server configuration as environment variables in the build.
Environment variables
SonarQube installations
List of SonarQube installations
Name
SonarQube
Server URL
Default is http://localhost:9000
http://localhost:9000
Server authentication token
SonarQube authentication token. Mandatory when anonymous access is disabled.
SonarQube Project Analysis Token
Add
Advanced
Add SonarQube
Save
Apply

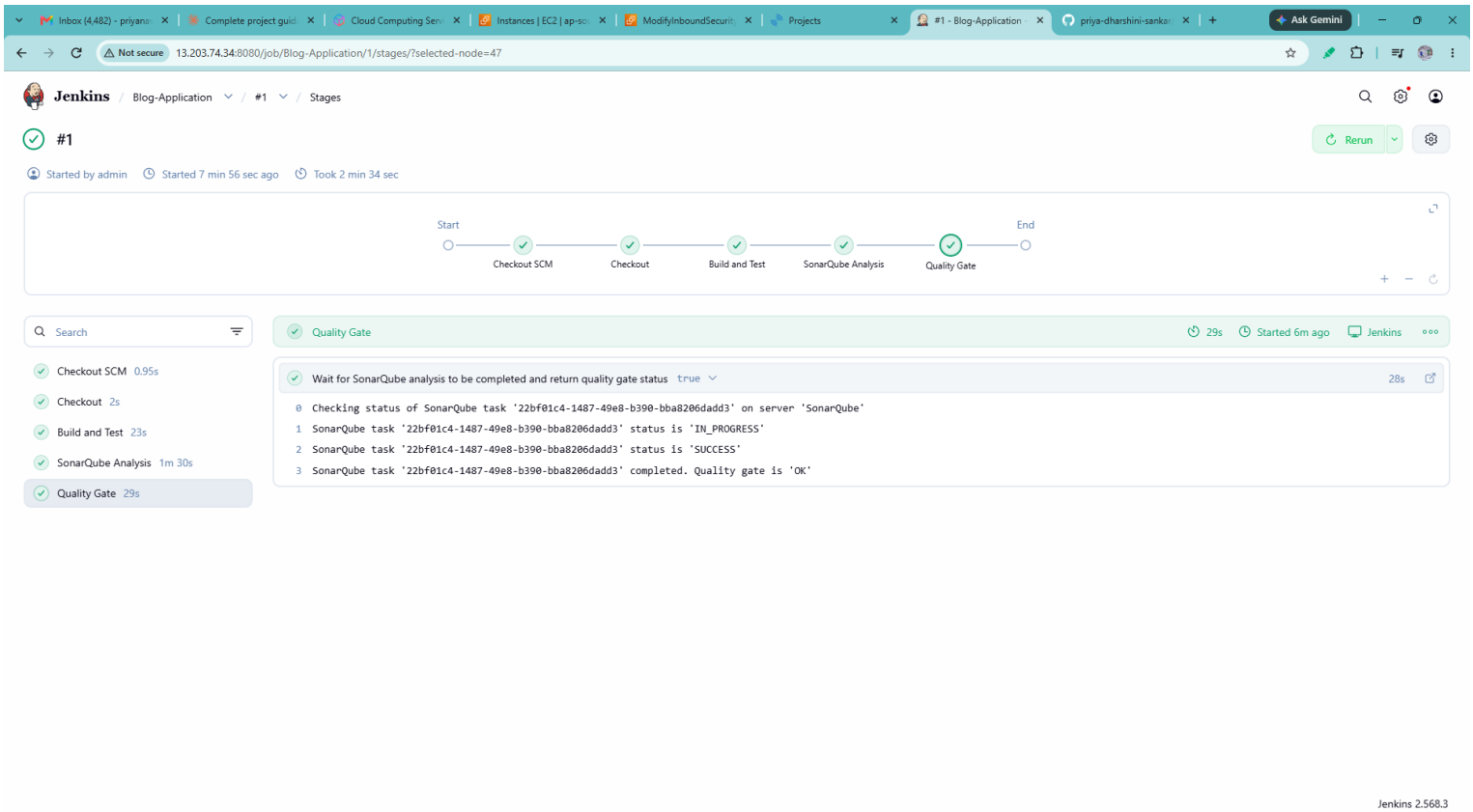

Jenkins / Manage Jenkins / Tools
Not secure 13.203.74.34:8080/manage/configureTools/
SonarScanner for MSBuild installations
Add SonarScanner for MSBuild
SonarQube Scanner installations
Add SonarQube Scanner
SonarQube Scanner
Name
SonarScanner
Install automatically
Install from Maven Central
Version
SonarQube Scanner 8.1.0.6389
Add Installer
Add SonarQube Scanner
Ant installations
Save
Apply

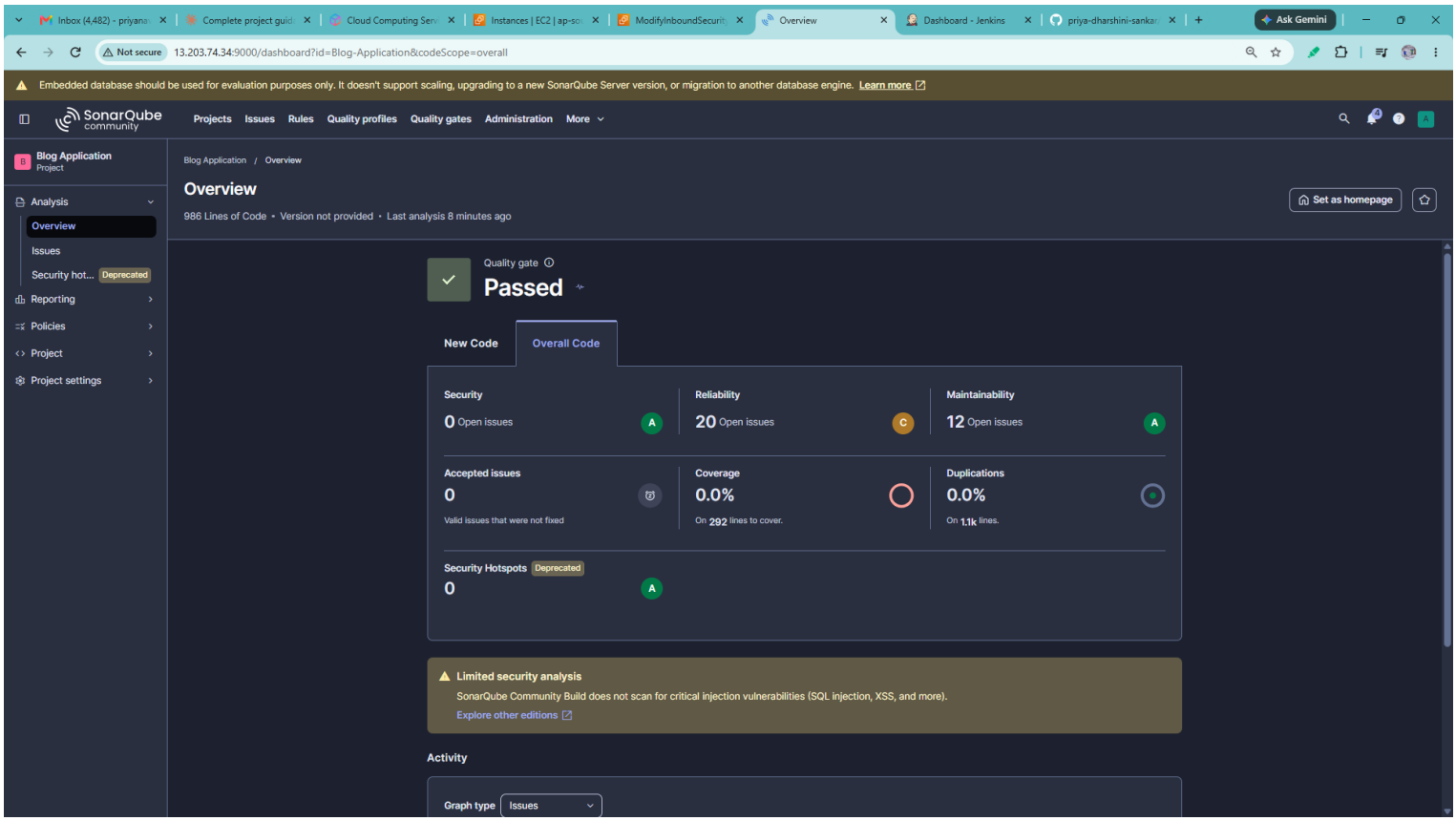

Not secure 13.203.74.34:9000/admin/webhooks
Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. Learn more
SonarQube community
Projects Issues Rules Quality profiles Quality gates Administration More
Administration
Configuration
General settings
Encryption
Webhooks
Security
Projects
System
Marketplace
Administration / Webhooks
Webhooks
Webhooks are used to notify external services when a project analysis is done.
An HTTP POST request including a JSON payload is sent to each of the provided URLs. Learn more in the Webhooks documentation.
Create
Name URL Has secret? Last delivery Actions
Jenkins http://13.203.74.34:8080/sonarqube-webhook/ No Never
SonarQube™ technology is powered by SonarSource Sàrl
Community Build • v26.9.0.129388 • • MQR MODE
LGPL v3 Community Documentation Plugins Web API


Not secure 13.203.74.34:9000/account/security
Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. Learn more
SonarQube community
Projects Issues Rules Quality profiles Quality gates Administration More
Administrator
User
Profile
Security
Notifications
Projects
Appearance
My account / Security
Security
If you want to enforce security by not providing credentials of a real SonarQube user to run your code scan or to invoke web services, you can provide a User Token as a replacement of the user login. This will increase the security of your installation by not letting your analysis user's password going through your network.
Generate Tokens
Name Type Expires in
Enter Token Name
Select Token Type
30 days
Generate
Name Type Project Last use Created Expiration
jenkins-token Project Blog Application Never September 22, 2026 October 22, 2026 Revoke
Enter a new password
Old Password *
Password *
Confirm Password *


Not secure 13.203.74.34:8080/job/Blog-Application/1/console
Jenkins / Blog-Application / #1
19:13:50.696 INFO Analysis report uploaded in 12321ms
19:13:50.724 INFO ANALYSIS SUCCESSFUL, you can find the results at: http://localhost:9000/dashboard?id=Blog-Application
19:13:50.742 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
19:13:50.742 INFO More about the report processing at http://localhost:9000/api/ce/task?id=22bf01c4-1487-49e8-b390-bba8206dadd3
19:13:53.256 INFO Analysis total time: 1:02.913 s
19:13:53.316 INFO SonarScanner Engine completed successfully
19:13:53.697 INFO EXECUTION SUCCESS
19:13:55.312 INFO Total time: 1:12.179s
[Pipeline] }
[Pipeline] // withSonarQubeEnv
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Quality Gate)
[Pipeline] timeout
Timeout set to expire in 10 min
[Pipeline] {
[Pipeline] waitForQualityGate
Checking status of SonarQube task '22bf01c4-1487-49e8-b390-bba8206dadd3' on server 'SonarQube'
SonarQube task '22bf01c4-1487-49e8-b390-bba8206dadd3' status is 'IN_PROGRESS'
SonarQube task '22bf01c4-1487-49e8-b390-bba8206dadd3' status is 'SUCCESS'
SonarQube task '22bf01c4-1487-49e8-b390-bba8206dadd3' completed. Quality gate is 'OK'
[Pipeline] }
[Pipeline] // timeout
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
Jenkins 2.568.3


Not secure 13.203.74.34:8080
Jenkins
New Item
Build History
Build Queue
No builds in the queue.
Build Executor Status
(0 of 2 executors busy)
Add description
All
S W Name ↓ Last Success Last Failure Last Duration
Blog-Application 8 min 51 sec #1 N/A 2 min 34 sec
Icon: S M L
REST API Jenkins 2.568.3


Jenkins / Blog-Application / #1 / Stages
#1
Rerun
Started by admin
Started 7 min 56 sec ago
Took 2 min 34 sec
Start
Checkout SCM
Checkout
Build and Test
SonarQube Analysis
Quality Gate
End
Search
Checkout SCM 0.95s
Checkout 2s
Build and Test 23s
SonarQube Analysis 1m 30s
Quality Gate 29s
Quality Gate
29s
Started 6m ago
Jenkins
Wait for SonarQube analysis to be completed and return quality gate status true
28s
0 Checking status of SonarQube task '22bf01c4-1487-49e8-b390-bba8206dadd3' on server 'SonarQube'
1 SonarQube task '22bf01c4-1487-49e8-b390-bba8206dadd3' status is 'IN_PROGRESS'
2 SonarQube task '22bf01c4-1487-49e8-b390-bba8206dadd3' status is 'SUCCESS'
3 SonarQube task '22bf01c4-1487-49e8-b390-bba8206dadd3' completed. Quality gate is 'OK'
Jenkins 2.568.3


Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. Learn more
SonarQube community
Projects Issues Rules Quality profiles Quality gates Administration More
Blog Application
Project
Analysis
Overview
Issues
Security hot... Deprecated
Reporting
Policies
Project
Project settings
Blog Application / Overview
Overview
986 Lines of Code • Version not provided • Last analysis 8 minutes ago
Set as homepage
Quality gate
Passed
New Code
Overall Code
Security
0 Open issues
A
Reliability
20 Open issues
C
Maintainability
12 Open issues
A
Accepted issues
0
Valid issues that were not fixed
Coverage
0.0%
On 292 lines to cover.
Duplications
0.0%
On 1.1k lines.
Security Hotspots Deprecated
0
A
Limited security analysis
SonarQube Community Build does not scan for critical injection vulnerabilities (SQL injection, XSS, and more).
Explore other editions
Activity
Graph type
Issues

Task 6 : Trivy

Overview

Trivy was integrated into the Jenkins CI/CD pipeline to perform security vulnerability scanning of the Frontend and Backend container images before deployment.

Trivy Configuration

Trivy was installed on the Jenkins CI/CD server and integrated into the Jenkins pipeline after the SonarQube Quality Gate.

The pipeline scans:

- Frontend: blog-frontend:tag1
- Backend: blog-backend:tag1

Pipeline flow:

Checkout → Build & Test → SonarQube Analysis → Quality Gate → Trivy Scan

Vulnerability Scan Results

Trivy successfully scanned both application images.

Frontend Image = CRITICAL: 0, HIGH: 0

Backend Image = CRITICAL: 0, HIGH: 4, MEDIUM: 5

The scan provided visibility into the security status of the container images. No CRITICAL vulnerabilities were detected in either the Frontend or Backend image.

Verification

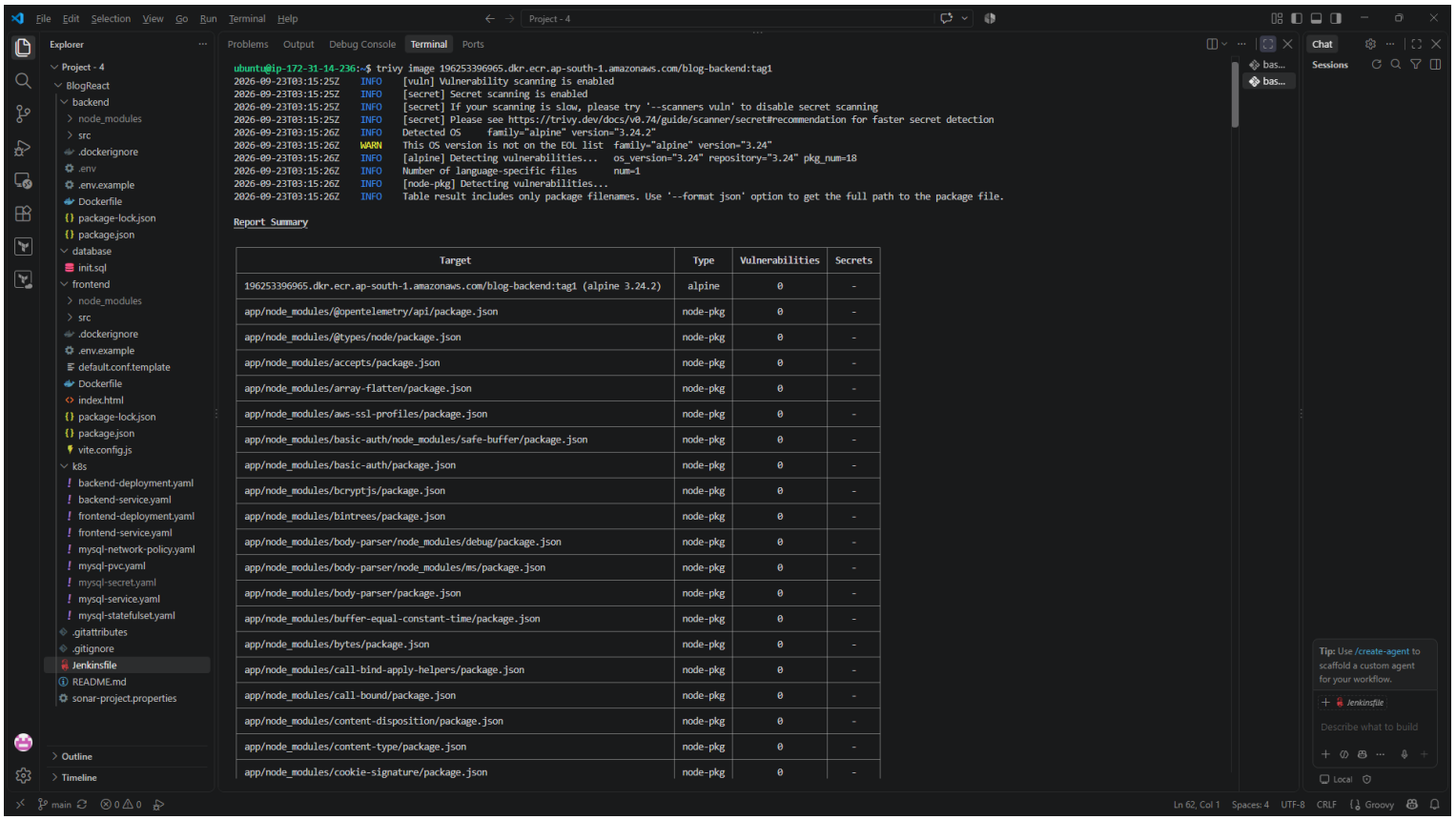
Jenkins Build #4 successfully executed the Trivy Scan stage for both Frontend and Backend images.

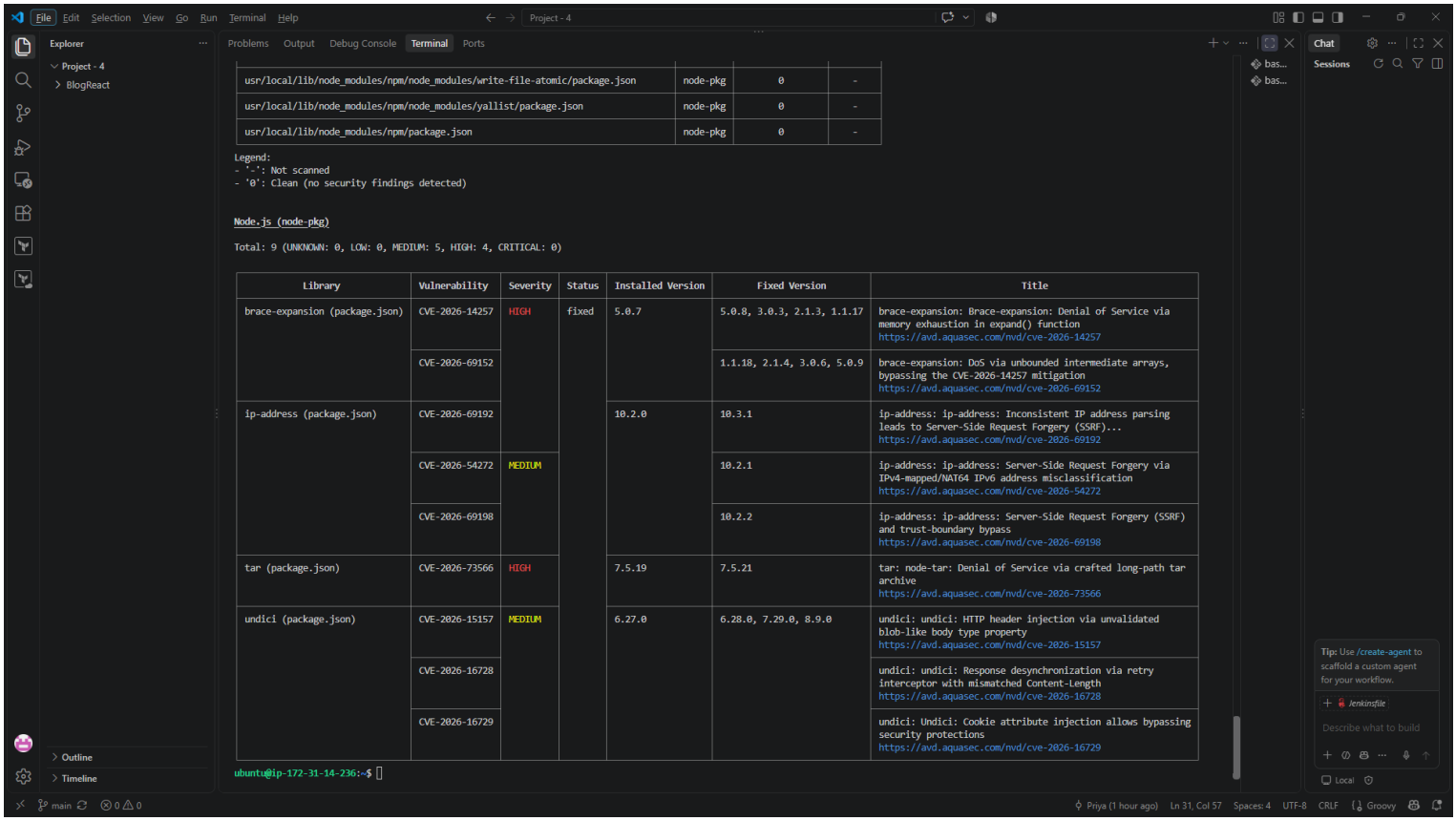
The pipeline completed successfully with:

Finished: SUCCESS

Result

Trivy vulnerability scanning was successfully integrated into the Jenkins CI/CD pipeline. Both Frontend and Backend container images were scanned before deployment, providing security visibility into the application images. The scan completed successfully, with no CRITICAL vulnerabilities detected.





usr/local/lib/node_modules/npm/node_modules/write-file-atomic/package.json	node-pkg	0	-
usr/local/lib/node_modules/npm/node_modules/yallist/package.json	node-pkg	0	-
usr/local/lib/node_modules/npm/package.json	node-pkg	0	-

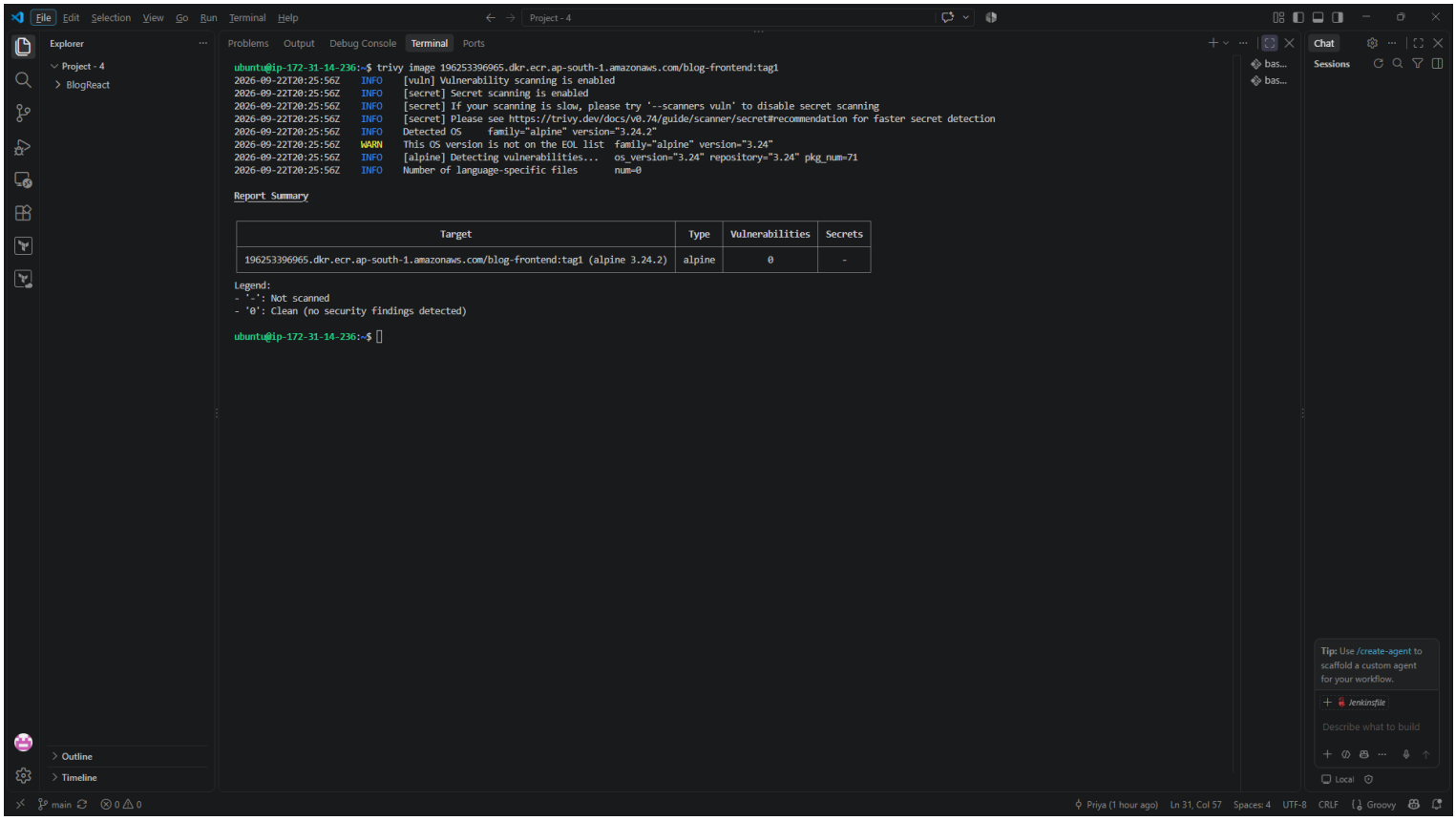
Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)

Node.js (node-pkg)

Total: 9 (UNKNOWN: 0, LOW: 0, MEDIUM: 5, HIGH: 4, CRITICAL: 0)

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
brace-expansion (package.json)	CVE-2026-14257	HIGH	fixed	5.0.7	5.0.8, 3.0.3, 2.1.3, 1.1.17	brace-expansion: Brace-expansion: Denial of Service via memory exhaustion in expand() function https://avd.aquasec.com/nvd/cve-2026-14257
	CVE-2026-69152				1.1.18, 2.1.4, 3.0.6, 5.0.9	brace-expansion: DoS via unbounded intermediate arrays, bypassing the CVE-2026-14257 mitigation https://avd.aquasec.com/nvd/cve-2026-69152
ip-address (package.json)	CVE-2026-69192			10.2.0	10.3.1	ip-address: ip-address: Inconsistent IP address parsing leads to Server-Side Request Forgery (SSRF)... https://avd.aquasec.com/nvd/cve-2026-69192
	CVE-2026-54272	MEDIUM			10.2.1	ip-address: ip-address: Server-Side Request Forgery via IPv4-mapped/NAT64 IPv6 address misclassification https://avd.aquasec.com/nvd/cve-2026-54272
	CVE-2026-69198				10.2.2	ip-address: ip-address: Server-Side Request Forgery (SSRF) and trust-boundary bypass https://avd.aquasec.com/nvd/cve-2026-69198
tar (package.json)	CVE-2026-73566	HIGH		7.5.19	7.5.21	tar: node-tar: Denial of Service via crafted long-path tar archive https://avd.aquasec.com/nvd/cve-2026-73566
undici (package.json)	CVE-2026-15157	MEDIUM		6.27.0	6.28.0, 7.29.0, 8.9.0	undici: undici: HTTP header injection via unvalidated blob-like body type property https://avd.aquasec.com/nvd/cve-2026-15157
	CVE-2026-16728					undici: undici: Response desynchronization via retry interceptor with mismatched Content-Length https://avd.aquasec.com/nvd/cve-2026-16728
	CVE-2026-16729					undici: Undici: Cookie attribute injection allows bypassing security protections https://avd.aquasec.com/nvd/cve-2026-16729

ubuntu@ip-172-31-14-236:~$



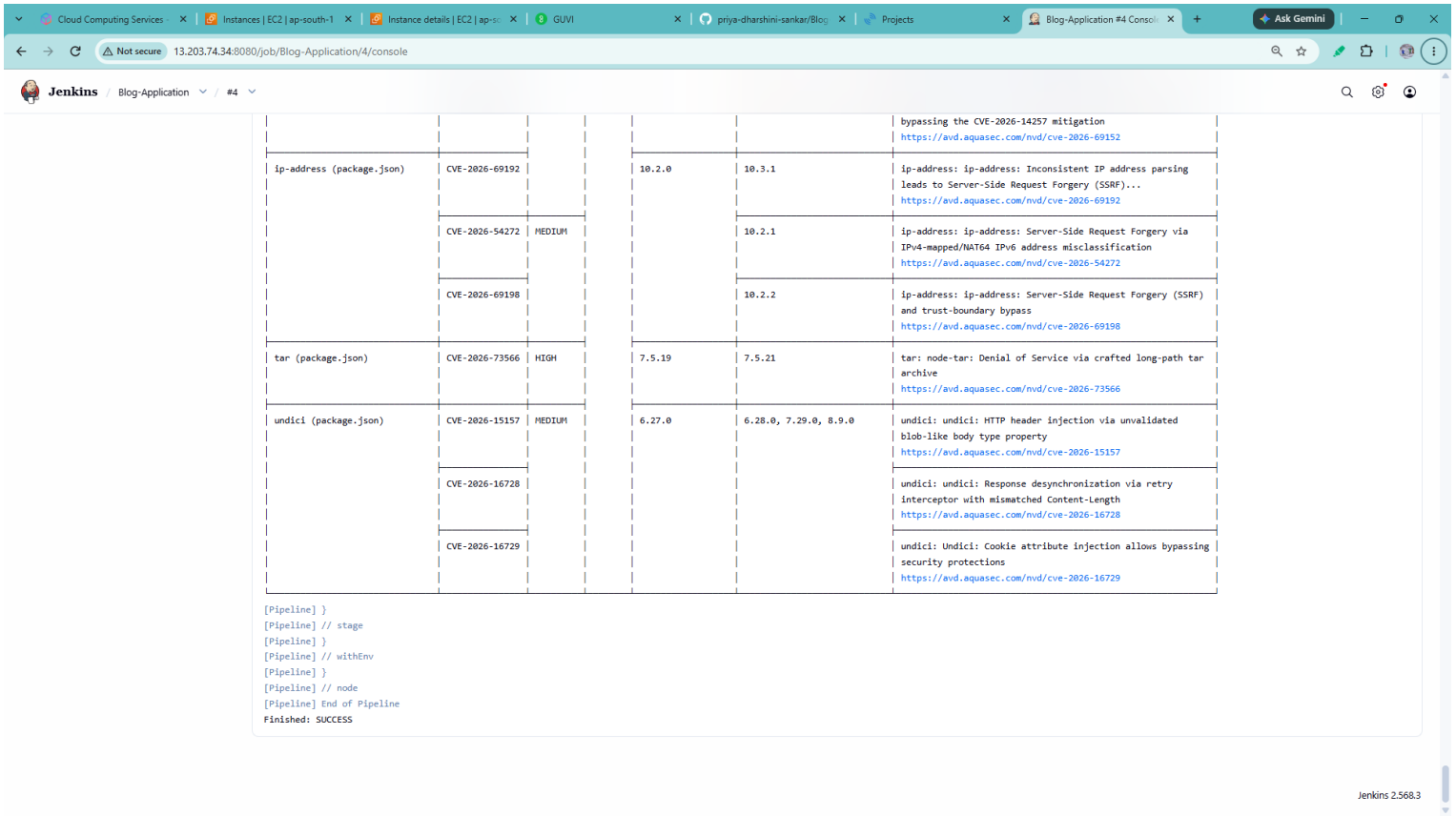

```
ubuntu@ip-172-31-14-236:~$ trivy image 196253396965.dkr.ecr.ap-south-1.amazonaws.com/blog-frontend:tag1
2026-09-22T20:25:56Z    INFO    [vuln] Vulnerability scanning is enabled
2026-09-22T20:25:56Z    INFO    [secret] Secret scanning is enabled
2026-09-22T20:25:56Z    INFO    [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2026-09-22T20:25:56Z    INFO    [secret] Please see https://trivy.dev/docs/v0.74/guide/scanner/secret#recommendation for faster secret detection
2026-09-22T20:25:56Z    INFO    Detected OS     family="alpine" version="3.24.2"
2026-09-22T20:25:56Z    WARN    This OS version is not on the EOL list  family="alpine" version="3.24"
2026-09-22T20:25:56Z    INFO    [alpine] Detecting vulnerabilities...   os_version="3.24" repository="3.24" pkg_num=71
2026-09-22T20:25:56Z    INFO    Number of language-specific files       num=0

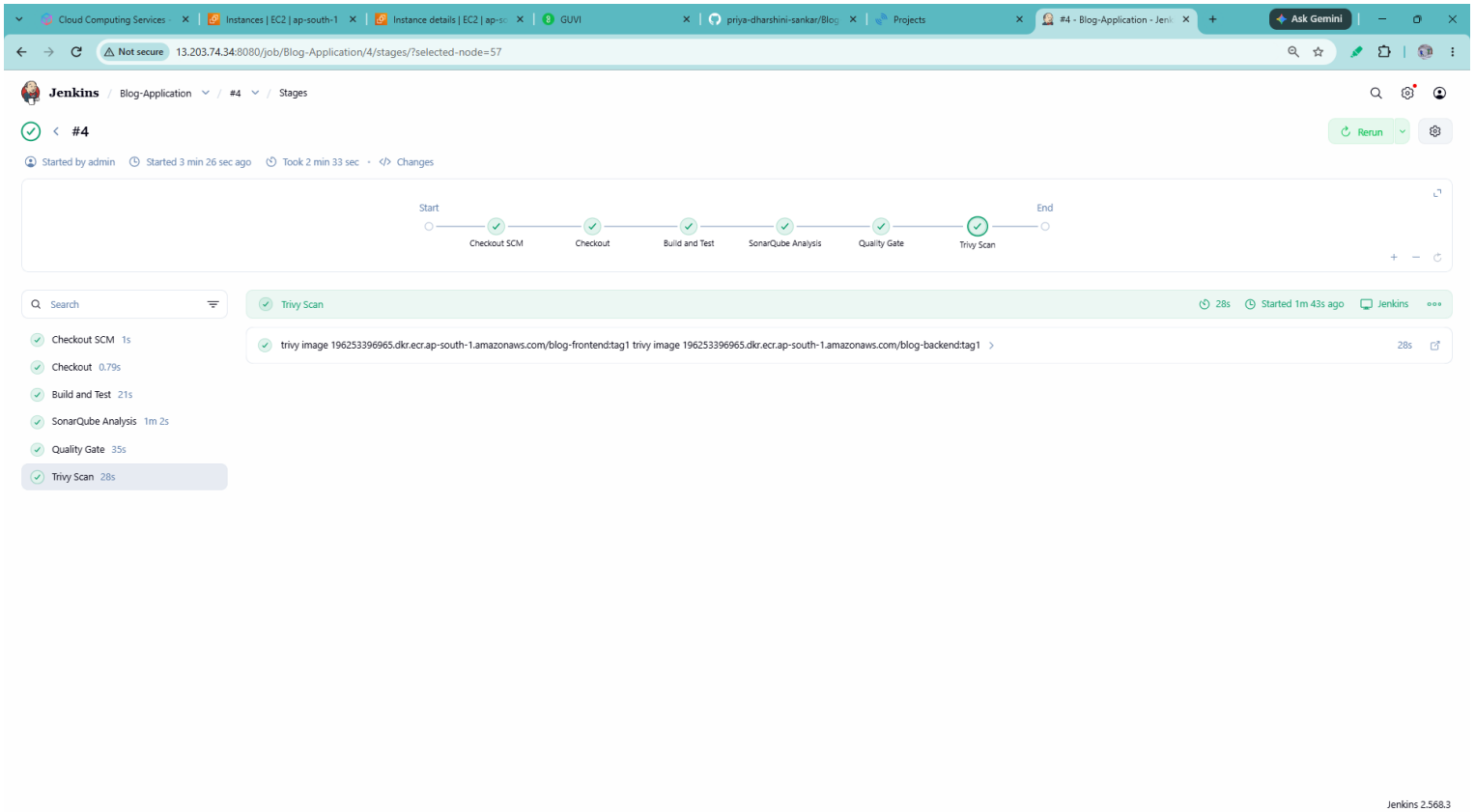
Report Summary
```

Target	Type	Vulnerabilities	Secrets
196253396965.dkr.ecr.ap-south-1.amazonaws.com/blog-frontend:tag1 (alpine 3.24.2)	alpine	0	-

Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)

ubuntu@ip-172-31-14-236:~$


Jenkins / Blog-Application / #4
bypassing the CVE-2026-14257 mitigation
https://avd.aquasec.com/nvd/cve-2026-69152
ip-address (package.json) | CVE-2026-69192 | | 10.2.0 | 10.3.1 | ip-address: ip-address: Inconsistent IP address parsing leads to Server-Side Request Forgery (SSRF)... https://avd.aquasec.com/nvd/cve-2026-69192
CVE-2026-54272 | MEDIUM | 10.2.1 | ip-address: ip-address: Server-Side Request Forgery via IPv4-mapped/NAT64 IPv6 address misclassification https://avd.aquasec.com/nvd/cve-2026-54272
CVE-2026-69198 | | 10.2.2 | ip-address: ip-address: Server-Side Request Forgery (SSRF) and trust-boundary bypass https://avd.aquasec.com/nvd/cve-2026-69198
tar (package.json) | CVE-2026-73566 | HIGH | 7.5.19 | 7.5.21 | tar: node-tar: Denial of Service via crafted long-path tar archive https://avd.aquasec.com/nvd/cve-2026-73566
undici (package.json) | CVE-2026-15157 | MEDIUM | 6.27.0 | 6.28.0, 7.29.0, 8.9.0 | undici: undici: HTTP header injection via unvalidated blob-like body type property https://avd.aquasec.com/nvd/cve-2026-15157
CVE-2026-16728 | | | | undici: undici: Response desynchronization via retry interceptor with mismatched Content-Length https://avd.aquasec.com/nvd/cve-2026-16728
CVE-2026-16729 | | | | undici: Undici: Cookie attribute injection allows bypassing security protections https://avd.aquasec.com/nvd/cve-2026-16729
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
Jenkins 2.568.3


Jenkins / Blog-Application / #4 / Stages
#4
Rerun
Started by admin
Started 3 min 26 sec ago
Took 2 min 33 sec
Changes
Start
Checkout SCM
Checkout
Build and Test
SonarQube Analysis
Quality Gate
Trivy Scan
End
Search
Checkout SCM 1s
Checkout 0.79s
Build and Test 21s
SonarQube Analysis 1m 2s
Quality Gate 35s
Trivy Scan 28s
Trivy Scan
28s
Started 1m 43s ago
Jenkins
trivy image 196253396965.dkr.ecr.ap-south-1.amazonaws.com/blog-frontend:tag1 trivy image 196253396965.dkr.ecr.ap-south-1.amazonaws.com/blog-backend:tag1
28s
Jenkins 2.568.3

Task 7 : Amazon EKS

Overview

The three-tier Blog Application was deployed on Amazon EKS using Kubernetes resources for the Frontend, Backend, and Database tiers.

Kubernetes Resources

The following resources were configured:

- Frontend: Deployment with LoadBalancer Service
- Backend: Deployment with ClusterIP Service
- Database: MySQL StatefulSet with ClusterIP Service
- ConfigMap: mysql-initdb for database initialization
- Secret: mysql-secret for database credentials
- Persistent Storage: 5 GiB PersistentVolumeClaim using gp2
- External Access: AWS LoadBalancer for the Frontend

Application Communication

The application communication was verified inside the EKS cluster.
The Frontend successfully accessed the Backend API through the /api route, and the API returned all 4 blog posts.
The Backend readiness check also returned : {"status":"ready"}

This confirmed that the application components were communicating successfully.

External Accessibility

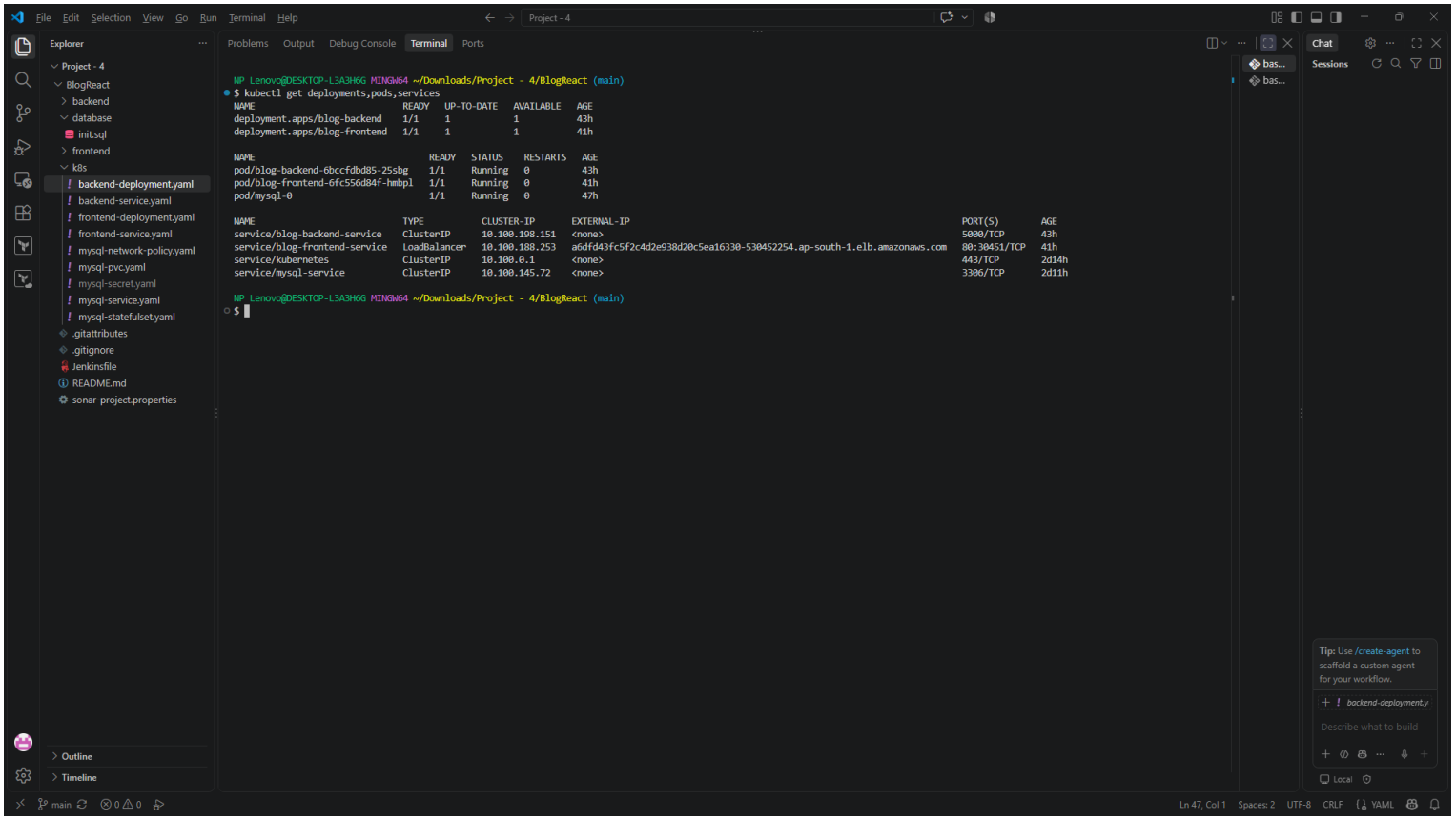
The Frontend was exposed through an AWS LoadBalancer.

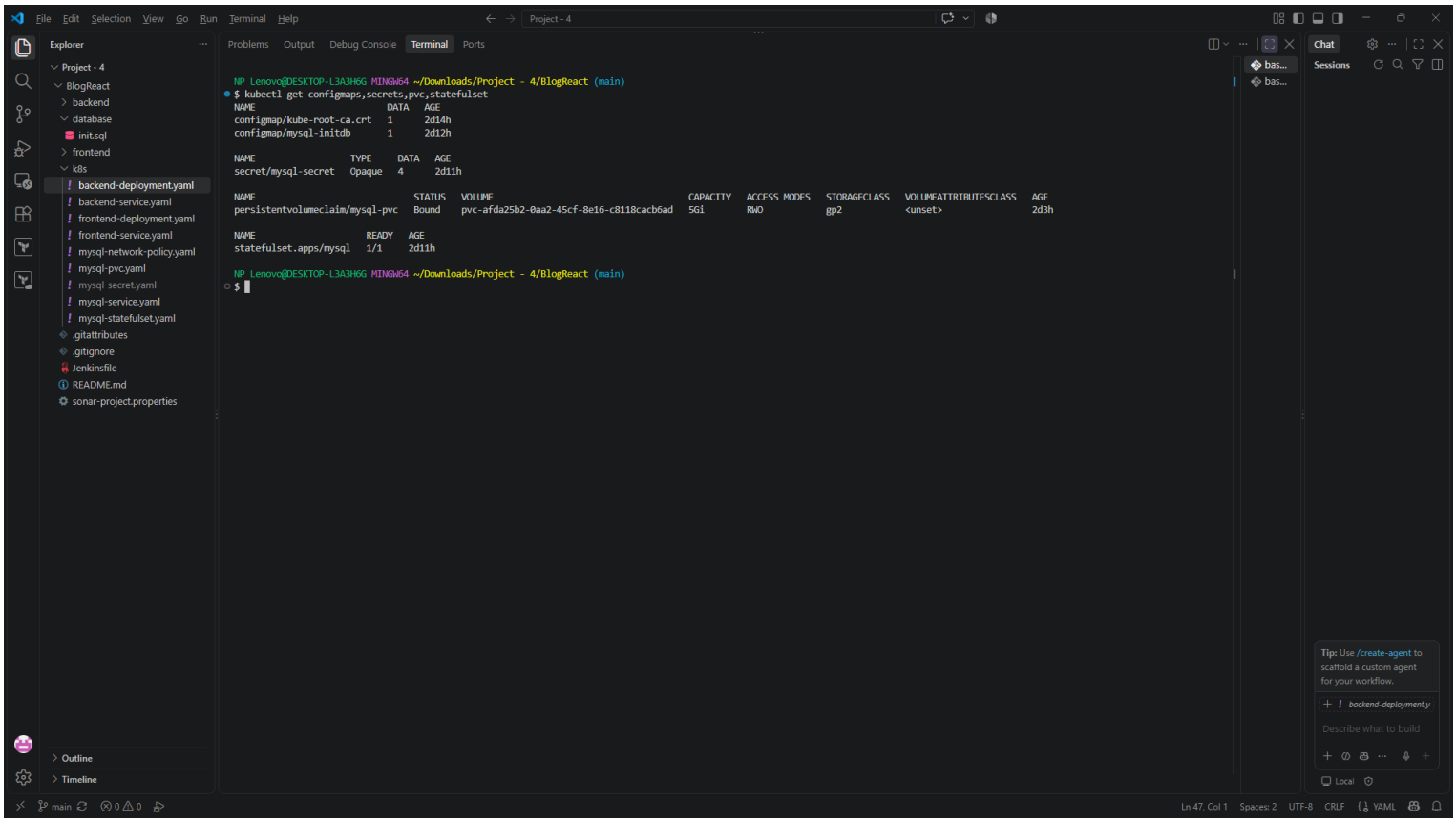
The application was successfully accessed through the LoadBalancer URL, and the blog posts were displayed in the browser.

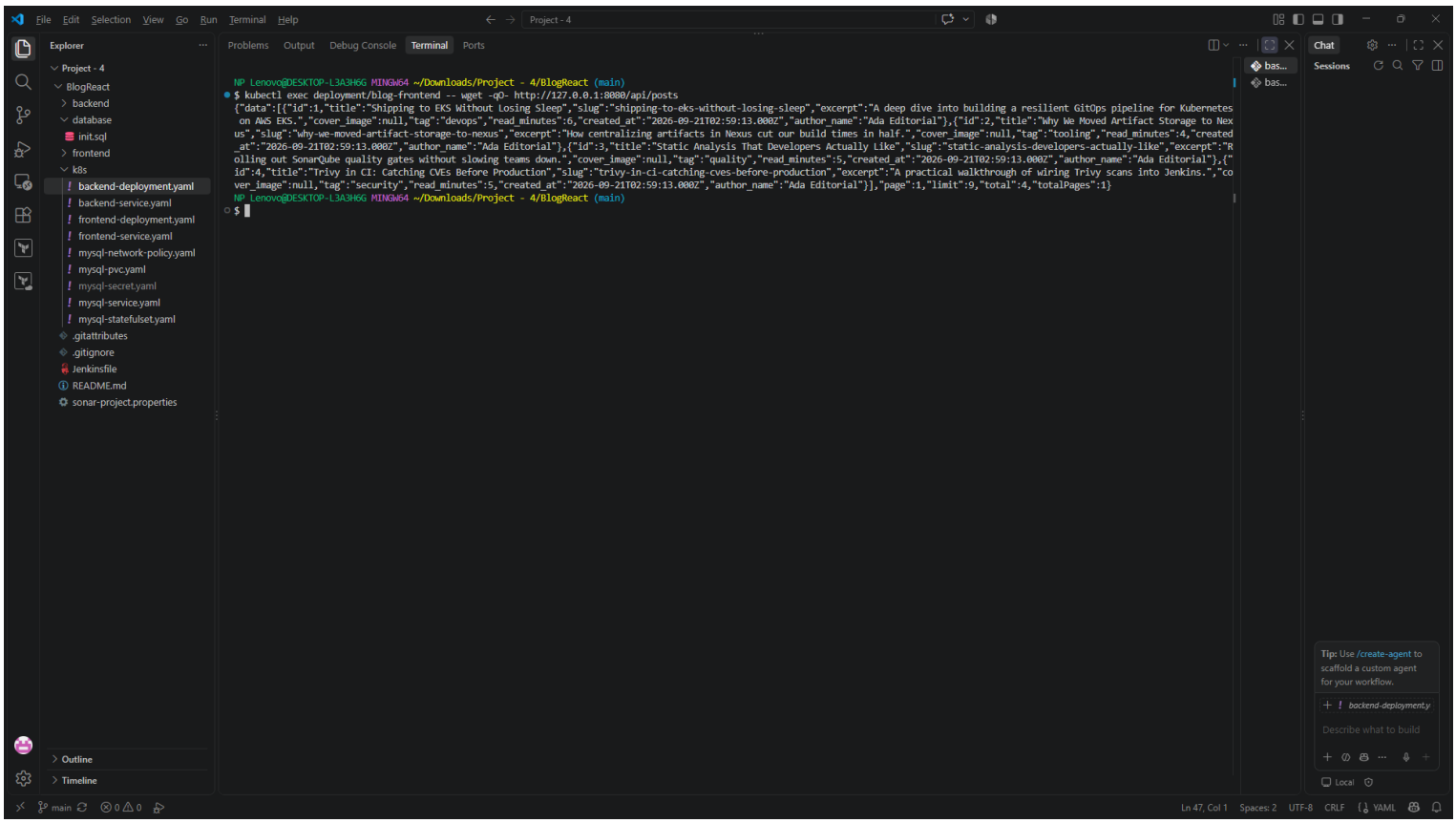
Result

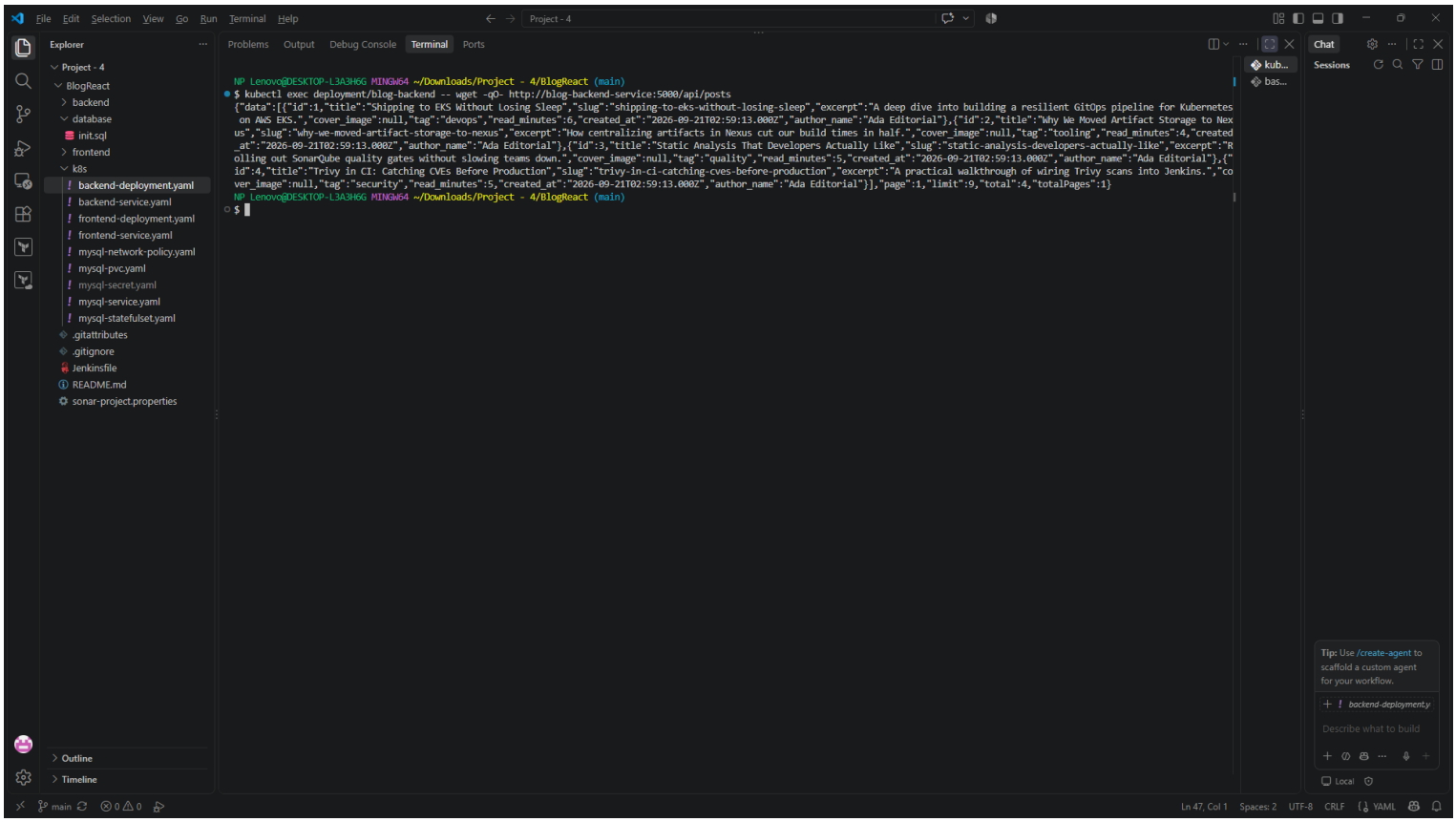
The three-tier Blog Application was successfully deployed and verified on Amazon EKS. The Frontend, Backend, and Database tiers communicated successfully, persistent storage was

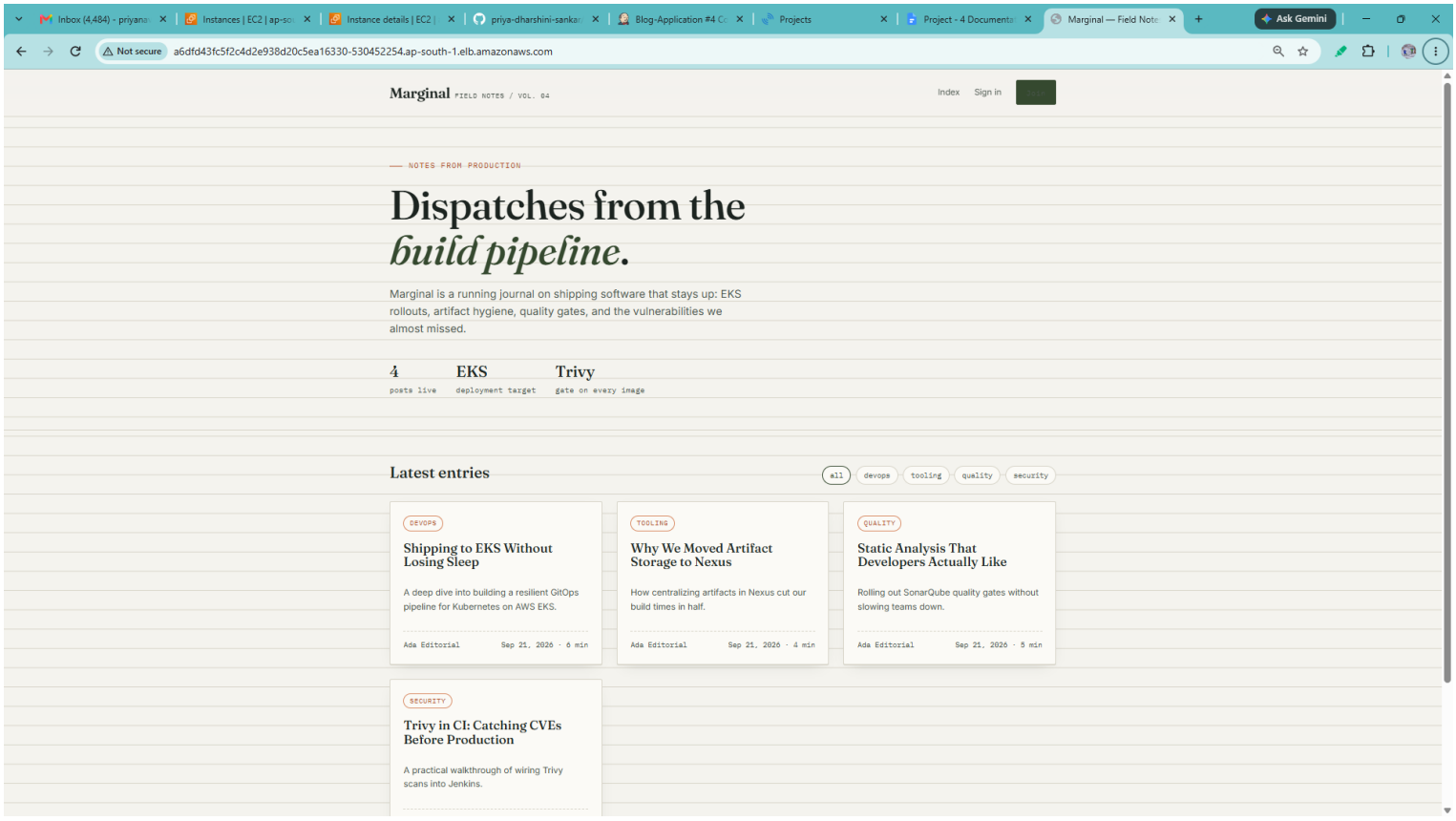
configured for MySQL, and the application was externally accessible through the AWS LoadBalancer.




File Edit Selection View Go Run Terminal Help
Project - 4
Explorer
Project - 4
BlogReact
backend
database
init.sql
frontend
k8s
backend-deployment.yaml
backend-service.yaml
frontend-deployment.yaml
frontend-service.yaml
mysql-network-policy.yaml
mysql-pvc.yaml
mysql-secret.yaml
mysql-service.yaml
mysql-statefulset.yaml
.gitattributes
.gitignore
Jenkinsfile
README.md
sonar-project.properties
Problems Output Debug Console Terminal Ports
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$ kubectl get configmaps,secrets,pvc,statefulset
NAME DATA AGE
configmap/kube-root-ca.crt 1 2d14h
configmap/mysql-initdb 1 2d12h
NAME TYPE DATA AGE
secret/mysql-secret Opaque 4 2d11h
NAME STATUS VOLUME CAPACITY ACCESS MODES STORAGECLASS VOLUMEATTRIBUTESCLASS AGE
persistentvolumeclaim/mysql-pvc Bound pvc-afda25b2-0aa2-45cf-8e16-c8118cacb6ad 5Gi RWO gp2 <unset> 2d3h
NAME READY AGE
statefulset.apps/mysql 1/1 2d11h
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Describe what to build
Outline
Timeline
main
Ln 47, Col 1 Spaces: 2 UTF-8 CRLF YAML


File Edit Selection View Go Run Terminal Help
Project - 4
Explorer
Project - 4
BlogReact
backend
database
init.sql
frontend
k8s
backend-deployment.yaml
backend-service.yaml
frontend-deployment.yaml
frontend-service.yaml
mysql-network-policy.yaml
mysql-pvc.yaml
mysql-secret.yaml
mysql-service.yaml
mysql-statefulset.yaml
.gitattributes
.gitignore
Jenkinsfile
README.md
sonar-project.properties
Problems Output Debug Console Terminal Ports
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$ kubectl exec deployment/blog-frontend -- wget -qO- http://127.0.0.1:8080/api/posts
{"data":[{"id":1,"title":"Shipping to EKS Without Losing Sleep","slug":"shipping-to-eks-without-losing-sleep","excerpt":"A deep dive into building a resilient GitOps pipeline for Kubernetes on AWS EKS.","cover_image":null,"tag":"devops","read_minutes":6,"created_at":"2026-09-21T02:59:13.000Z","author_name":"Ada Editorial"},{"id":2,"title":"Why We Moved Artifact Storage to Nexus","slug":"why-we-moved-artifact-storage-to-nexus","excerpt":"How centralizing artifacts in Nexus cut our build times in half.","cover_image":null,"tag":"tooling","read_minutes":4,"created_at":"2026-09-21T02:59:13.000Z","author_name":"Ada Editorial"},{"id":3,"title":"Static Analysis That Developers Actually Like","slug":"static-analysis-developers-actually-like","excerpt":"Rolling out SonarQube quality gates without slowing teams down.","cover_image":null,"tag":"quality","read_minutes":5,"created_at":"2026-09-21T02:59:13.000Z","author_name":"Ada Editorial"},{"id":4,"title":"Trivy in CI: Catching CVEs Before Production","slug":"trivy-in-ci-catching-cves-before-production","excerpt":"A practical walkthrough of wiring Trivy scans into Jenkins.","cover_image":null,"tag":"security","read_minutes":5,"created_at":"2026-09-21T02:59:13.000Z","author_name":"Ada Editorial"}],"page":1,"limit":9,"total":4,"totalPages":1}
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Describe what to build
Outline
Timeline
main
Ln 47, Col 1 Spaces: 2 UTF-8 CRLF YAML


File Edit Selection View Go Run Terminal Help
Project - 4
Explorer
Project - 4
BlogReact
backend
database
init.sql
frontend
k8s
backend-deployment.yaml
backend-service.yaml
frontend-deployment.yaml
frontend-service.yaml
mysql-network-policy.yaml
mysql-pvc.yaml
mysql-secret.yaml
mysql-service.yaml
mysql-statefulset.yaml
.gitattributes
.gitignore
Jenkinsfile
README.md
sonar-project.properties
Problems Output Debug Console Terminal Ports
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$ kubectl exec deployment/blog-backend -- wget -qO- http://blog-backend-service:5000/api/posts
{"data":[{"id":1,"title":"Shipping to EKS Without Losing Sleep","slug":"shipping-to-eks-without-losing-sleep","excerpt":"A deep dive into building a resilient GitOps pipeline for Kubernetes on AWS EKS.","cover_image":null,"tag":"devops","read_minutes":6,"created_at":"2026-09-21T02:59:13.000Z","author_name":"Ada Editorial"},{"id":2,"title":"Why We Moved Artifact Storage to Nexus","slug":"why-we-moved-artifact-storage-to-nexus","excerpt":"How centralizing artifacts in Nexus cut our build times in half.","cover_image":null,"tag":"tooling","read_minutes":4,"created_at":"2026-09-21T02:59:13.000Z","author_name":"Ada Editorial"},{"id":3,"title":"Static Analysis That Developers Actually Like","slug":"static-analysis-developers-actually-like","excerpt":"Rolling out SonarQube quality gates without slowing teams down.","cover_image":null,"tag":"quality","read_minutes":5,"created_at":"2026-09-21T02:59:13.000Z","author_name":"Ada Editorial"},{"id":4,"title":"Trivy in CI: Catching CVEs Before Production","slug":"trivy-in-ci-catching-cves-before-production","excerpt":"A practical walkthrough of wiring Trivy scans into Jenkins.","cover_image":null,"tag":"security","read_minutes":5,"created_at":"2026-09-21T02:59:13.000Z","author_name":"Ada Editorial"}],"page":1,"limit":9,"total":4,"totalPages":1}
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Describe what to build
Outline
Timeline
main
Ln 47, Col 1 Spaces: 2 UTF-8 CRLF YAML


Not secure
a6dfd43fc5f2c4d2e938d20c5ea16330-530452254.ap-south-1.elb.amazonaws.com
Ask Gemini
Marginal FIELD NOTES / VOL. 04
Index
Sign in
NOTES FROM PRODUCTION
Dispatches from the build pipeline.
Marginal is a running journal on shipping software that stays up: EKS rollouts, artifact hygiene, quality gates, and the vulnerabilities we almost missed.
4
posts live
EKS
deployment target
Trivy
gate on every image
Latest entries
all
devops
tooling
quality
security
DEVOPS
Shipping to EKS Without Losing Sleep
A deep dive into building a resilient GitOps pipeline for Kubernetes on AWS EKS.
Ada Editorial
Sep 21, 2026 · 6 min
TOOLING
Why We Moved Artifact Storage to Nexus
How centralizing artifacts in Nexus cut our build times in half.
Ada Editorial
Sep 21, 2026 · 4 min
QUALITY
Static Analysis That Developers Actually Like
Rolling out SonarQube quality gates without slowing teams down.
Ada Editorial
Sep 21, 2026 · 5 min
SECURITY
Trivy in CI: Catching CVEs Before Production
A practical walkthrough of wiring Trivy scans into Jenkins.

Task 8 : Kubernetes Deployment

Overview

The Frontend and Backend deployments were configured in Amazon EKS to improve application availability and automatically detect unhealthy workloads.

Deployment Configuration

The following configurations were implemented:

- Frontend: 2 replicas
- Backend: 2 replicas
- Readiness probes configured for Frontend and Backend
- Liveness probes configured for Frontend and Backend
- CPU and memory resource requests and limits configured
- RollingUpdate strategy configured with maxUnavailable: 0 and maxSurge: 1
- Kubernetes Services configured for application service discovery

Rolling Update

The Frontend and Backend deployments use the Kubernetes RollingUpdate strategy.

During an update, Kubernetes gradually replaces the existing pods with new pods while maintaining available replicas. This helps keep the application available during deployments.

Service Discovery

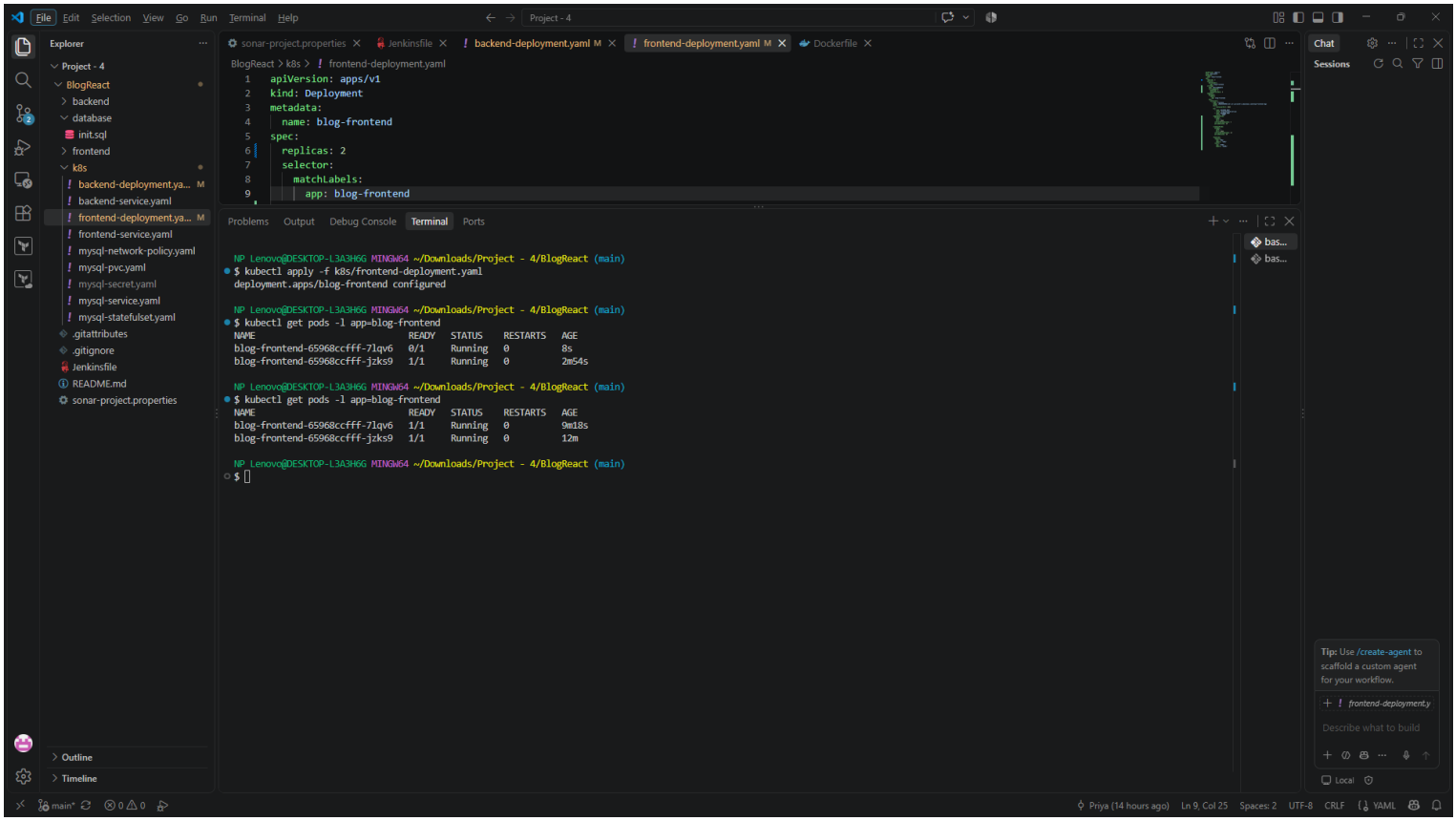
Kubernetes Services provide stable names for communication between application components.

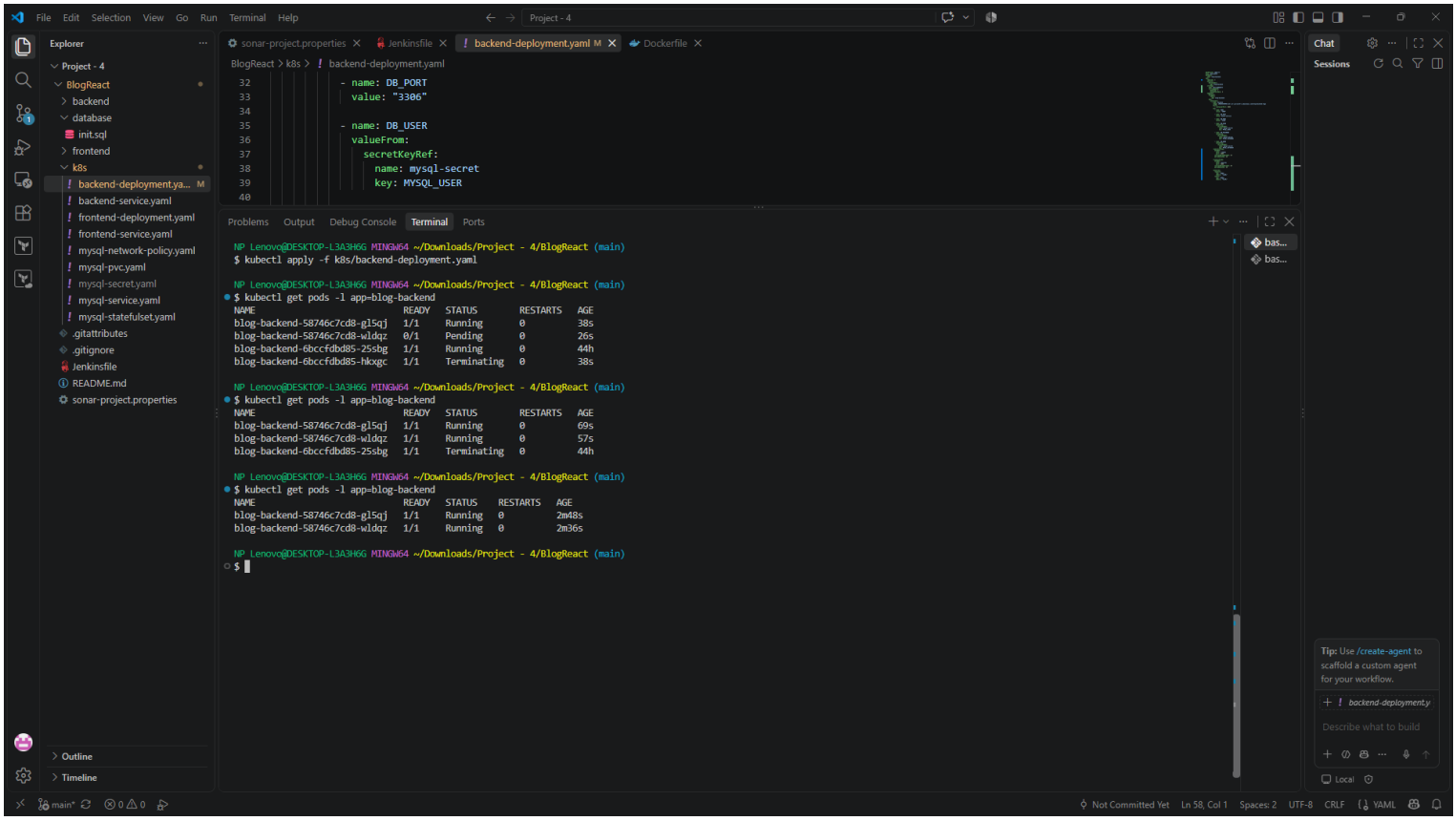
- Frontend → blog-backend-service:5000
- Backend → mysql-service:3306

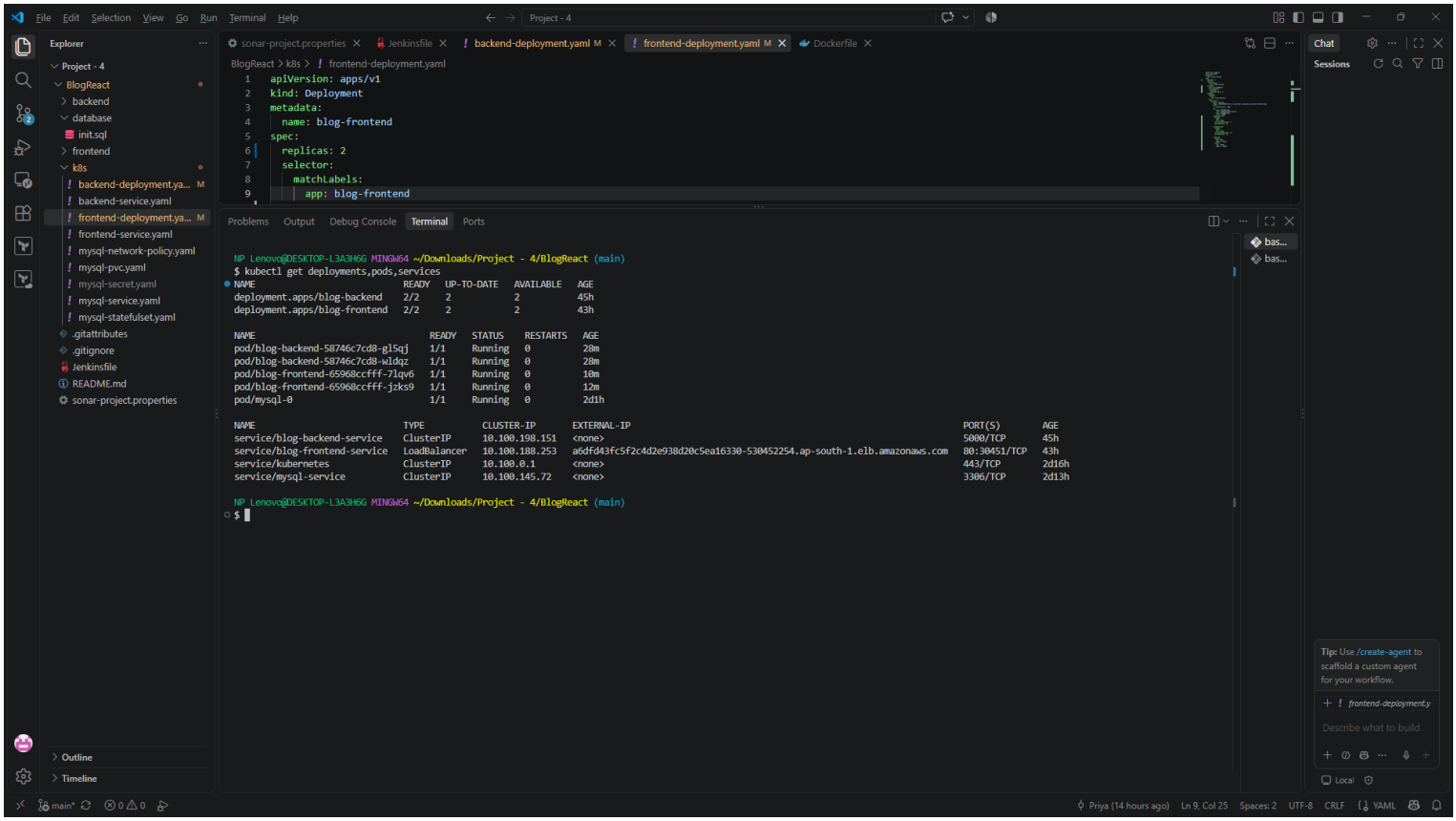
This allows the application components to communicate using Kubernetes Service names instead of pod IP addresses.

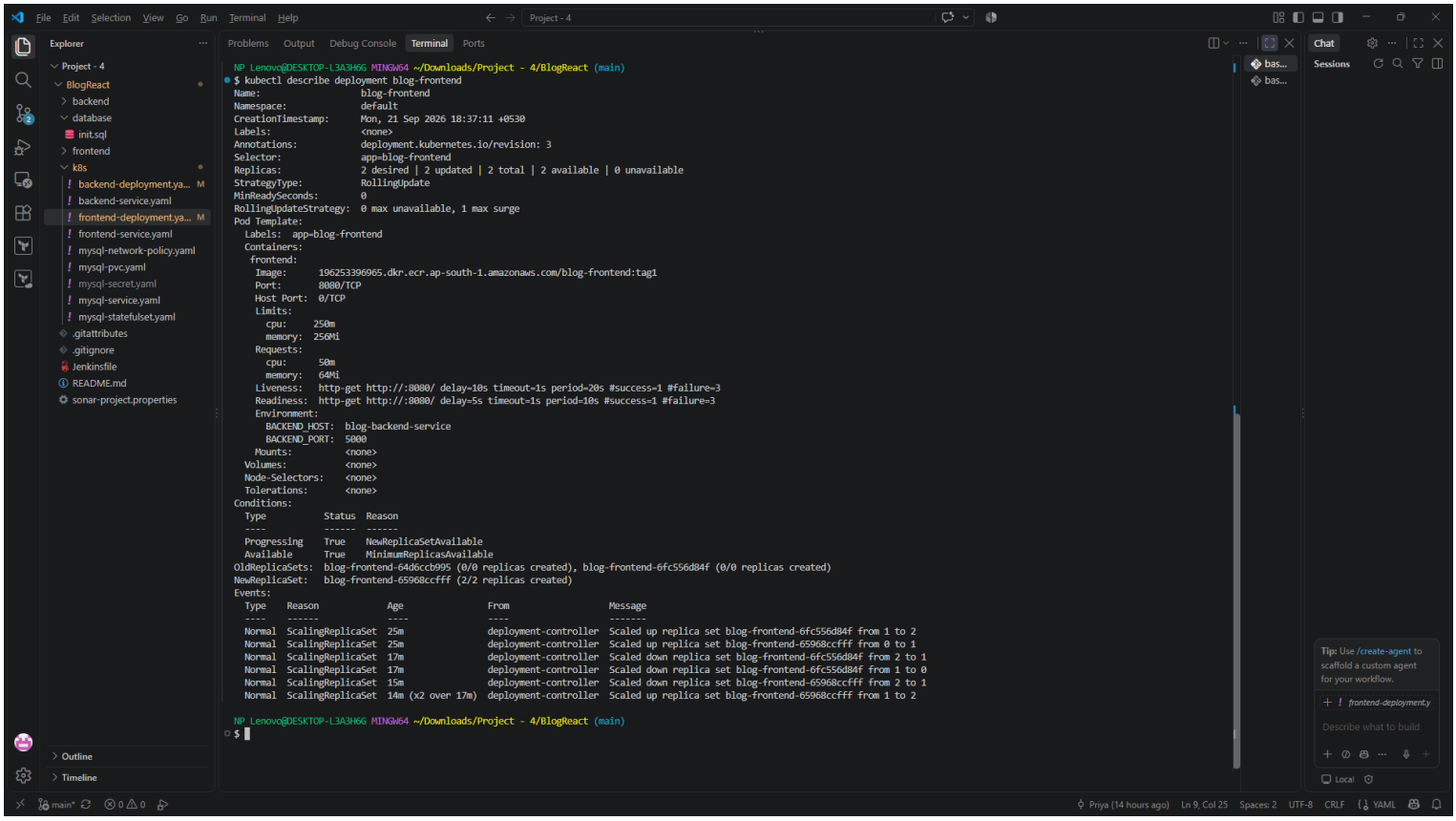
Result

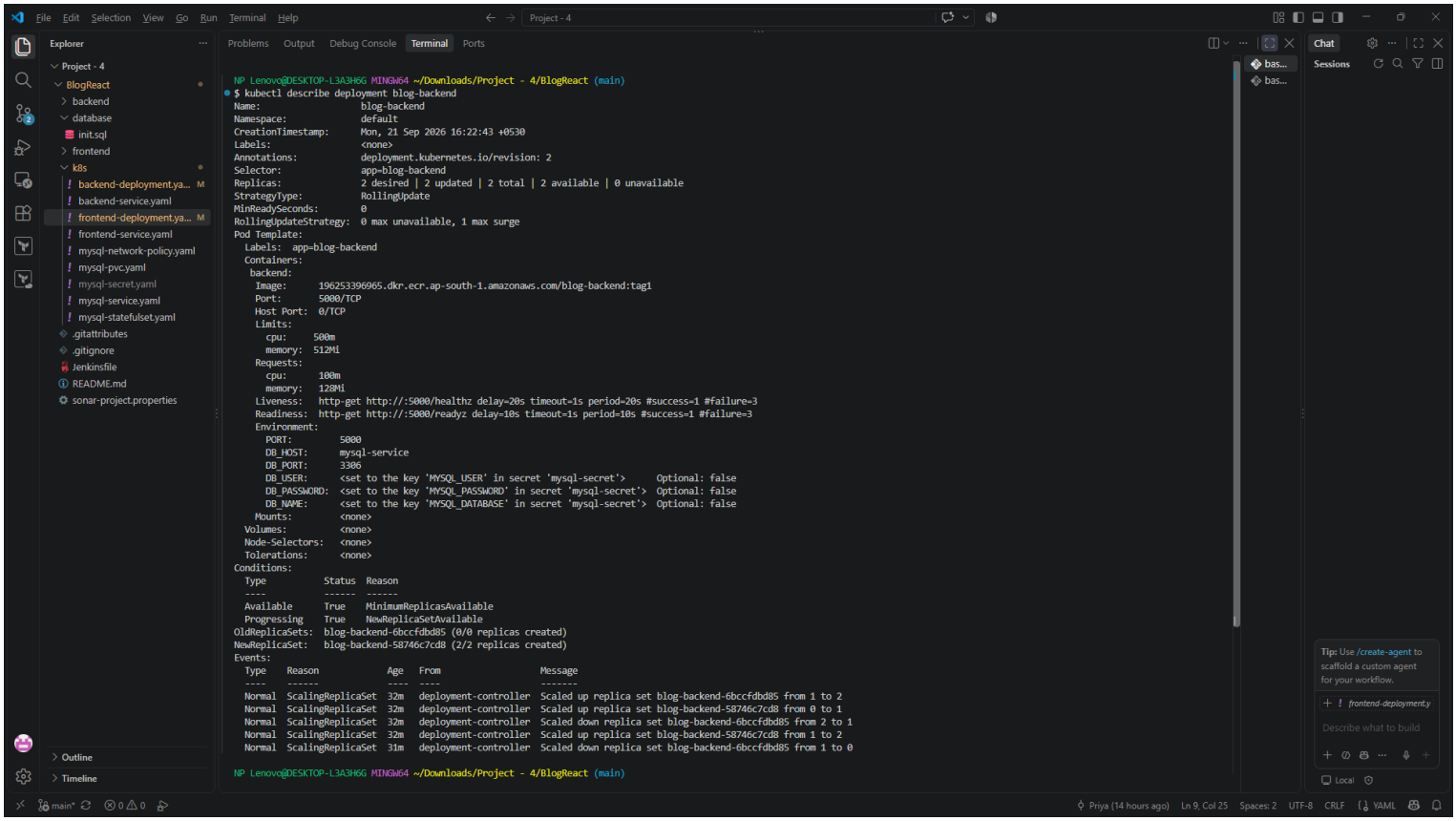
The Frontend and Backend workloads were successfully configured with multiple replicas, readiness and liveness probes, resource management, rolling updates, and Kubernetes service discovery as required.


File Edit Selection View Go Run Terminal Help
Project - 4
Explorer
Project - 4
BlogReact
backend
database
init.sql
frontend
k8s
backend-deployment.ya... M
backend-service.yaml
frontend-deployment.ya... M
frontend-service.yaml
mysql-network-policy.yaml
mysql-pvc.yaml
mysql-secret.yaml
mysql-service.yaml
mysql-statefulset.yaml
.gitattributes
.gitignore
Jenkinsfile
README.md
sonar-project.properties
Outline
Timeline
sonar-project.properties
Jenkinsfile
backend-deployment.yaml M
frontend-deployment.yaml M
Dockerfile
BlogReact > k8s > ! frontend-deployment.yaml
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4 name: blog-frontend
5 spec:
6 replicas: 2
7 selector:
8 matchLabels:
9 app: blog-frontend
Problems Output Debug Console Terminal Ports
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$ kubectl apply -f k8s/frontend-deployment.yaml
deployment.apps/blog-frontend configured
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$ kubectl get pods -l app=blog-frontend
NAME READY STATUS RESTARTS AGE
blog-frontend-65968ccfff-7lqv6 0/1 Running 0 8s
blog-frontend-65968ccfff-jzks9 1/1 Running 0 2m54s
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$ kubectl get pods -l app=blog-frontend
NAME READY STATUS RESTARTS AGE
blog-frontend-65968ccfff-7lqv6 1/1 Running 0 9m18s
blog-frontend-65968ccfff-jzks9 1/1 Running 0 12m
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
frontend-deployment.y
Describe what to build
Local
main*
Priya (14 hours ago) Ln 9, Col 25 Spaces: 2 UTF-8 CRLF YAML


File Edit Selection View Go Run Terminal Help
Project - 4
Explorer
Project - 4
BlogReact
backend
database
init.sql
frontend
k8s
backend-deployment.ya... M
backend-service.yaml
frontend-deployment.yaml
frontend-service.yaml
mysql-network-policy.yaml
mysql-pvc.yaml
mysql-secret.yaml
mysql-service.yaml
mysql-statefulset.yaml
.gitattributes
.gitignore
Jenkinsfile
README.md
sonar-project.properties
Outline
Timeline
sonar-project.properties
Jenkinsfile
backend-deployment.yaml M
Dockerfile
BlogReact > k8s > ! backend-deployment.yaml
- name: DB_PORT
value: "3306"
- name: DB_USER
valueFrom:
secretKeyRef:
name: mysql-secret
key: MYSQL_USER
Problems Output Debug Console Terminal Ports
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$ kubectl apply -f k8s/backend-deployment.yaml
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$ kubectl get pods -l app=blog-backend
NAME READY STATUS RESTARTS AGE
blog-backend-58746c7cd8-gl5qj 1/1 Running 0 38s
blog-backend-58746c7cd8-wldqz 0/1 Pending 0 26s
blog-backend-6bccfdbd85-25sbg 1/1 Running 0 44h
blog-backend-6bccfdbd85-hkxgc 1/1 Terminating 0 38s
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$ kubectl get pods -l app=blog-backend
NAME READY STATUS RESTARTS AGE
blog-backend-58746c7cd8-gl5qj 1/1 Running 0 69s
blog-backend-58746c7cd8-wldqz 1/1 Running 0 57s
blog-backend-6bccfdbd85-25sbg 1/1 Terminating 0 44h
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$ kubectl get pods -l app=blog-backend
NAME READY STATUS RESTARTS AGE
blog-backend-58746c7cd8-gl5qj 1/1 Running 0 2m48s
blog-backend-58746c7cd8-wldqz 1/1 Running 0 2m36s
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
backend-deployment.y
Describe what to build
Local
main*
Not Committed Yet Ln 58, Col 1 Spaces: 2 UTF-8 CRLF YAML


File Edit Selection View Go Run Terminal Help
Project - 4
Explorer
Project - 4
BlogReact
backend
database
init.sql
frontend
k8s
backend-deployment.ya... M
backend-service.yaml
frontend-deployment.ya... M
frontend-service.yaml
mysql-network-policy.yaml
mysql-pvc.yaml
mysql-secret.yaml
mysql-service.yaml
mysql-statefulset.yaml
.gitattributes
.gitignore
Jenkinsfile
README.md
sonar-project.properties
sonar-project.properties
Jenkinsfile
backend-deployment.yaml M
frontend-deployment.yaml M
Dockerfile
BlogReact > k8s > frontend-deployment.yaml
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4 name: blog-frontend
5 spec:
6 replicas: 2
7 selector:
8 matchLabels:
9 app: blog-frontend
Problems Output Debug Console Terminal Ports
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$ kubectl get deployments,pods,services
NAME READY UP-TO-DATE AVAILABLE AGE
deployment.apps/blog-backend 2/2 2 2 45h
deployment.apps/blog-frontend 2/2 2 2 43h
NAME READY STATUS RESTARTS AGE
pod/blog-backend-58746c7cd8-gl5qj 1/1 Running 0 28m
pod/blog-backend-58746c7cd8-wldqz 1/1 Running 0 28m
pod/blog-frontend-65968ccfff-7lqv6 1/1 Running 0 10m
pod/blog-frontend-65968ccfff-jzks9 1/1 Running 0 12m
pod/mysql-0 1/1 Running 0 2d1h
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
service/blog-backend-service ClusterIP 10.100.198.151 <none> 5000/TCP 45h
service/blog-frontend-service LoadBalancer 10.100.188.253 a6dfd43fc5f2c4d2e938d20c5ea16330-530452254.ap-south-1.elb.amazonaws.com 80:30451/TCP 43h
service/kubernetes ClusterIP 10.100.0.1 <none> 443/TCP 2d16h
service/mysql-service ClusterIP 10.100.145.72 <none> 3306/TCP 2d13h
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Describe what to build
Outline
Timeline
main*
Priya (14 hours ago) Ln 9, Col 25 Spaces: 2 UTF-8 CRLF YAML


File Edit Selection View Go Run Terminal Help
Project - 4
Explorer
Project - 4
BlogReact
backend
database
init.sql
frontend
k8s
backend-deployment.ya... M
backend-service.yaml
frontend-deployment.ya... M
frontend-service.yaml
mysql-network-policy.yaml
mysql-pvc.yaml
mysql-secret.yaml
mysql-service.yaml
mysql-statefulset.yaml
.gitattributes
.gitignore
Jenkinsfile
README.md
sonar-project.properties
Problems Output Debug Console Terminal Ports
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$ kubectl describe deployment blog-frontend
Name: blog-frontend
Namespace: default
CreationTimestamp: Mon, 21 Sep 2026 18:37:11 +0530
Labels: <none>
Annotations: deployment.kubernetes.io/revision: 3
Selector: app=blog-frontend
Replicas: 2 desired | 2 updated | 2 total | 2 available | 0 unavailable
StrategyType: RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 0 max unavailable, 1 max surge
Pod Template:
Labels: app=blog-frontend
Containers:
frontend:
Image: 196253396965.dkr.ecr.ap-south-1.amazonaws.com/blog-frontend:tag1
Port: 8080/TCP
Host Port: 0/TCP
Limits:
cpu: 250m
memory: 256Mi
Requests:
cpu: 50m
memory: 64Mi
Liveness: http-get http://:8080/ delay=10s timeout=1s period=20s #success=1 #failure=3
Readiness: http-get http://:8080/ delay=5s timeout=1s period=10s #success=1 #failure=3
Environment:
BACKEND_HOST: blog-backend-service
BACKEND_PORT: 5000
Mounts: <none>
Volumes: <none>
Node-Selectors: <none>
Tolerations: <none>
Conditions:
Type Status Reason
Progressing True NewReplicaSetAvailable
Available True MinimumReplicasAvailable
OldReplicaSets: blog-frontend-64d6ccb995 (0/0 replicas created), blog-frontend-6fc556d84f (0/0 replicas created)
NewReplicaSet: blog-frontend-65968ccfff (2/2 replicas created)
Events:
Type Reason Age From Message
Normal ScalingReplicaSet 25m deployment-controller Scaled up replica set blog-frontend-6fc556d84f from 1 to 2
Normal ScalingReplicaSet 25m deployment-controller Scaled up replica set blog-frontend-65968ccfff from 0 to 1
Normal ScalingReplicaSet 17m deployment-controller Scaled down replica set blog-frontend-6fc556d84f from 2 to 1
Normal ScalingReplicaSet 17m deployment-controller Scaled down replica set blog-frontend-6fc556d84f from 1 to 0
Normal ScalingReplicaSet 15m deployment-controller Scaled down replica set blog-frontend-65968ccfff from 2 to 1
Normal ScalingReplicaSet 14m (x2 over 17m) deployment-controller Scaled up replica set blog-frontend-65968ccfff from 1 to 2
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Describe what to build
Outline
Timeline
main*
Priya (14 hours ago) Ln 9, Col 25 Spaces: 2 UTF-8 CRLF YAML


File Edit Selection View Go Run Terminal Help
Project - 4
Explorer
Project - 4
BlogReact
backend
database
init.sql
frontend
k8s
backend-deployment.ya... M
backend-service.yaml
frontend-deployment.ya... M
frontend-service.yaml
mysql-network-policy.yaml
mysql-pvc.yaml
mysql-secret.yaml
mysql-service.yaml
mysql-statefulset.yaml
.gitattributes
.gitignore
Jenkinsfile
README.md
sonar-project.properties
Outline
Timeline
Problems Output Debug Console Terminal Ports
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$ kubectl describe deployment blog-backend
Name: blog-backend
Namespace: default
CreationTimestamp: Mon, 21 Sep 2026 16:22:43 +0530
Labels: <none>
Annotations: deployment.kubernetes.io/revision: 2
Selector: app=blog-backend
Replicas: 2 desired | 2 updated | 2 total | 2 available | 0 unavailable
StrategyType: RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 0 max unavailable, 1 max surge
Pod Template:
Labels: app=blog-backend
Containers:
backend:
Image: 196253396965.dkr.ecr.ap-south-1.amazonaws.com/blog-backend:tag1
Port: 5000/TCP
Host Port: 0/TCP
Limits:
cpu: 500m
memory: 512Mi
Requests:
cpu: 100m
memory: 128Mi
Liveness: http-get http://:5000/healthz delay=20s timeout=1s period=20s #success=1 #failure=3
Readiness: http-get http://:5000/readyz delay=10s timeout=1s period=10s #success=1 #failure=3
Environment:
PORT: 5000
DB_HOST: mysql-service
DB_PORT: 3306
DB_USER: <set to the key 'MYSQL_USER' in secret 'mysql-secret'> Optional: false
DB_PASSWORD: <set to the key 'MYSQL_PASSWORD' in secret 'mysql-secret'> Optional: false
DB_NAME: <set to the key 'MYSQL_DATABASE' in secret 'mysql-secret'> Optional: false
Mounts: <none>
Volumes: <none>
Node-Selectors: <none>
Tolerations: <none>
Conditions:
Type Status Reason
Available True MinimumReplicasAvailable
Progressing True NewReplicaSetAvailable
OldReplicaSets: blog-backend-6bccfdbd85 (0/0 replicas created)
NewReplicaSet: blog-backend-58746c7cd8 (2/2 replicas created)
Events:
Type Reason Age From Message
Normal ScalingReplicaSet 32m deployment-controller Scaled up replica set blog-backend-6bccfdbd85 from 1 to 2
Normal ScalingReplicaSet 32m deployment-controller Scaled up replica set blog-backend-58746c7cd8 from 0 to 1
Normal ScalingReplicaSet 32m deployment-controller Scaled down replica set blog-backend-6bccfdbd85 from 2 to 1
Normal ScalingReplicaSet 32m deployment-controller Scaled up replica set blog-backend-58746c7cd8 from 1 to 2
Normal ScalingReplicaSet 31m deployment-controller Scaled down replica set blog-backend-6bccfdbd85 from 1 to 0
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Describe what to build
Local
main*
Priya (14 hours ago) Ln 9, Col 25 Spaces: 2 UTF-8 CRLF YAML

Task 9 : Configuration & Secrets

Overview

Application configuration was separated from the application images using Kubernetes ConfigMaps and Secrets.

ConfigMap

A Kubernetes ConfigMap named `mysql-initdb` was used to store the MySQL database initialization script.

The ConfigMap provides the database initialization script separately from the application images.

Kubernetes Secret

A Kubernetes Secret named `mysql-secret` was created to store sensitive MySQL database credentials.

The Backend and MySQL workloads use the Secret to obtain the required database credentials.

Secret Configuration

The Backend Deployment uses `secretKeyRef` to retrieve:

- Database username
- Database password
- Database name

This keeps the sensitive database values separate from the Backend Deployment configuration.

Git Protection

The Secret manifest was excluded from Git tracking using .gitignore: `k8s/mysql-secret.yaml`

The following command was used to verify that the Secret manifest is not tracked by Git:

```
git ls-files k8s/mysql-secret.yaml
```

No output was returned.

Verification

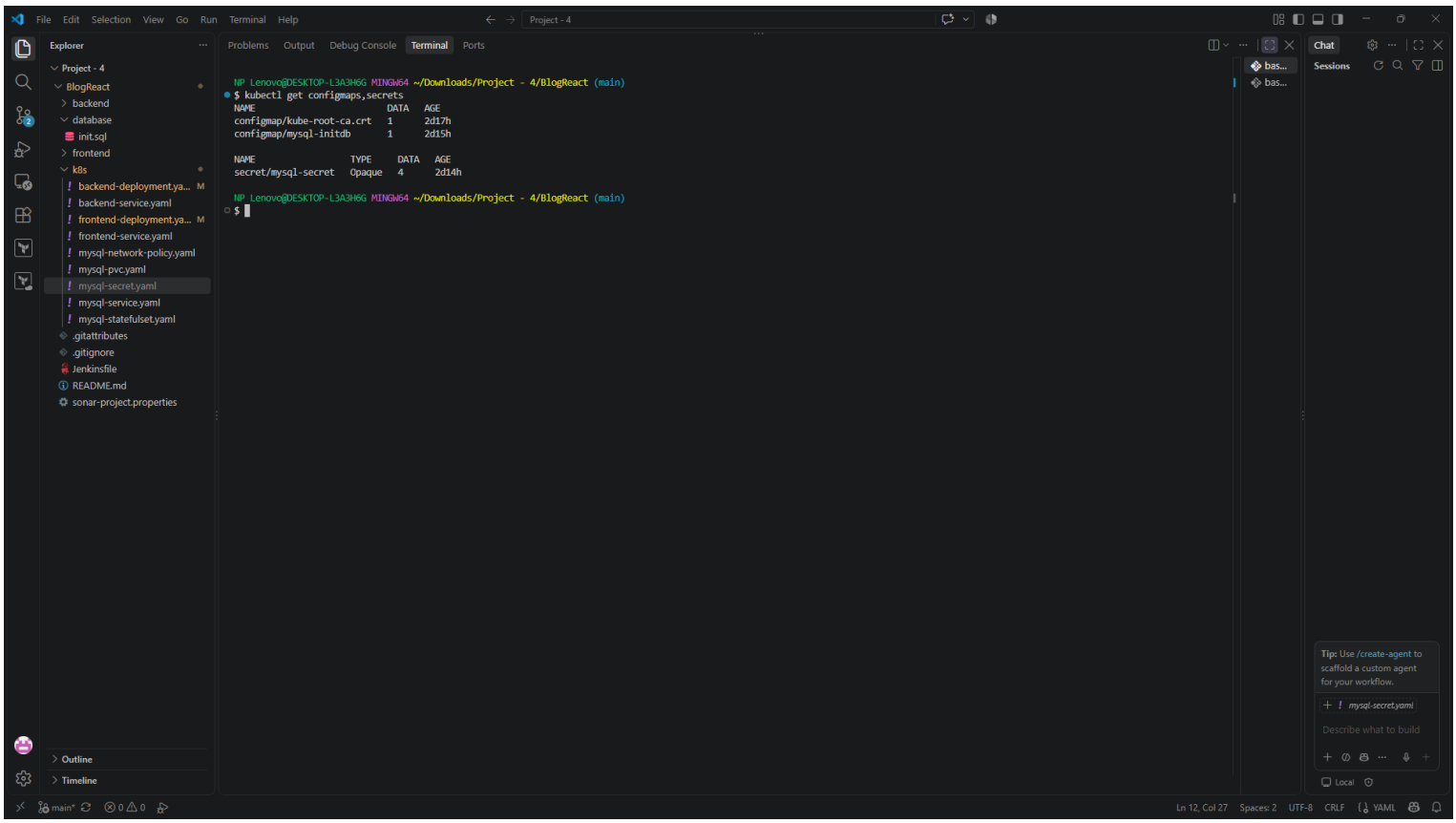
The ConfigMap and Secret were verified in the EKS cluster.

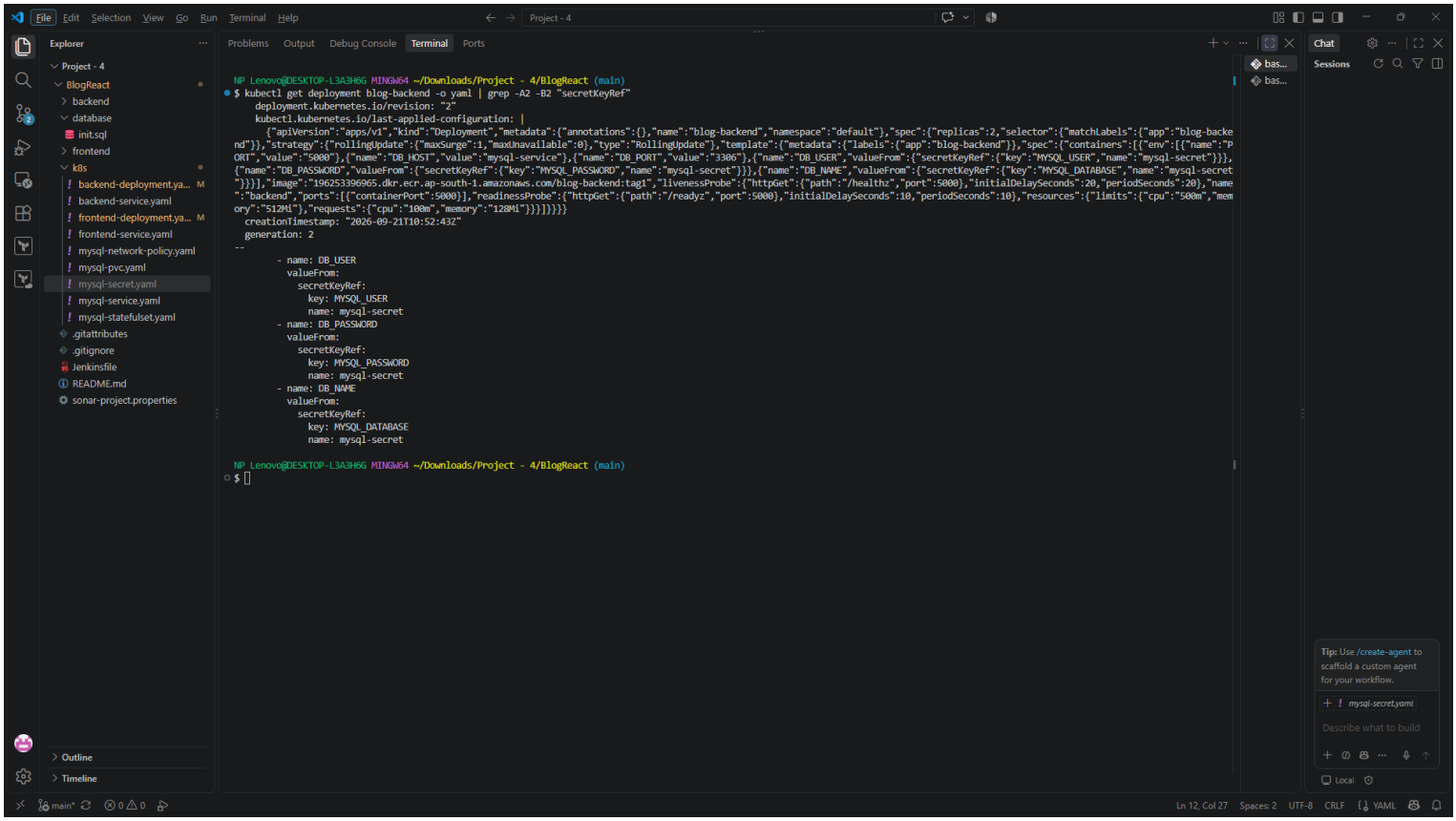
The Backend Deployment was verified to use secretKeyRef for the database credentials.

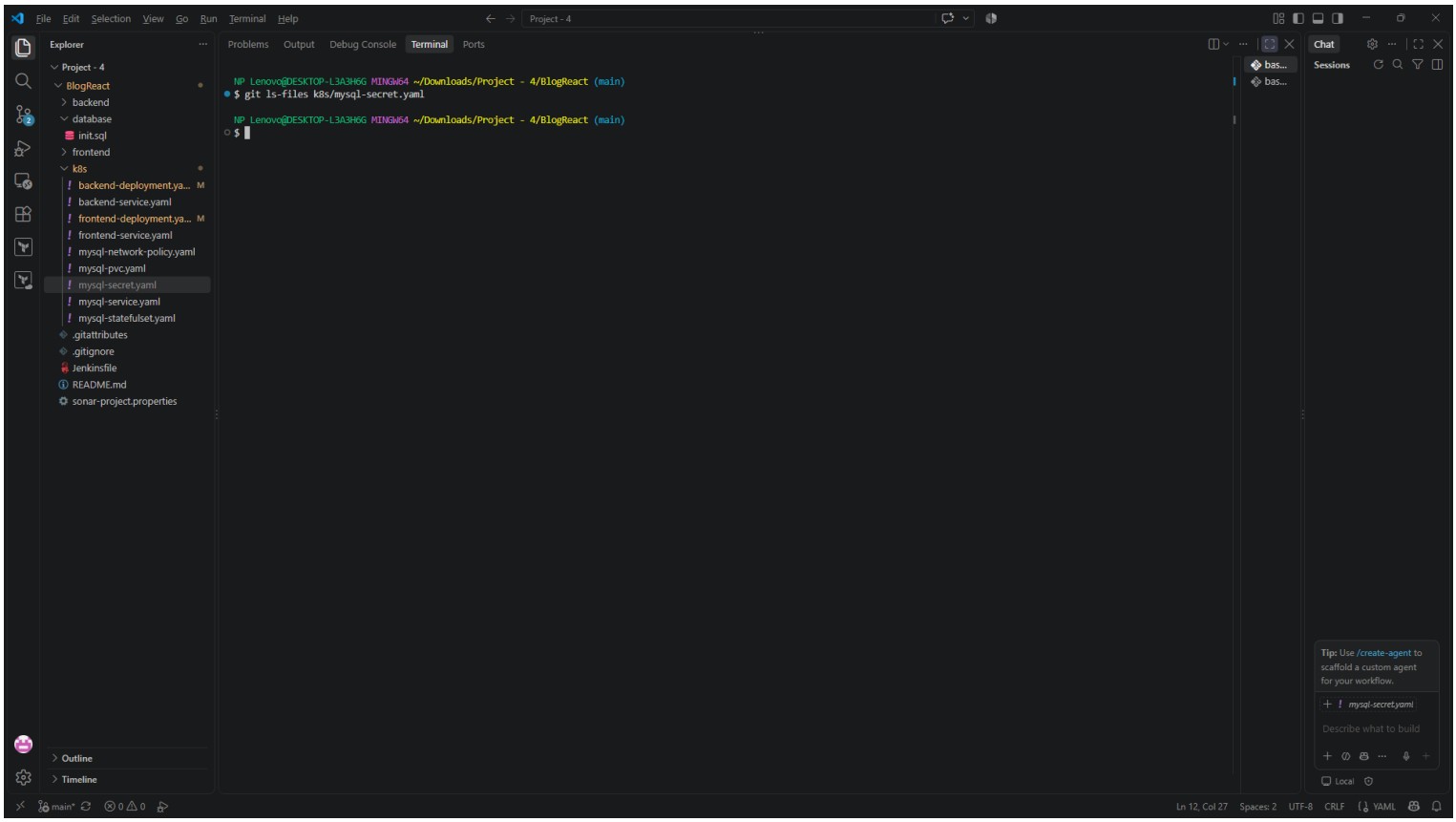
Git tracking was also verified to ensure that k8s/mysql-secret.yaml was not included in the Git repository.

Result

The application configuration was successfully separated from the application images. Non-sensitive configuration was handled using a ConfigMap, while sensitive database credentials were managed using a Kubernetes Secret and the Secret manifest was excluded from Git tracking.




File Edit Selection View Go Run Terminal Help
Project - 4
Explorer
Project - 4
BlogReact
backend
database
init.sql
frontend
k8s
backend-deployment.ya... M
backend-service.yaml
frontend-deployment.ya... M
frontend-service.yaml
mysql-network-policy.yaml
mysql-pvc.yaml
mysql-secret.yaml
mysql-service.yaml
mysql-statefulset.yaml
.gitattributes
.gitignore
Jenkinsfile
README.md
sonar-project.properties
Problems Output Debug Console Terminal Ports
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$ kubectl get deployment blog-backend -o yaml | grep -A2 -B2 "secretKeyRef"
deployment.kubernetes.io/revision: "2"
kubectl.kubernetes.io/last-applied-configuration: |
{"apiVersion":"apps/v1","kind":"Deployment","metadata":{"annotations":{},"name":"blog-backend","namespace":"default"},"spec":{"replicas":2,"selector":{"matchLabels":{"app":"blog-backend"}},"strategy":{"rollingUpdate":{"maxSurge":1,"maxUnavailable":0},"type":"RollingUpdate"},"template":{"metadata":{"labels":{"app":"blog-backend"}},"spec":{"containers":[{"env":[{"name":"PORT","value":"5000"},{"name":"DB_HOST","value":"mysql-service"},{"name":"DB_PORT","value":"3306"},{"name":"DB_USER","valueFrom":{"secretKeyRef":{"key":"MYSQL_USER","name":"mysql-secret"}}},{"name":"DB_PASSWORD","valueFrom":{"secretKeyRef":{"key":"MYSQL_PASSWORD","name":"mysql-secret"}}},{"name":"DB_NAME","valueFrom":{"secretKeyRef":{"key":"MYSQL_DATABASE","name":"mysql-secret"}}}],"image":"196253396965.dkr.ecr.ap-south-1.amazonaws.com/blog-backend:tag1","livenessProbe":{"httpGet":{"path":"/healthz","port":5000},"initialDelaySeconds":20,"periodSeconds":20},"name":"backend","ports":[{"containerPort":5000}],"readinessProbe":{"httpGet":{"path":"/readyz","port":5000},"initialDelaySeconds":10,"periodSeconds":10},"resources":{"limits":{"cpu":"500m","memory":"512Mi"},"requests":{"cpu":"100m","memory":"128Mi"}}}]}}}}
creationTimestamp: "2026-09-21T10:52:43Z"
generation: 2
--
- name: DB_USER
valueFrom:
secretKeyRef:
key: MYSQL_USER
name: mysql-secret
- name: DB_PASSWORD
valueFrom:
secretKeyRef:
key: MYSQL_PASSWORD
name: mysql-secret
- name: DB_NAME
valueFrom:
secretKeyRef:
key: MYSQL_DATABASE
name: mysql-secret
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
mysql-secret.yaml
Describe what to build
Local
Outline
Timeline
main*
Ln 12, Col 27
Spaces: 2
UTF-8
CRLF
YAML


File Edit Selection View Go Run Terminal Help
Project - 4
Explorer
Project - 4
BlogReact
backend
database
init.sql
frontend
k8s
backend-deployment.ya... M
backend-service.yaml
frontend-deployment.ya... M
frontend-service.yaml
mysql-network-policy.yaml
mysql-pvc.yaml
mysql-secret.yaml
mysql-service.yaml
mysql-statefulset.yaml
.gitattributes
.gitignore
Jenkinsfile
README.md
sonar-project.properties
Problems Output Debug Console Terminal Ports
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$ git ls-files k8s/mysql-secret.yaml
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
mysql-secret.yaml
Describe what to build
Local
Outline
Timeline
main*
Ln 12, Col 27
Spaces: 2
UTF-8
CRLF
YAML

Task 10 : CI/CD Pipeline

Overview

Implemented an automated Jenkins CI/CD pipeline to build, validate, scan, publish, and deploy the Blog Application from GitHub to Amazon EKS.

Pipeline Stages

The pipeline includes:

1. Checkout – Retrieves source code from GitHub.
2. Build – Installs Frontend and Backend dependencies and builds the application.
3. Test – Runs the available application test commands.
4. SonarQube Analysis – Performs code-quality analysis.
5. Quality Gate – Validates the SonarQube Quality Gate before proceeding.
6. Docker Build – Builds separate Frontend and Backend images.
7. Trivy Scan – Scans both images for vulnerabilities before deployment.
8. Image Versioning – Uses Jenkins BUILD_NUMBER for unique image tags.
9. ECR Push – Pushes versioned images to Amazon ECR.
10. EKS Deployment – Deploys the updated images to Frontend and Backend workloads on EKS.
11. Deployment Validation – Verifies Deployments, Pods, and Services.

Automation

A GitHub Webhook was configured to trigger Jenkins automatically whenever code is pushed to the repository.

Evidence

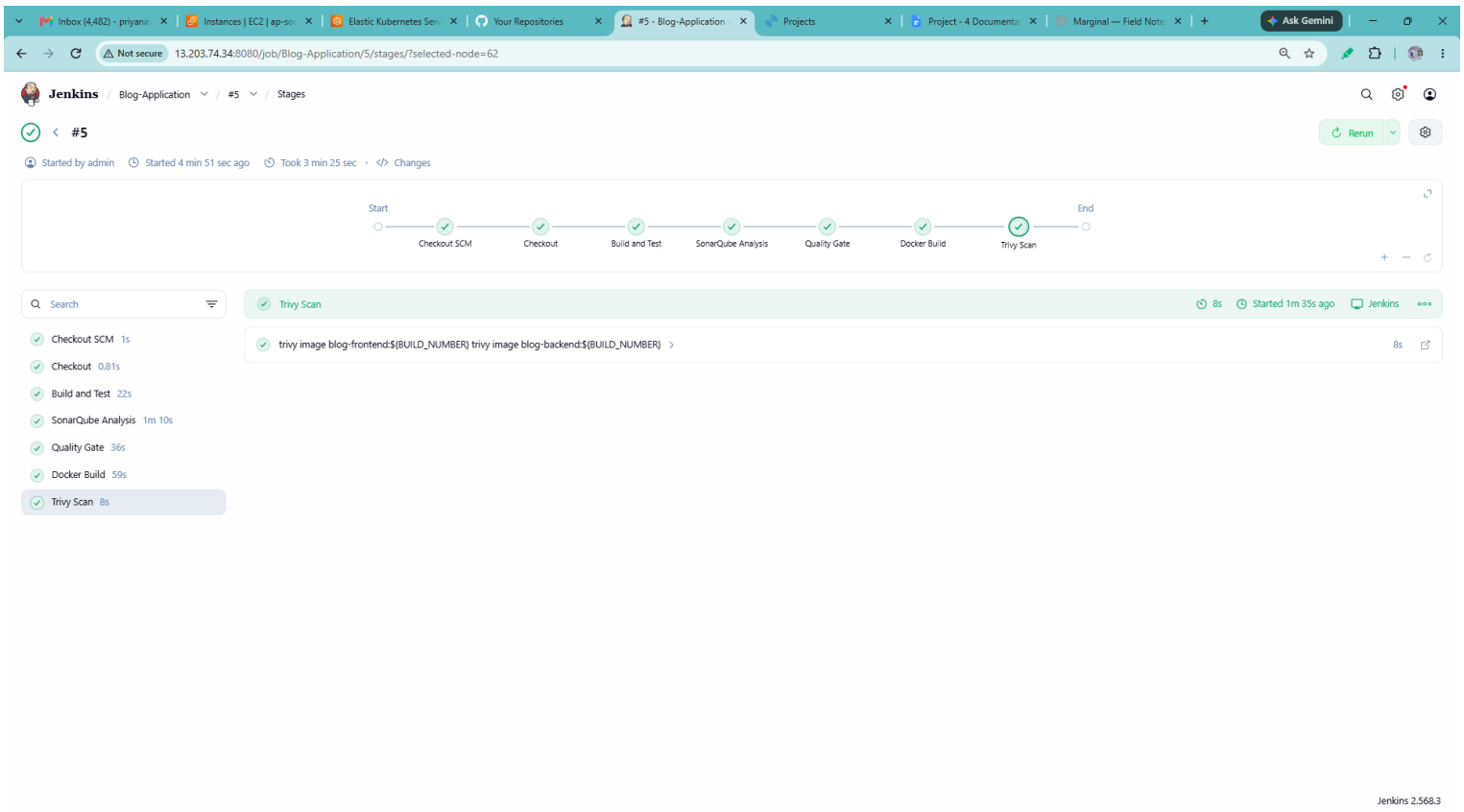
Jenkins **Build #10** was automatically triggered by a GitHub push and successfully completed the complete CI/CD pipeline :

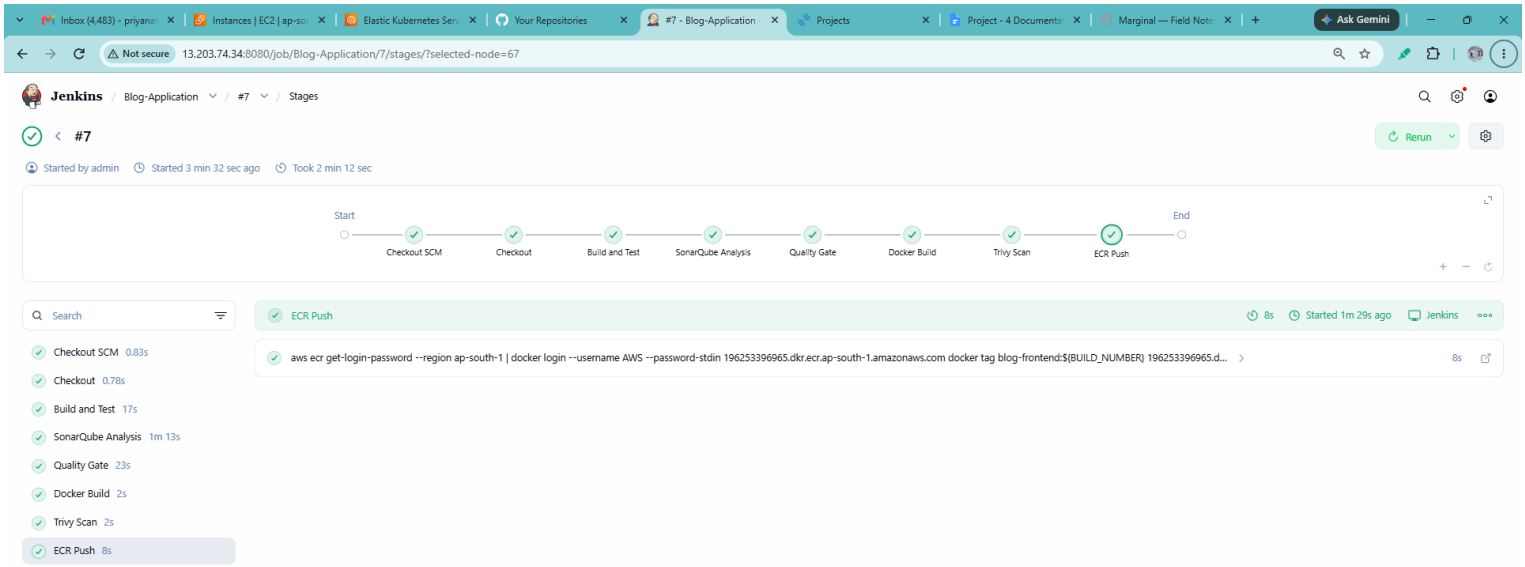
Checkout → Build → Test → SonarQube → Quality Gate → Docker Build → Trivy → ECR Push → EKS Deployment → Validation

Final Status: SUCCESS ✅

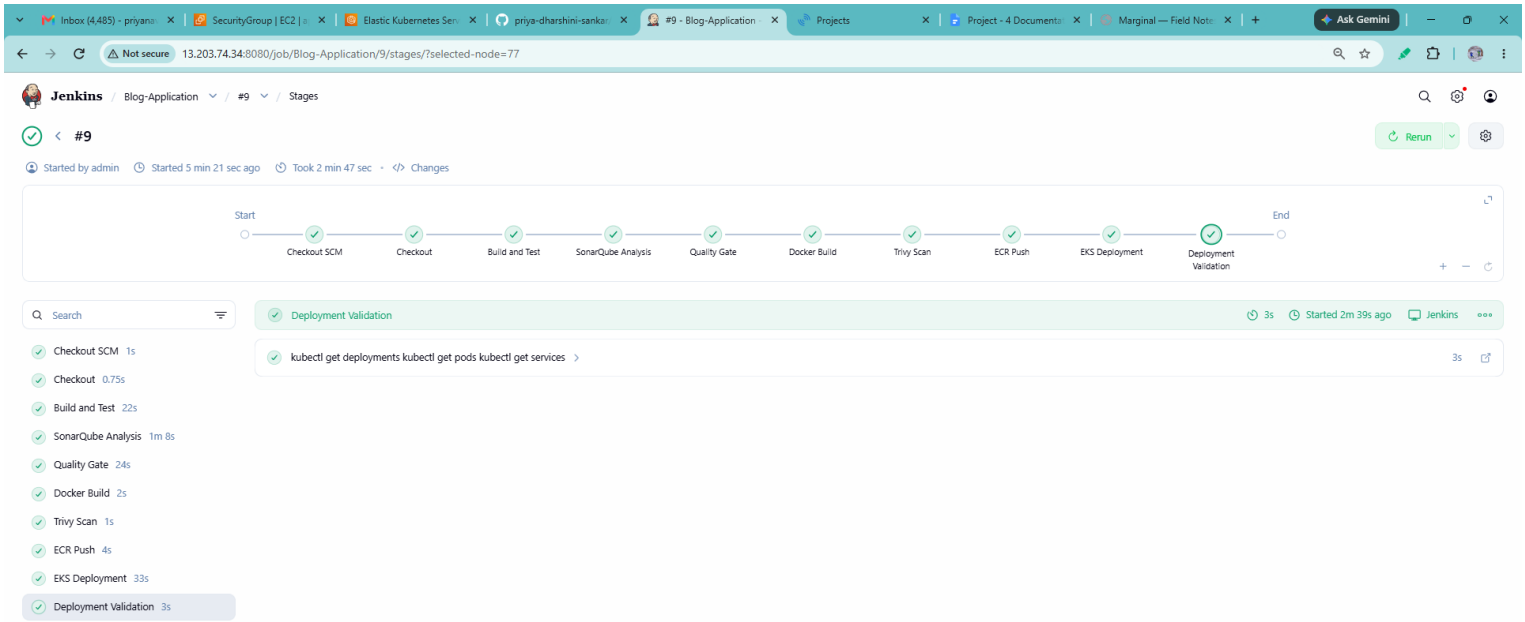
Result

The Blog Application now has an automated CI/CD workflow that performs code checkout, build, testing, quality validation, security scanning, versioned image publishing, EKS deployment, and post-deployment validation.

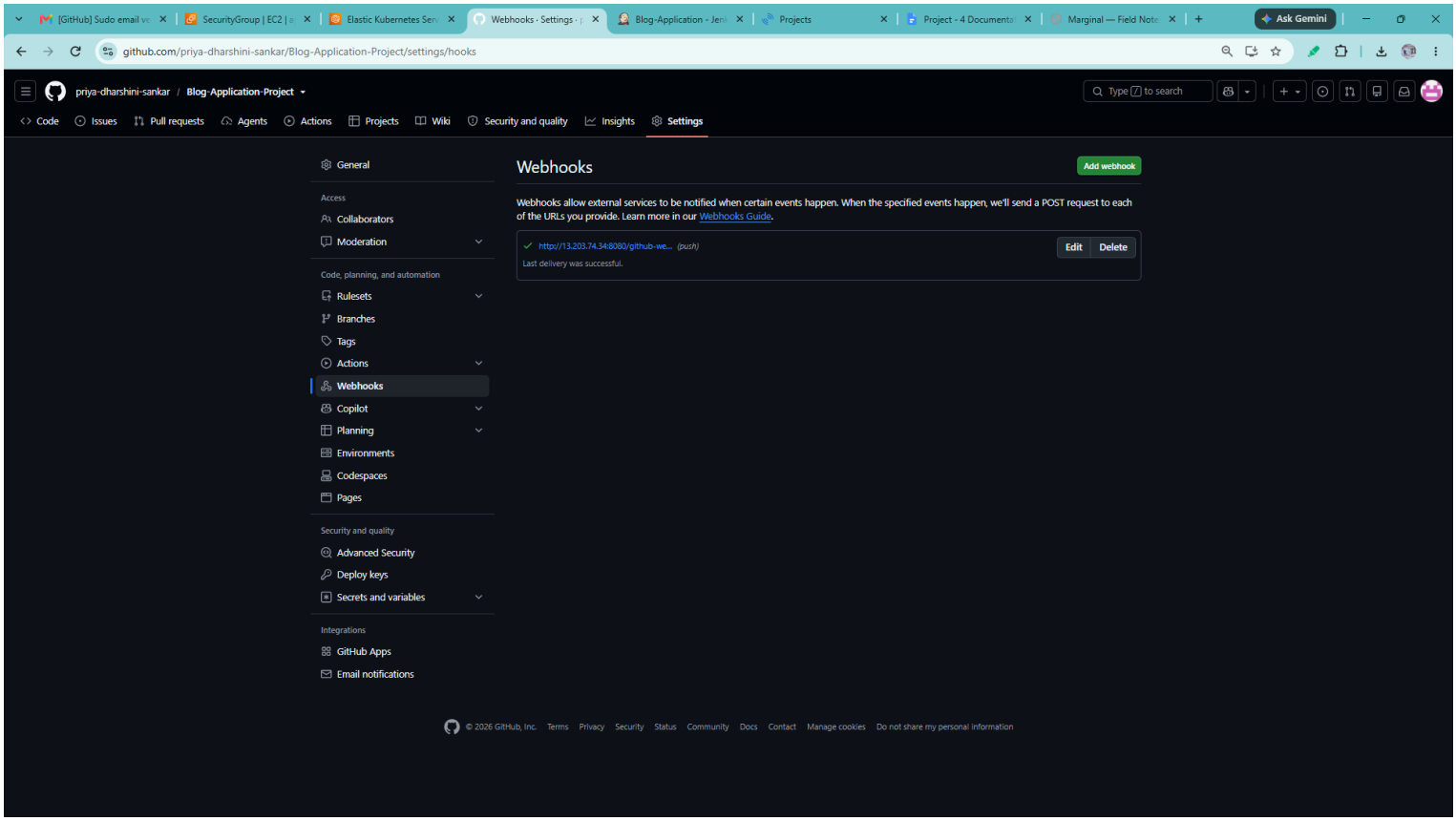


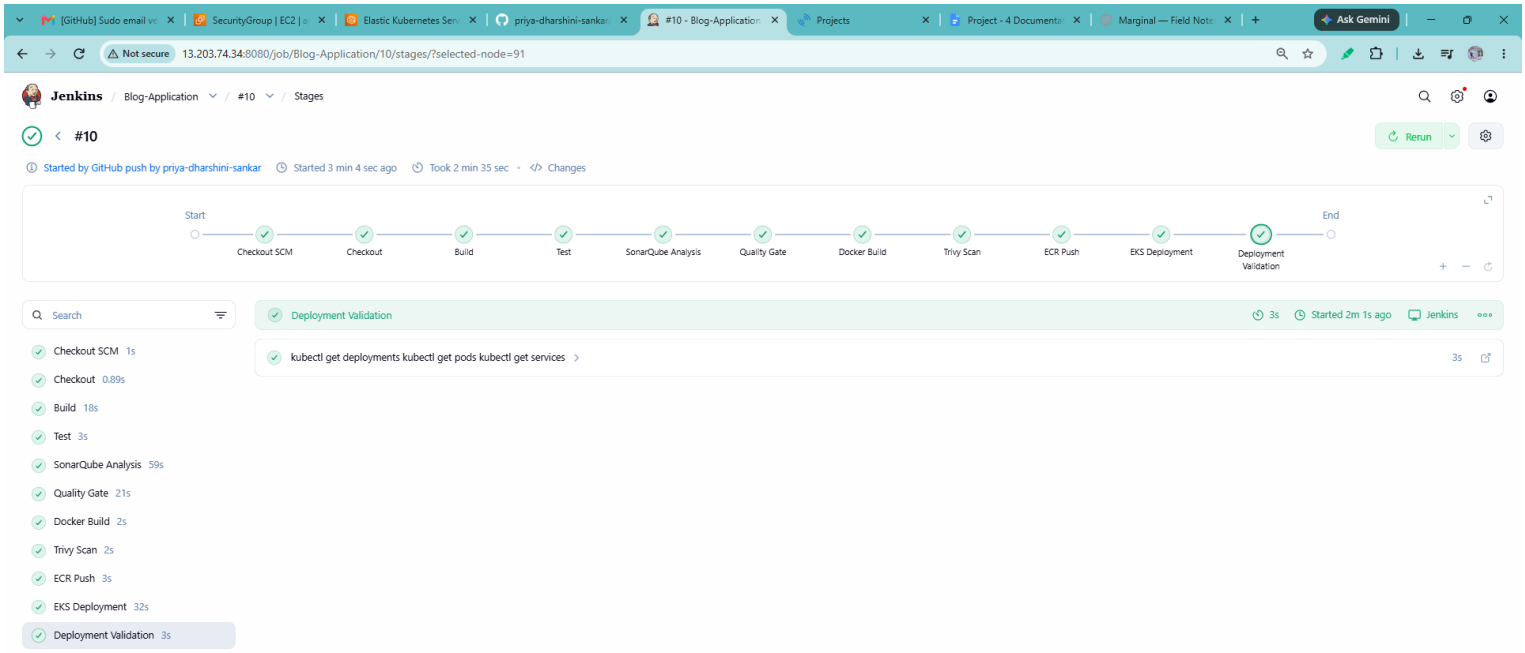


Jenkins 2.568.3



Jenkins 2.568.3


github.com/priya-dharshini-sankar/Blog-Application-Project/settings/hooks
priya-dharshini-sankar / Blog-Application-Project
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
General
Access
Collaborators
Moderation
Code, planning, and automation
Rulesets
Branches
Tags
Actions
Webhooks
Copilot
Planning
Environments
Codespaces
Pages
Security and quality
Advanced Security
Deploy keys
Secrets and variables
Integrations
GitHub Apps
Email notifications
Webhooks
Add webhook
Webhooks allow external services to be notified when certain events happen. When the specified events happen, we'll send a POST request to each of the URLs you provide. Learn more in our Webhooks Guide.
http://13.203.74.34:8080/github-we... (push)
Last delivery was successful.
Edit
Delete
© 2026 GitHub, Inc.
Terms
Privacy
Security
Status
Community
Docs
Contact
Manage cookies
Do not share my personal information


Not secure
13.203.74.34:8080/job/Blog-Application/10/stages/?selected-node=91
Jenkins
Blog-Application
#10
Stages
#10
Rerun
Started by GitHub push by priya-dharshini-sankar
Started 3 min 4 sec ago
Took 2 min 35 sec
Changes
Start
Checkout SCM
Checkout
Build
Test
SonarQube Analysis
Quality Gate
Docker Build
Trivy Scan
ECR Push
EKS Deployment
Deployment Validation
End
Search
Checkout SCM 1s
Checkout 0.89s
Build 18s
Test 3s
SonarQube Analysis 59s
Quality Gate 21s
Docker Build 2s
Trivy Scan 2s
ECR Push 3s
EKS Deployment 32s
Deployment Validation 3s
Deployment Validation
3s
Started 2m 1s ago
Jenkins
kubectl get deployments kubectl get pods kubectl get services
3s
Jenkins 2.568.3

Task 11 : Deployment Strategy

Overview

A Kubernetes RollingUpdate strategy was implemented for the Frontend and Backend workloads to deploy new application versions while maintaining application availability.

Rolling Update Configuration

The Frontend and Backend Deployments were configured with:

- 2 replicas
- RollingUpdate strategy
- maxUnavailable: 0
- maxSurge: 1
- Readiness probes
- Liveness probes

Deployment

A new Frontend application version was committed to GitHub. The GitHub push automatically triggered Jenkins Build #11.

The pipeline successfully completed:

Checkout → Build → Test → SonarQube → Quality Gate → Docker Build → Trivy Scan → ECR Push → EKS Deployment → Validation

During the deployment, Kubernetes created new pods while the previous pods were being terminated, demonstrating the RollingUpdate process.

Verification

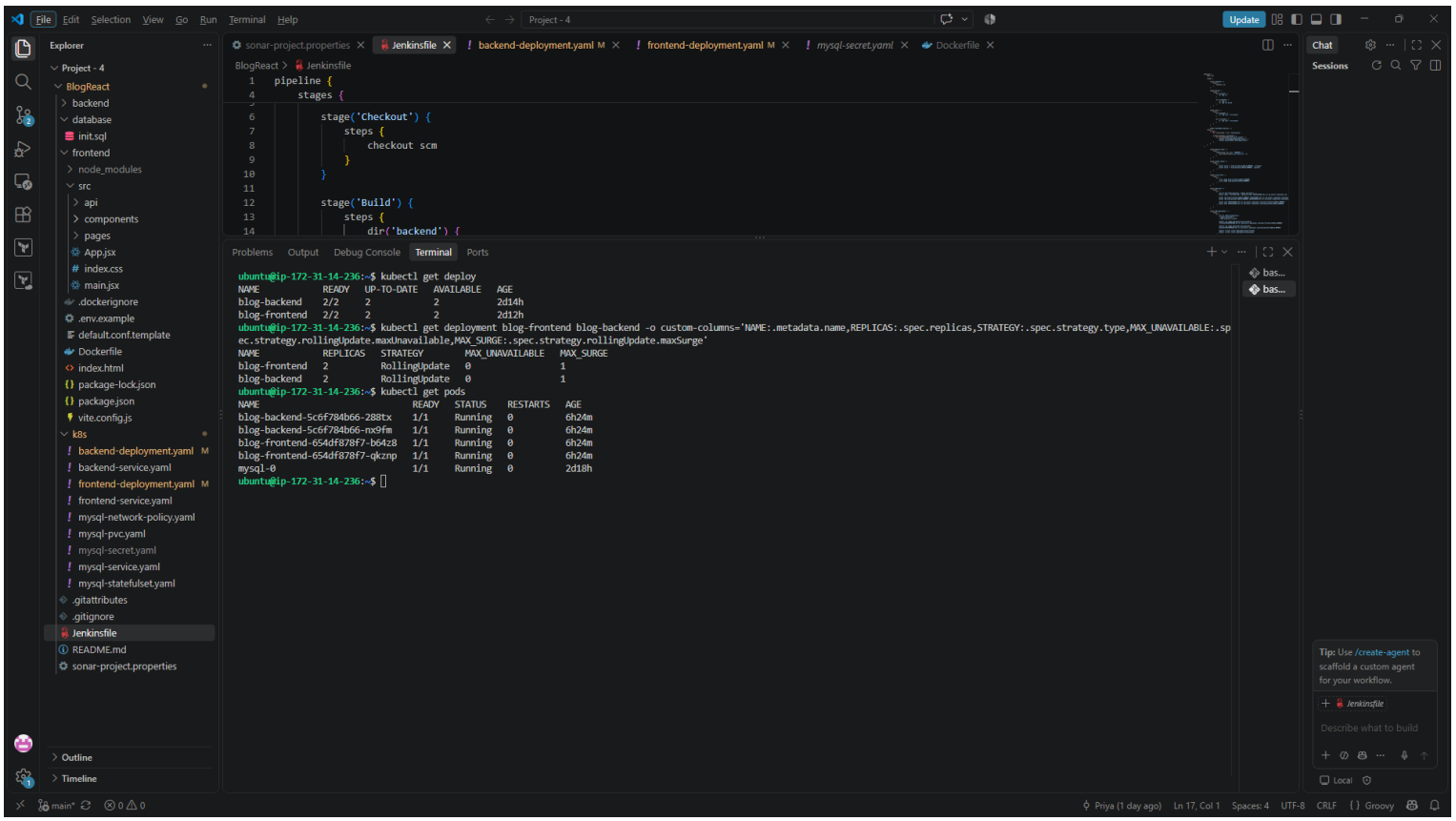
The final Kubernetes deployment status was:

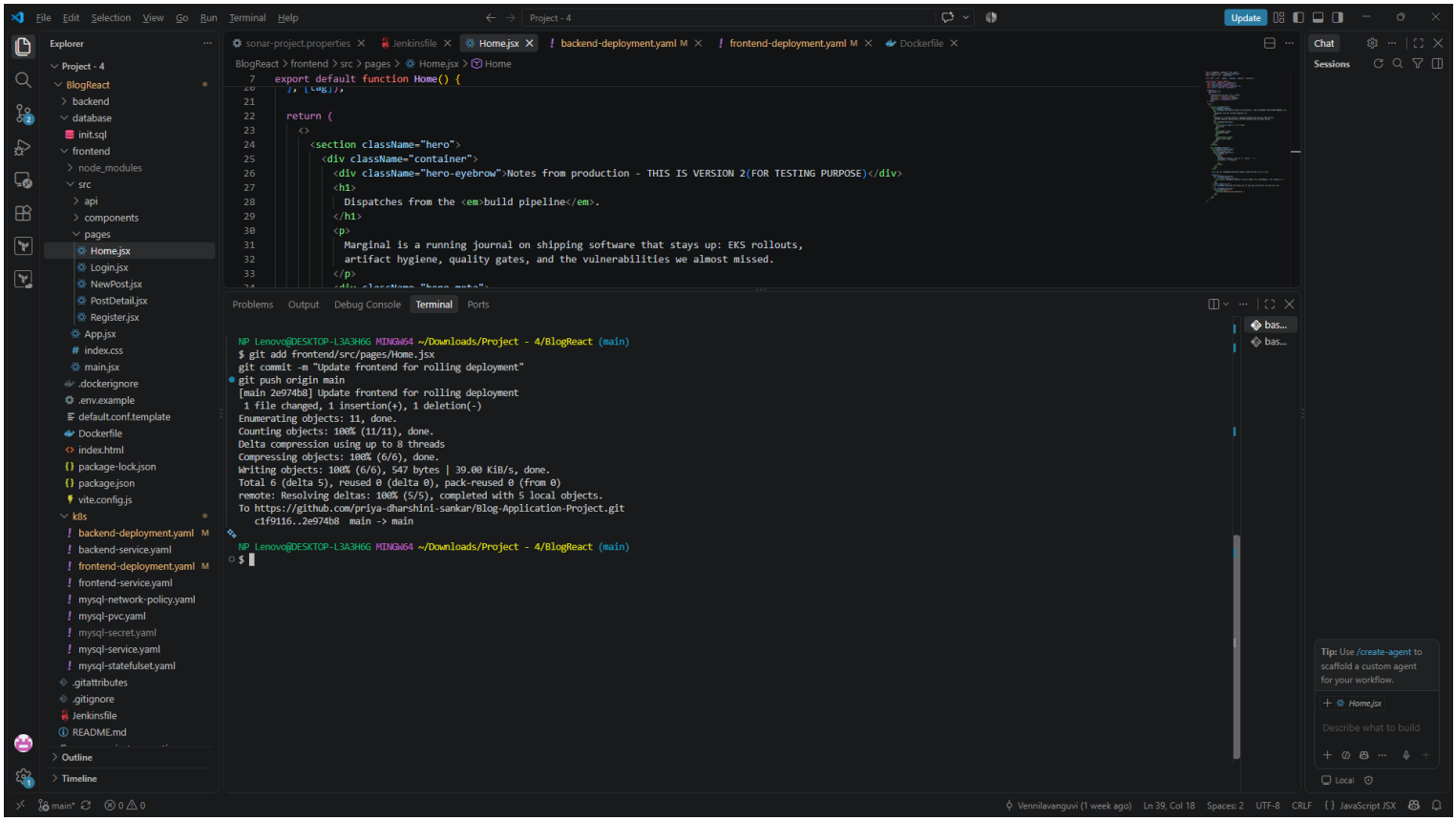
blog-backend 2/2 2 2
blog-frontend 2/2 2 2

The deployment validation also showed new pods in Running state while previous pods were being terminated during the rollout.

Result

The Frontend and Backend application versions were successfully deployed using Kubernetes RollingUpdate.The required Frontend and Backend replicas were available after the rolling deployment, and the final deployment state was verified successfully.

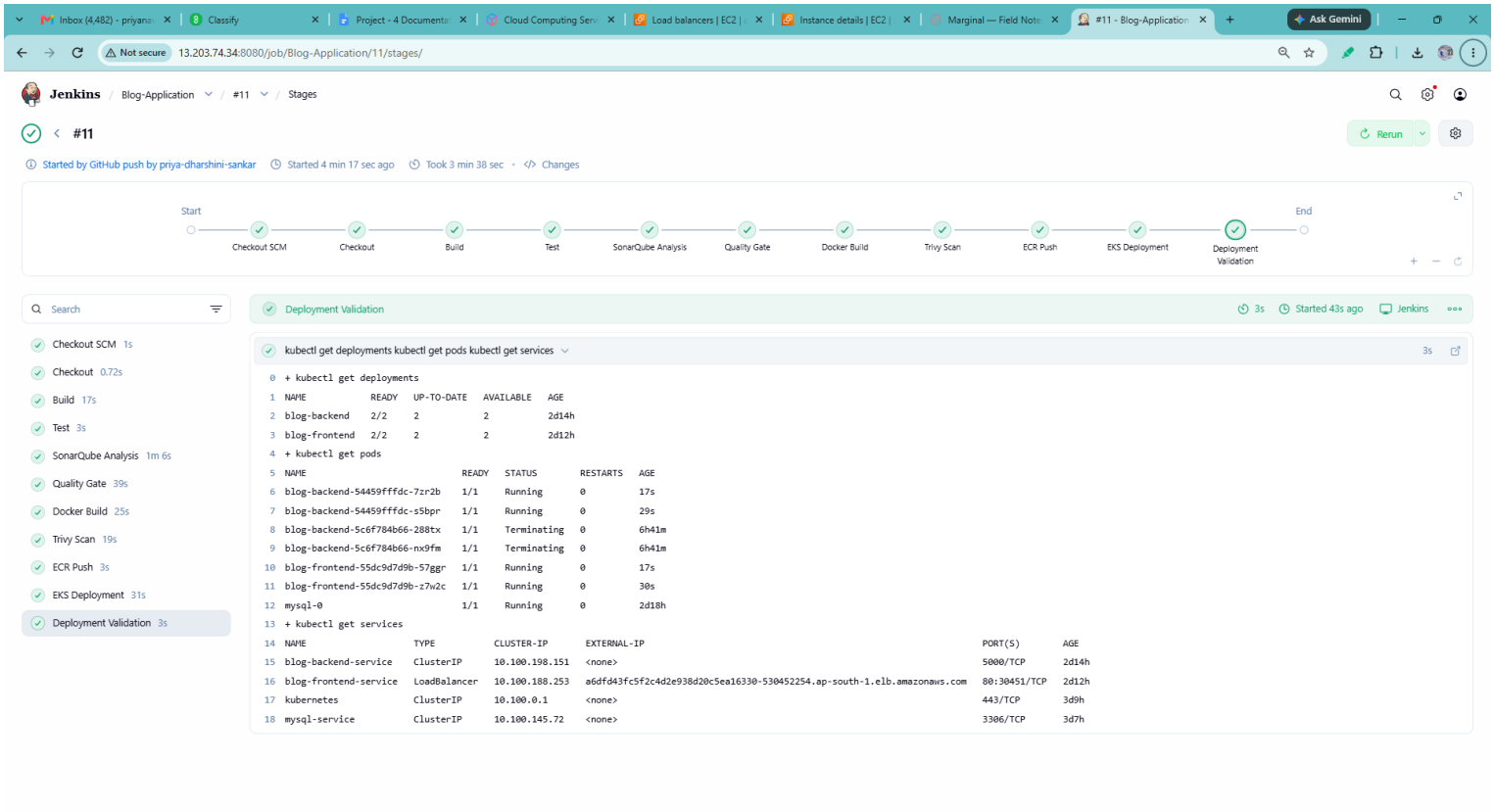


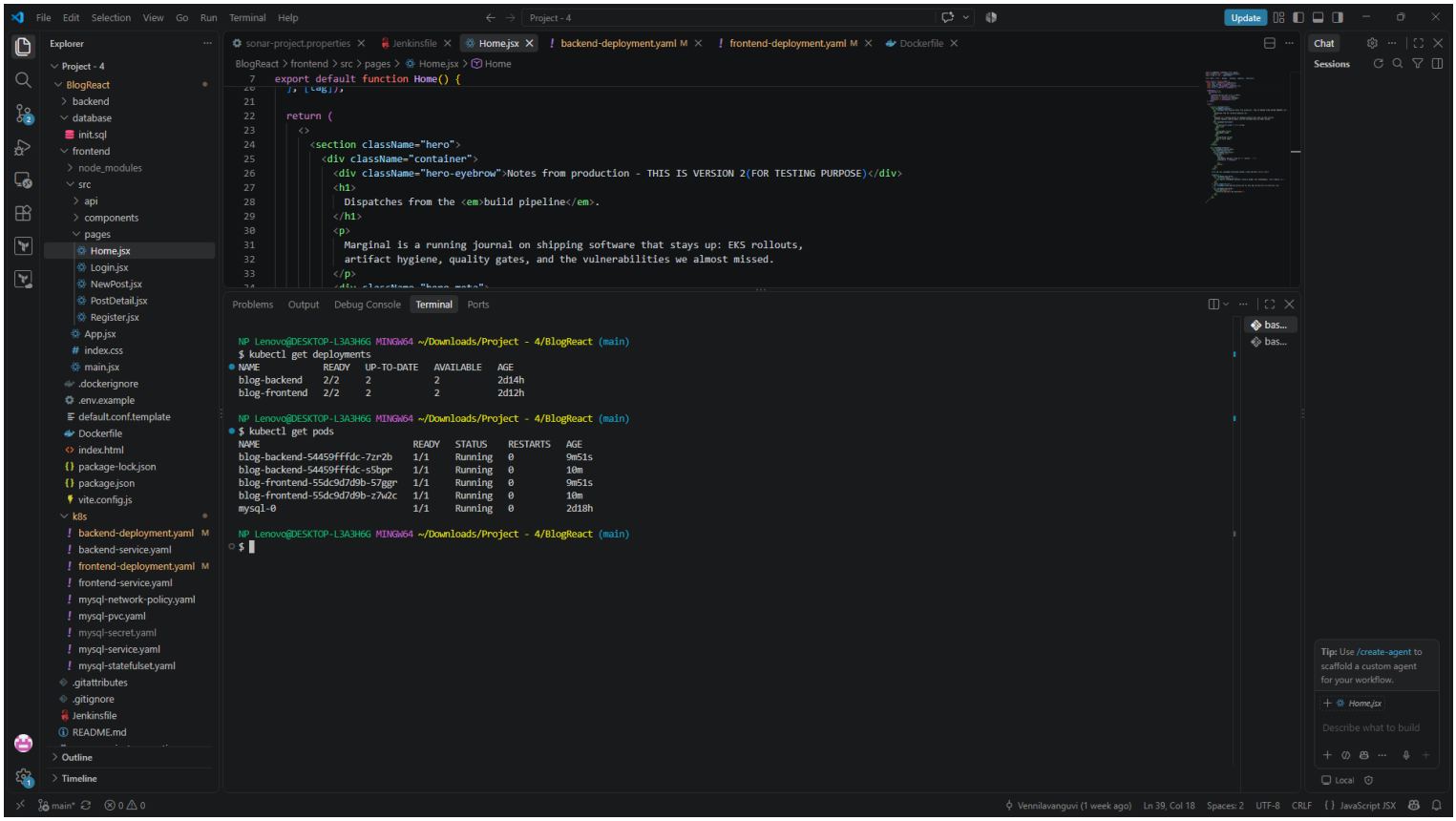

File Edit Selection View Go Run Terminal Help
Project - 4
Update
Explorer
Project - 4
BlogReact
backend
database
init.sql
frontend
node_modules
src
api
components
pages
Home.jsx
Login.jsx
NewPost.jsx
PostDetail.jsx
Register.jsx
App.jsx
index.css
main.jsx
.dockerignore
.env.example
default.conf.template
Dockerfile
index.html
package-lock.json
package.json
vite.config.js
k8s
backend-deployment.yaml M
backend-service.yaml
frontend-deployment.yaml M
frontend-service.yaml
mysql-network-policy.yaml
mysql-pvc.yaml
mysql-secret.yaml
mysql-service.yaml
mysql-statefulset.yaml
.gitattributes
.gitignore
Jenkinsfile
README.md
Outline
Timeline
sonar-project.properties
Jenkinsfile
Home.jsx
backend-deployment.yaml M
frontend-deployment.yaml M
Dockerfile
BlogReact > frontend > src > pages > Home.jsx > Home
export default function Home() {
}, [tags]);
return (
<>
<section className="hero">

Notes from production - THIS IS VERSION 2(FOR TESTING PURPOSE)
<h1>
Dispatches from the build pipeline.
</h1>

Marginal is a running journal on shipping software that stays up: EKS rollouts,
artifact hygiene, quality gates, and the vulnerabilities we almost missed.

Problems Output Debug Console Terminal Ports
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$ git add frontend/src/pages/Home.jsx
git commit -m "Update frontend for rolling deployment"
git push origin main
[main 2e974b8] Update frontend for rolling deployment
1 file changed, 1 insertion(+), 1 deletion(-)
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 547 bytes | 39.00 KiB/s, done.
Total 6 (delta 5), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (5/5), completed with 5 local objects.
To https://github.com/priya-dharshini-sankar/Blog-Application-Project.git
c1f9116..2e974b8 main -> main
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Describe what to build
Local
main*
Vennilavanguvi (1 week ago)
Ln 39, Col 18
Spaces: 2
UTF-8
CRLF
JavaScript JSX

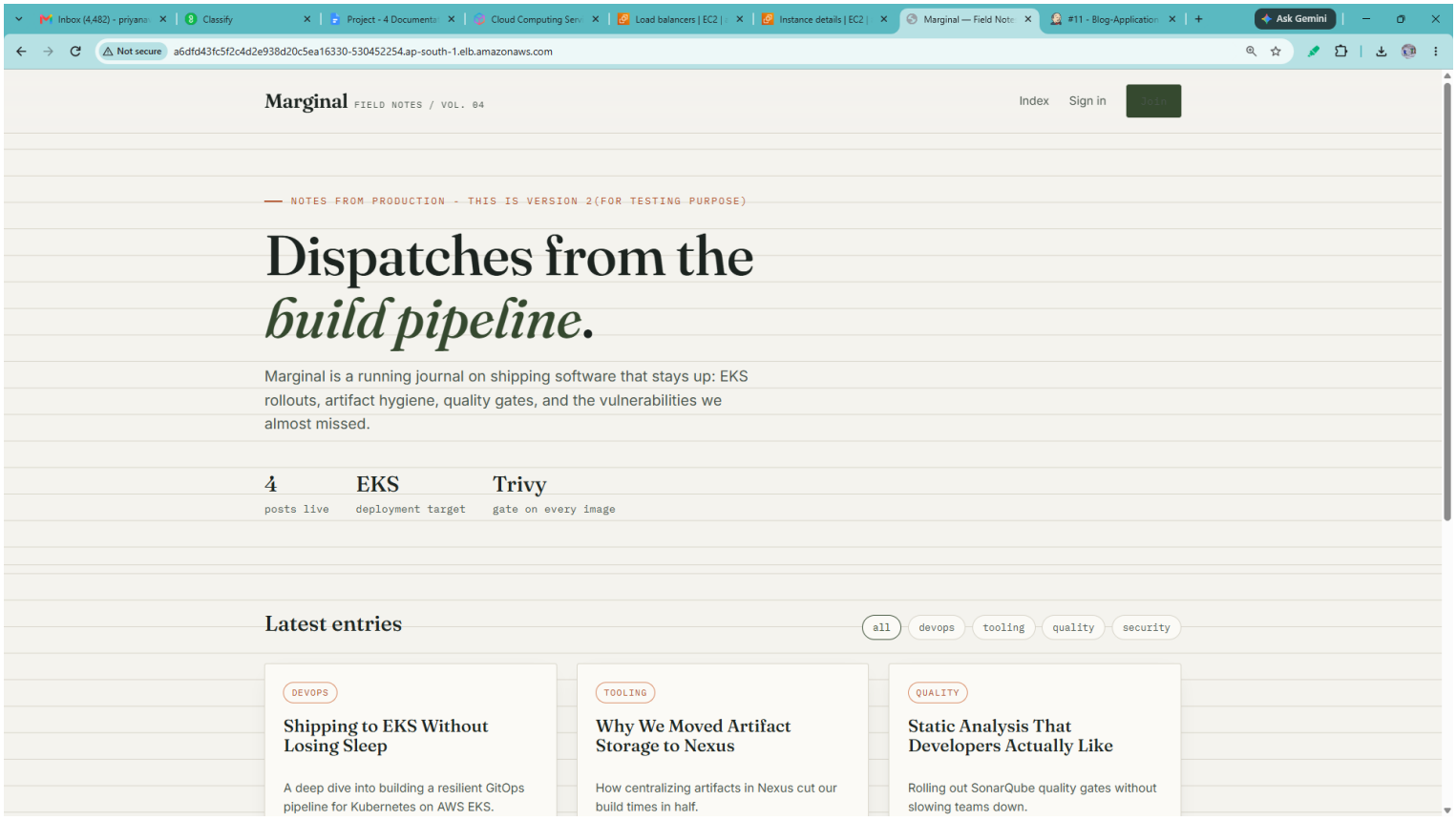

Inbox (4,482) - priyan
Classify
Project - 4 Documenta
Cloud Computing Serv
Load balancers | EC2 |
Instance details | EC2 |
Marginal — Field Note
#11 - Blog-Application
Ask Gemini
Not secure
13.203.74.34:8080/job/Blog-Application/11/stages/
Jenkins
Blog-Application
#11
Stages
#11
Rerun
Started by GitHub push by priya-dharshini-sankar
Started 4 min 17 sec ago
Took 3 min 38 sec
Changes
Start
Checkout SCM
Checkout
Build
Test
SonarQube Analysis
Quality Gate
Docker Build
Trivy Scan
ECR Push
EKS Deployment
Deployment Validation
End
Search
Checkout SCM 1s
Checkout 0.72s
Build 17s
Test 3s
SonarQube Analysis 1m 6s
Quality Gate 39s
Docker Build 25s
Trivy Scan 19s
ECR Push 3s
EKS Deployment 31s
Deployment Validation 3s
Deployment Validation
3s
Started 43s ago
Jenkins
kubectl get deployments kubectl get pods kubectl get services
3s
0 + kubectl get deployments
1 NAME READY UP-TO-DATE AVAILABLE AGE
2 blog-backend 2/2 2 2 2d14h
3 blog-frontend 2/2 2 2 2d12h
4 + kubectl get pods
5 NAME READY STATUS RESTARTS AGE
6 blog-backend-54459fffdc-7zr2b 1/1 Running 0 17s
7 blog-backend-54459fffdc-s5bpr 1/1 Running 0 29s
8 blog-backend-5c6f784b66-288tx 1/1 Terminating 0 6h41m
9 blog-backend-5c6f784b66-nx9fm 1/1 Terminating 0 6h41m
10 blog-frontend-55dc9d7d9b-57ggr 1/1 Running 0 17s
11 blog-frontend-55dc9d7d9b-z7w2c 1/1 Running 0 30s
12 mysql-0 1/1 Running 0 2d18h
13 + kubectl get services
14 NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
15 blog-backend-service ClusterIP 10.100.198.151 <none> 5000/TCP 2d14h
16 blog-frontend-service LoadBalancer 10.100.188.253 a6dfd43fc5f2c4d2e938d20c5ea16330-530452254.ap-south-1.elb.amazonaws.com 80:30451/TCP 2d12h
17 kubernetes ClusterIP 10.100.0.1 <none> 443/TCP 3d9h
18 mysql-service ClusterIP 10.100.145.72 <none> 3306/TCP 3d7h
Jenkins 2.568.3


File Edit Selection View Go Run Terminal Help
Project - 4
Update
Explorer
BlogReact
backend
database
init.sql
frontend
node_modules
src
api
components
pages
Home.jsx
Login.jsx
NewPost.jsx
PostDetail.jsx
Register.jsx
App.jsx
index.css
main.jsx
.dockerignore
.env.example
default.conf.template
Dockerfile
index.html
package-lock.json
package.json
vite.config.js
k8s
backend-deployment.yaml
backend-service.yaml
frontend-deployment.yaml
frontend-service.yaml
mysql-network-policy.yaml
mysql-pvc.yaml
mysql-secret.yaml
mysql-service.yaml
mysql-statefulset.yaml
.gitattributes
.gitignore
Jenkinsfile
README.md
Outline
Timeline
sonar-project.properties
Jenkinsfile
Home.jsx
backend-deployment.yaml M
frontend-deployment.yaml M
Dockerfile
BlogReact > frontend > src > pages > Home.jsx > Home
export default function Home() {
}, [tags]);
return (
<>
<section className="hero">

Notes from production - THIS IS VERSION 2(FOR TESTING PURPOSE)
<h1>
Dispatches from the build pipeline.
</h1>

Marginal is a running journal on shipping software that stays up: EKS rollouts,
artifact hygiene, quality gates, and the vulnerabilities we almost missed.

Problems Output Debug Console Terminal Ports
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$ kubectl get deployments
NAME READY UP-TO-DATE AVAILABLE AGE
blog-backend 2/2 2 2 2d14h
blog-frontend 2/2 2 2 2d12h
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$ kubectl get pods
NAME READY STATUS RESTARTS AGE
blog-backend-54459fffdc-7zr2b 1/1 Running 0 9m51s
blog-backend-54459fffdc-s5bpr 1/1 Running 0 10m
blog-frontend-55dc9d7d9b-57ggr 1/1 Running 0 9m51s
blog-frontend-55dc9d7d9b-z7w2c 1/1 Running 0 10m
mysql-0 1/1 Running 0 2d18h
NP Lenovo@DESKTOP-L3A3H6G MINGW64 ~/Downloads/Project - 4/BlogReact (main)
$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Describe what to build
Local
main*
Vennilavanguvi (1 week ago) Ln 39, Col 18 Spaces: 2 UTF-8 CRLF JavaScript JSX


Inbox (4,482) - priyana
Classify
Project - 4 Documenta
Cloud Computing Serv
Load balancers | EC2 |
Instance details | EC2 |
Marginal — Field Note
#11 - Blog-Application
Ask Gemini
Not secure
a6dfd43fc5f2c4d2e938d20c5ea16330-530452254.ap-south-1.elb.amazonaws.com
Marginal FIELD NOTES / VOL. 04
Index
Sign in
Join
— NOTES FROM PRODUCTION - THIS IS VERSION 2(FOR TESTING PURPOSE)
Dispatches from the build pipeline.
Marginal is a running journal on shipping software that stays up: EKS rollouts, artifact hygiene, quality gates, and the vulnerabilities we almost missed.
4
posts live
EKS
deployment target
Trivy
gate on every image
Latest entries
all
devops
tooling
quality
security
DEVOPS
Shipping to EKS Without Losing Sleep
A deep dive into building a resilient GitOps pipeline for Kubernetes on AWS EKS.
TOOLING
Why We Moved Artifact Storage to Nexus
How centralizing artifacts in Nexus cut our build times in half.
QUALITY
Static Analysis That Developers Actually Like
Rolling out SonarQube quality gates without slowing teams down.

Task 12 : Monitoring

Overview

Prometheus and Grafana monitoring were implemented for the Amazon EKS environment using the kube-prometheus-stack.

Prometheus & Grafana

Prometheus was configured to collect Kubernetes and application metrics, while Grafana was used to visualize the monitored environment.

The monitoring stack was deployed in the monitoring namespace.

Metrics Monitored

The following metrics were verified:

- CPU utilization
- Memory utilization
- Pod status
- Pod restarts
- Node health
- Backend application metrics

Application Metrics

The Backend exposes a Prometheus-compatible `/metrics` endpoint.

A Kubernetes ServiceMonitor named blog-backend was configured to scrape the Backend metrics.

Prometheus Target Health verified:

serviceMonitor/monitoring/blog-backend/0
2 / 2 UP

Both Backend replicas were successfully discovered and scraped through the /metrics endpoint.

Verification

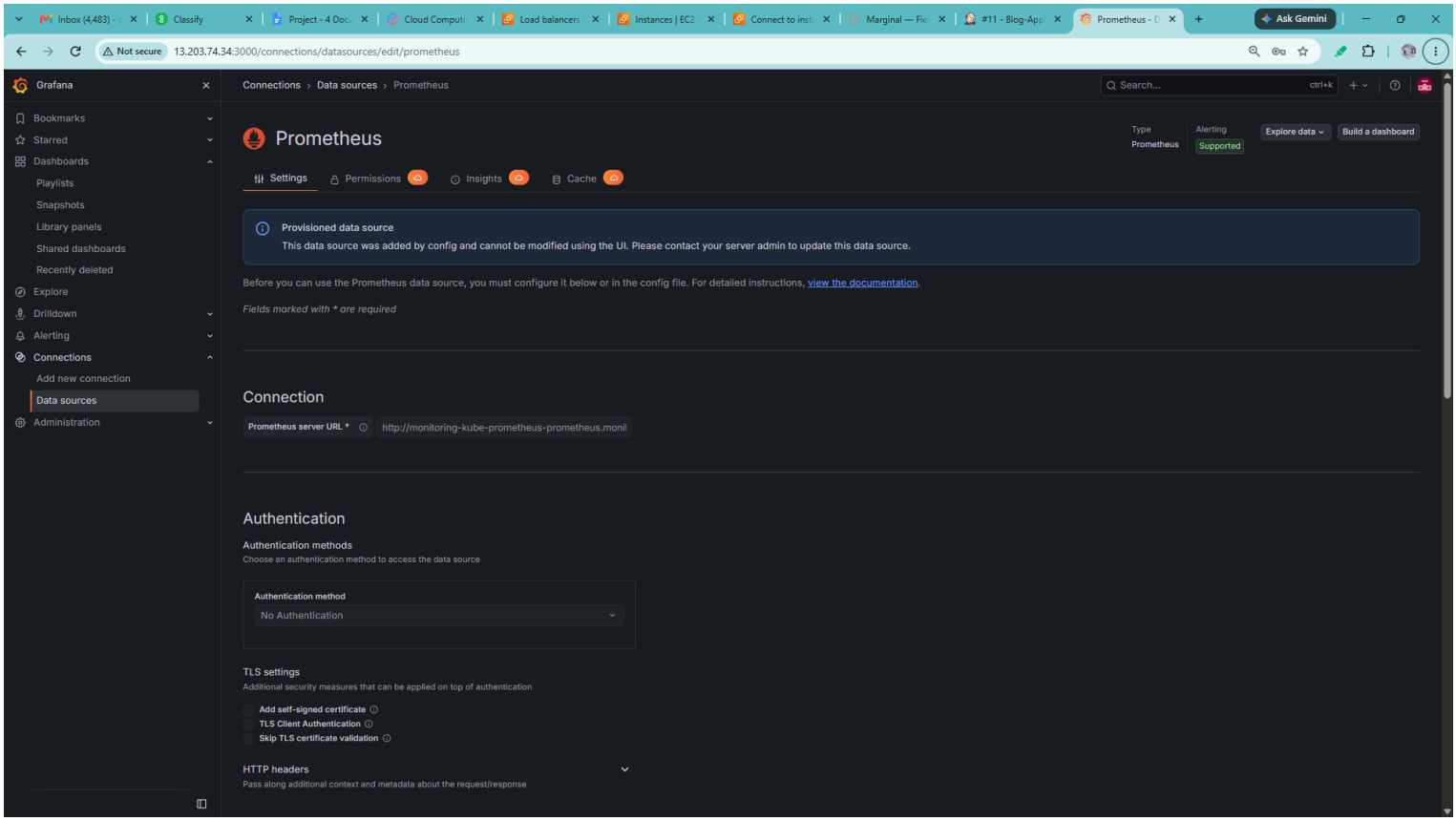
Prometheus Target Health showed the Backend monitoring targets as : 2 / 2 UP

Grafana dashboards were used to verify Kubernetes cluster, node, namespace, and pod resource usage.

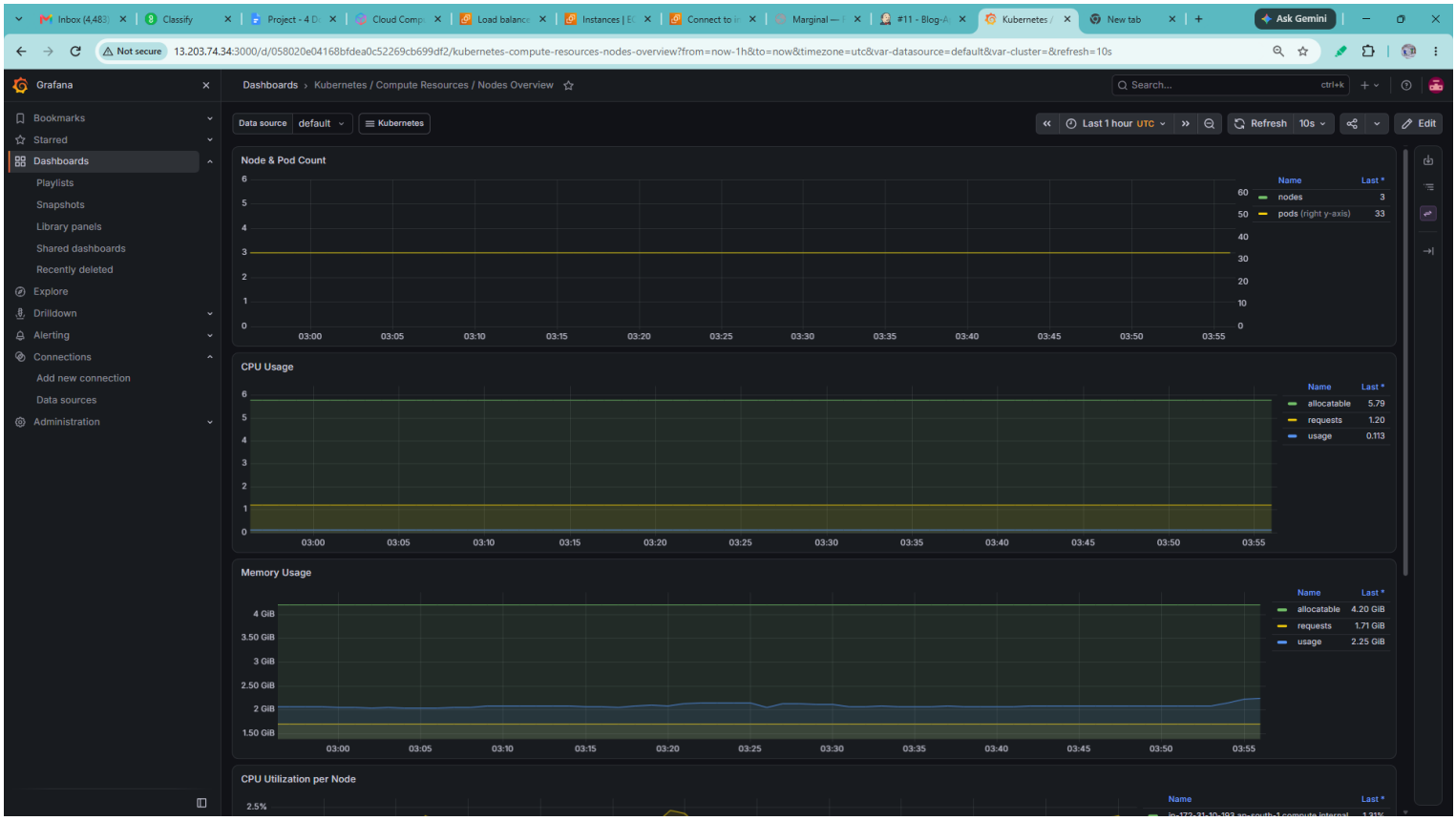
Pod restart metrics were also verified through Prometheus/Grafana.

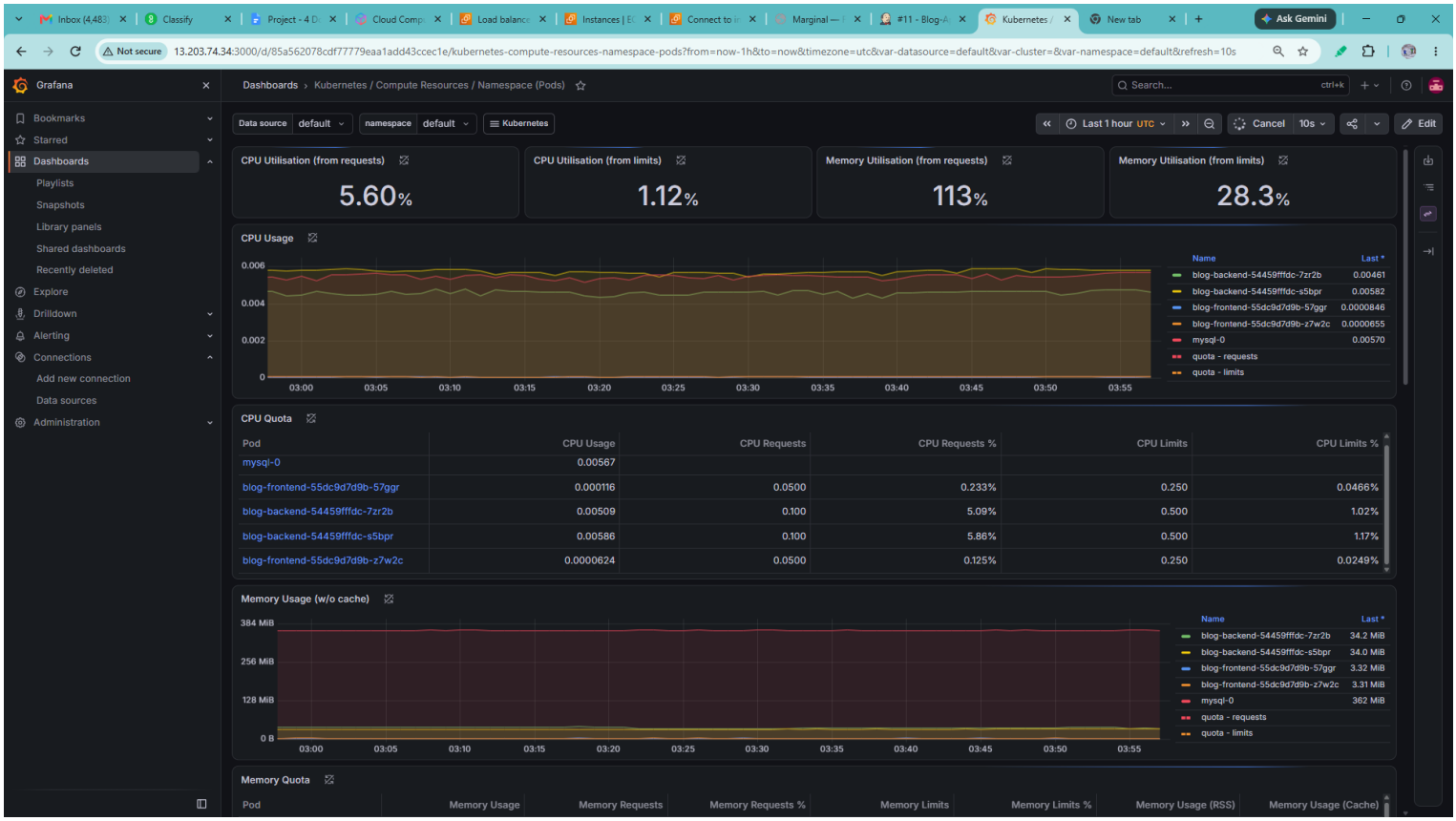
Result

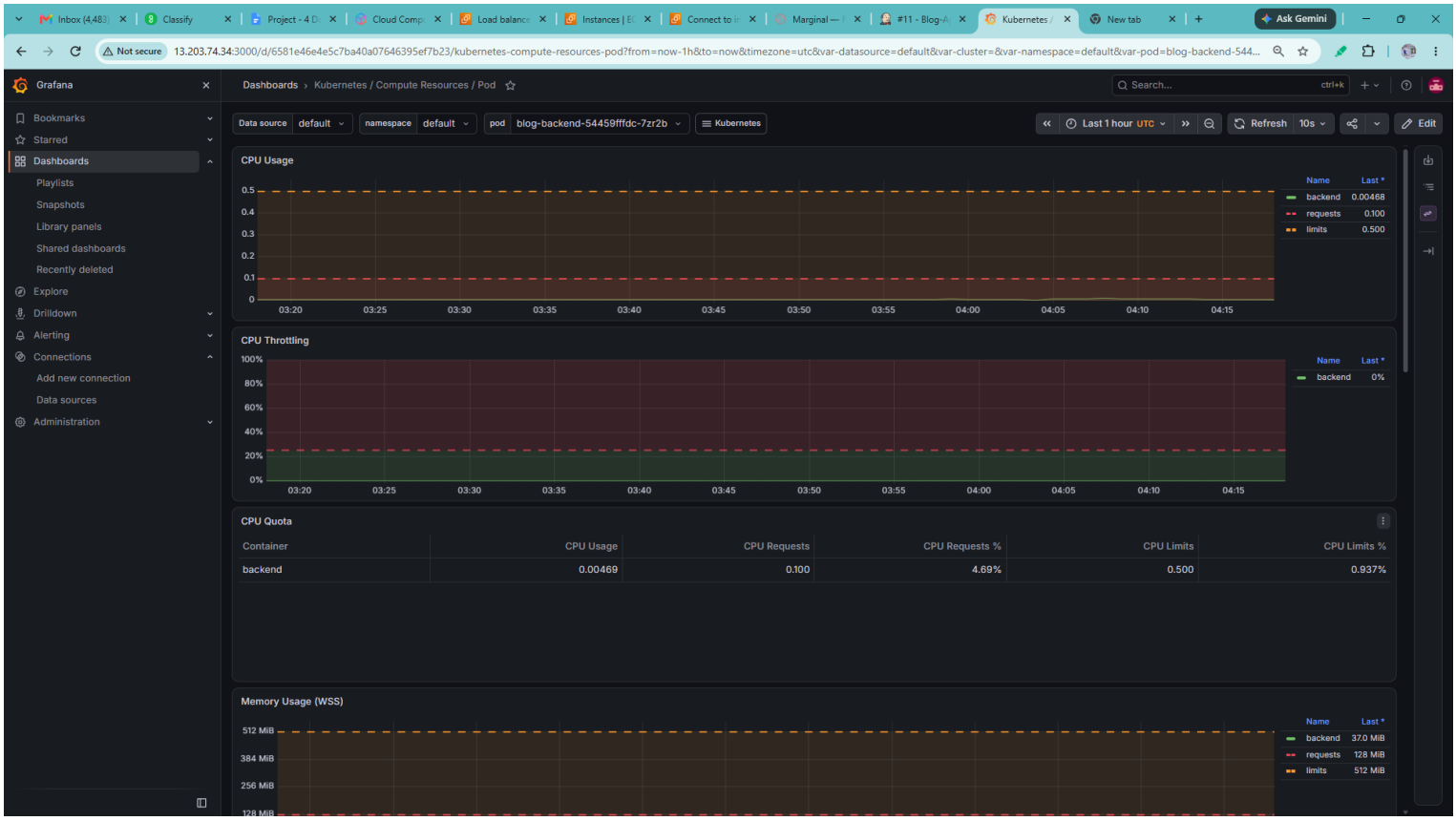
Prometheus and Grafana successfully provide monitoring for the EKS environment, including Kubernetes resource metrics, workload health, pod restarts, node health, and Backend application metrics.

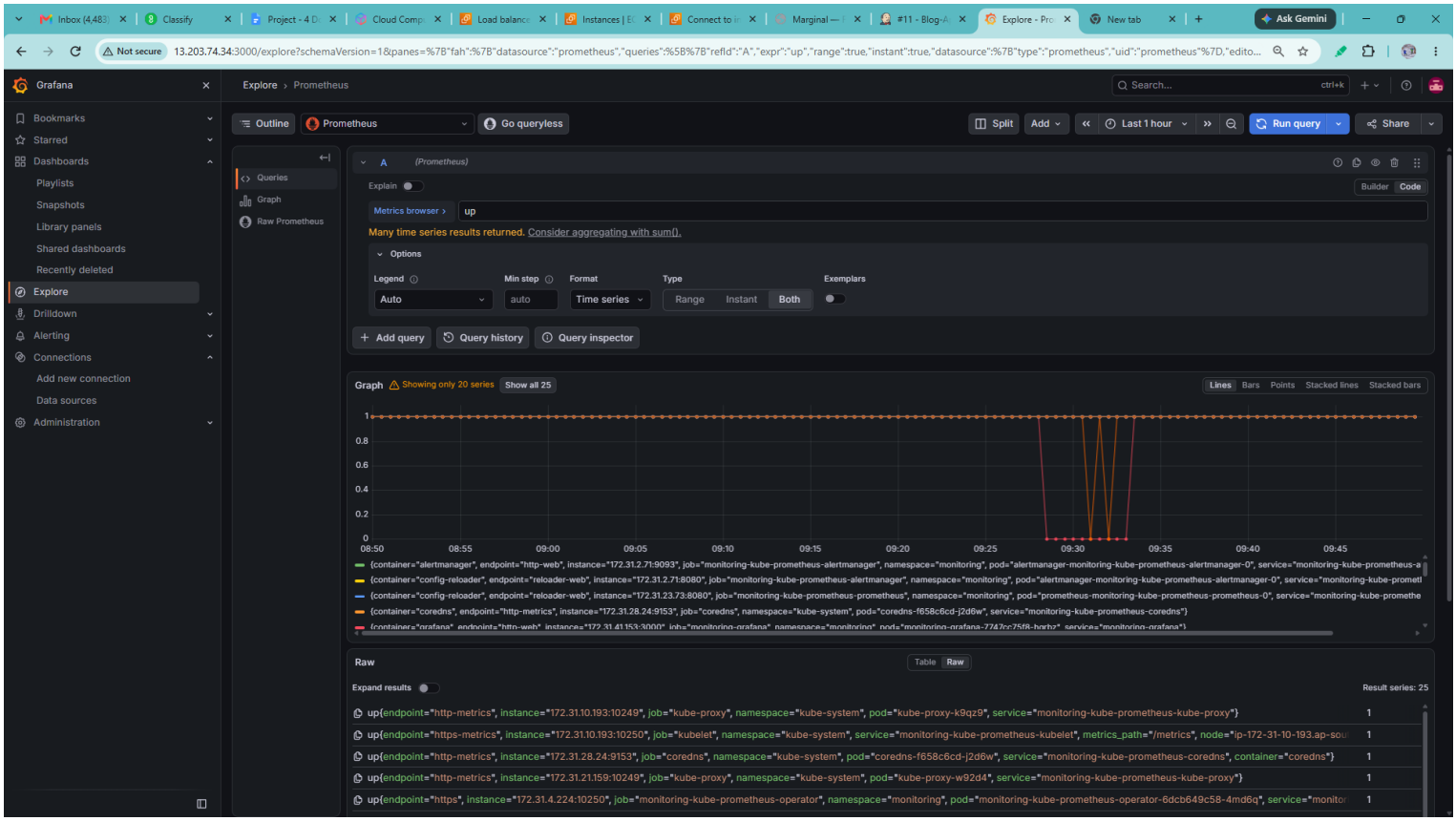


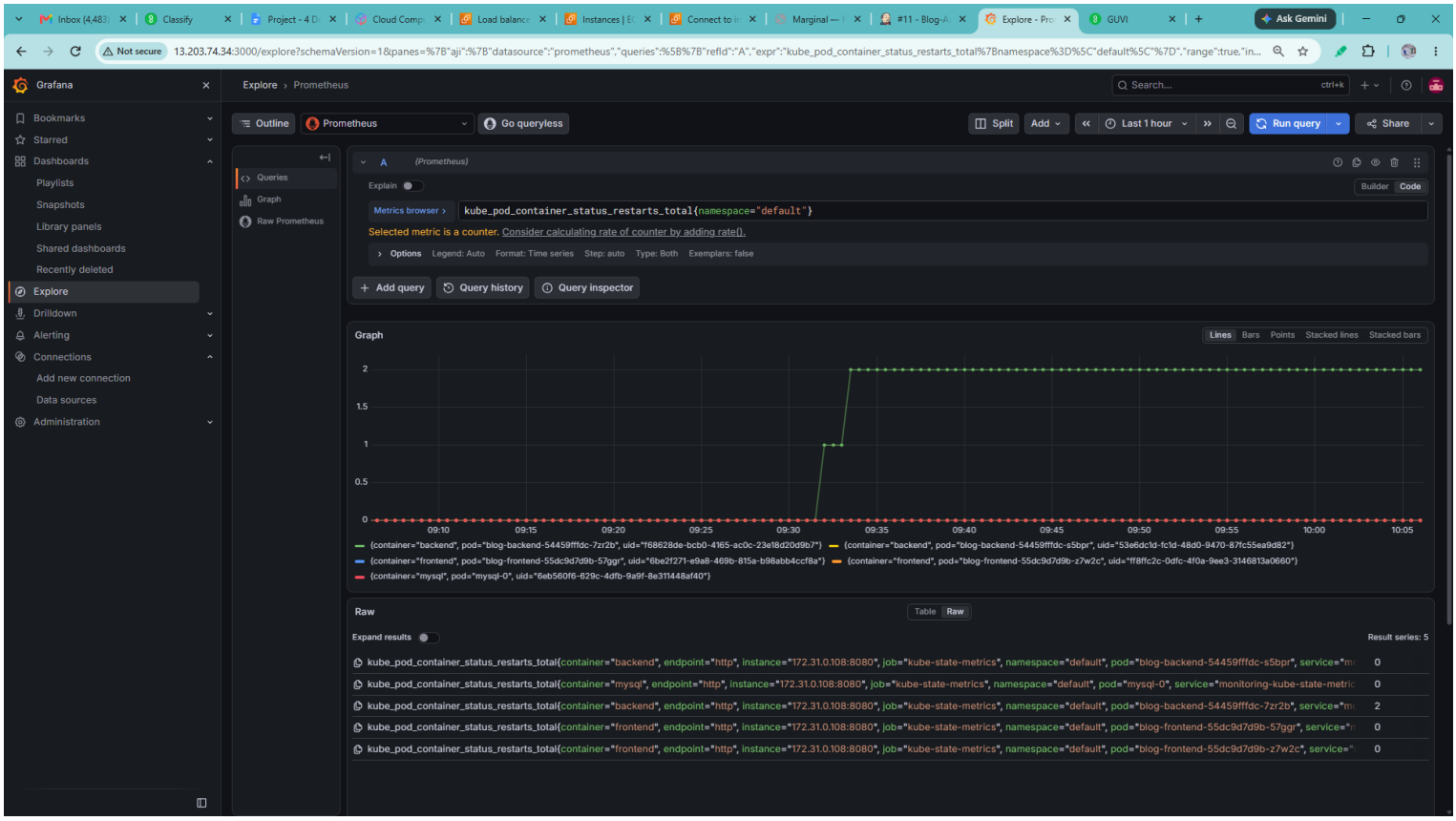

Grafana
Dashboards › Kubernetes / Compute Resources / Cluster
Data source default
Kubernetes
Last 1 hour UTC
Refresh 10s
Edit
CPU Utilisation
4.90%
CPU Requests Commitment
20.7%
CPU Limits Commitment
38.9%
Memory Utilisation
64.0%
Memory Requests Commitm...
40.7%
Memory Limits Commitment
148%
CPU Usage
Name Last *
default 0.0163
external-dns 0.000394
kube-system 0.0510
monitoring 0.0415
CPU Quota
Namespace Pods Workloads CPU Usage CPU Requests CPU Requests % CPU Limits CPU Limits %
kube-system 19 8 0.0510 0.890 5.73% 0.750 6.80%
monitoring 8 6 0.0452
default 5 3 0.0163 0.300 5.45% 1.50 1.09%
external-dns 1 1 0.000401 0.0100 4.01%
Memory
Name Last *
default 426 MiB
external-dns 12.9 MiB
kube-system 476 MiB
monitoring 853 MiB
Memory Requests by Namespace
Namespace Pods Workloads Memory Usage Memory Requests Memory Requests % Memory Limits Memory Limits %
kube-system 19 8 473 MiB 1.11 GiB 41.6% 4.64 GiB 9.94%
monitoring 8 6 925 MiB 200 MiB 462%
default 5 3 426 MiB 384 MiB 111% 1.50 GiB 27.7%

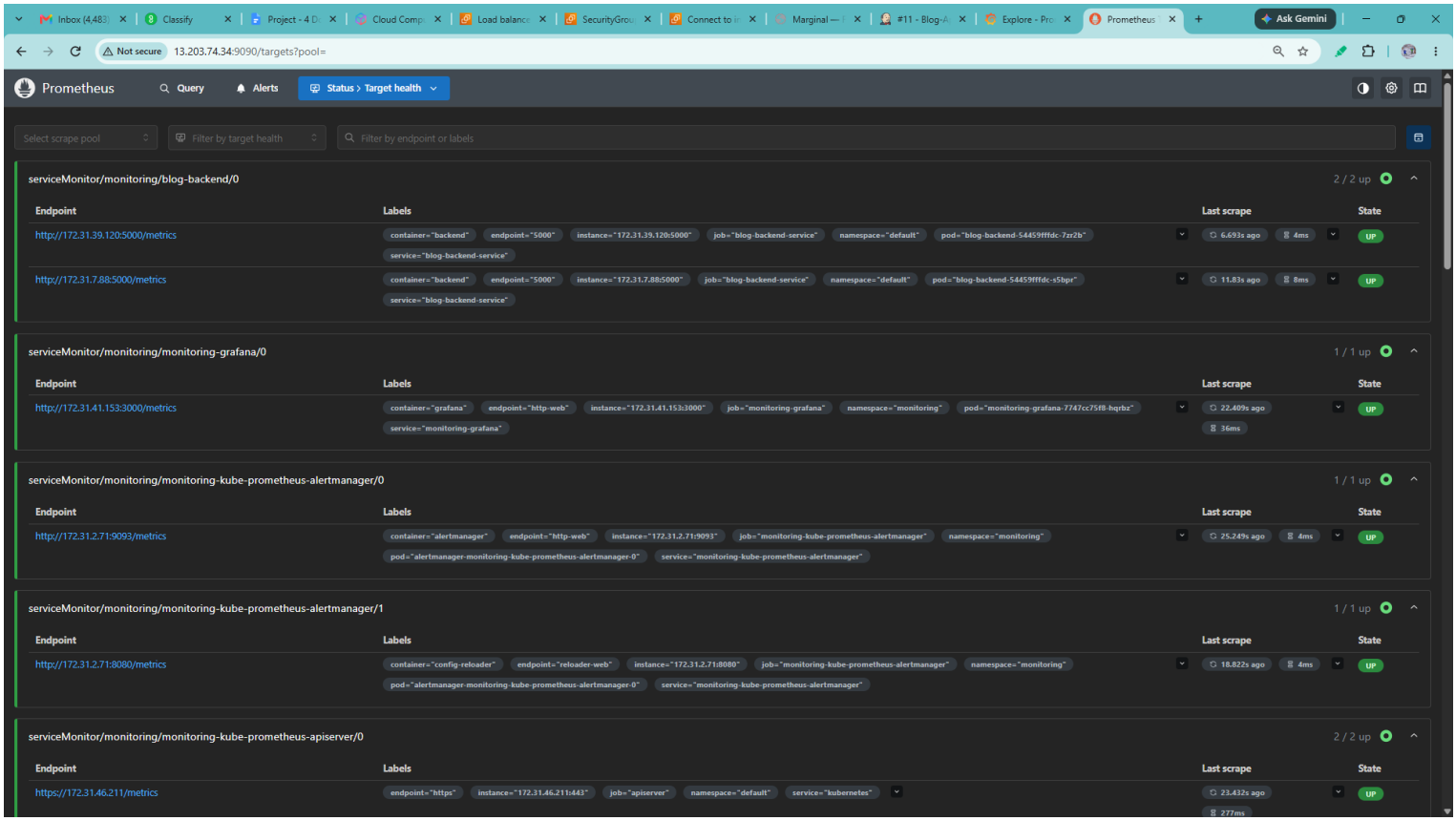

Grafana
Dashboards › Kubernetes / Compute Resources / Nodes Overview
Data source default
Kubernetes
Last 1 hour UTC
Refresh 10s
Edit
Node & Pod Count
Name Last *
nodes 3
pods (right y-axis) 33
CPU Usage
Name Last *
allocatable 5.79
requests 1.20
usage 0.113
Memory Usage
Name Last *
allocatable 4.20 GiB
requests 1.71 GiB
usage 2.25 GiB
CPU Utilization per Node


Grafana
Dashboards › Kubernetes / Compute Resources / Namespace (Pods)
Data source default
namespace default
Kubernetes
Last 1 hour UTC
Cancel
10s
Edit
CPU Utilisation (from requests)
5.60%
CPU Utilisation (from limits)
1.12%
Memory Utilisation (from requests)
113%
Memory Utilisation (from limits)
28.3%
CPU Usage
Name
Last *
blog-backend-54459fffdc-7zr2b 0.00461
blog-backend-54459fffdc-s5bpr 0.00582
blog-frontend-55dc9d7d9b-57ggr 0.0000846
blog-frontend-55dc9d7d9b-z7w2c 0.0000655
mysql-0 0.00570
quota - requests
quota - limits
CPU Quota
Pod
CPU Usage
CPU Requests
CPU Requests %
CPU Limits
CPU Limits %
mysql-0 0.00567
blog-frontend-55dc9d7d9b-57ggr 0.000116 0.0500 0.233% 0.250 0.0466%
blog-backend-54459fffdc-7zr2b 0.00509 0.100 5.09% 0.500 1.02%
blog-backend-54459fffdc-s5bpr 0.00586 0.100 5.86% 0.500 1.17%
blog-frontend-55dc9d7d9b-z7w2c 0.0000624 0.0500 0.125% 0.250 0.0249%
Memory Usage (w/o cache)
Memory Quota


Grafana
Dashboards › Kubernetes / Compute Resources / Pod
Data source default
namespace default
pod blog-backend-54459fffdc-7zr2b
Kubernetes
Last 1 hour UTC
Refresh
10s
Edit
CPU Usage
Name
Last *
backend 0.00468
requests 0.100
limits 0.500
CPU Throttling
backend 0%
CPU Quota
Container
CPU Usage
CPU Requests
CPU Requests %
CPU Limits
CPU Limits %
backend 0.00469 0.100 4.69% 0.500 0.937%
Memory Usage (WSS)
backend 37.0 MiB
requests 128 MiB
limits 512 MiB


Grafana
Explore › Prometheus
Outline
Prometheus
Go queryless
Split
Add
Last 1 hour
Run query
Share
Queries
Graph
Raw Prometheus
Explain
Builder
Code
Metrics browser ›
up
Many time series results returned. Consider aggregating with sum().
Options
Legend
Min step
Format
Type
Exemplars
Auto
auto
Time series
Range
Instant
Both
Add query
Query history
Query inspector
Graph
Showing only 20 series
Show all 25
Lines
Bars
Points
Stacked lines
Stacked bars
Raw
Table
Raw
Expand results
Result series: 25
up{endpoint="http-metrics", instance="172.31.10.193:10249", job="kube-proxy", namespace="kube-system", pod="kube-proxy-k9qz9", service="monitoring-kube-prometheus-kube-proxy"}
up{endpoint="https-metrics", instance="172.31.10.193:10250", job="kubelet", namespace="kube-system", service="monitoring-kube-prometheus-kubelet", metrics_path="/metrics", node="ip-172-31-10-193.ap-sou
up{endpoint="http-metrics", instance="172.31.28.24:9153", job="coredns", namespace="kube-system", pod="coredns-f658c6cd-j2d6w", service="monitoring-kube-prometheus-coredns", container="coredns"}
up{endpoint="http-metrics", instance="172.31.21.159:10249", job="kube-proxy", namespace="kube-system", pod="kube-proxy-w92d4", service="monitoring-kube-prometheus-kube-proxy"}
up{endpoint="https", instance="172.31.4.224:10250", job="monitoring-kube-prometheus-operator", namespace="monitoring", pod="monitoring-kube-prometheus-operator-6dcb649c58-4md6q", service="monitor


Grafana
Explore › Prometheus
Outline
Prometheus
Go queryless
Split
Add
Last 1 hour
Run query
Share
Queries
Graph
Raw Prometheus
Explain
Builder
Code
Metrics browser ›
kube_pod_container_status_restarts_total{namespace="default"}
Selected metric is a counter. Consider calculating rate of counter by adding rate().
Options
Legend: Auto
Format: Time series
Step: auto
Type: Both
Exemplars: false
Add query
Query history
Query inspector
Graph
Lines
Bars
Points
Stacked lines
Stacked bars
Raw
Table
Raw
Expand results
Result series: 5
kube_pod_container_status_restarts_total{container="backend", endpoint="http", instance="172.31.0.108:8080", job="kube-state-metrics", namespace="default", pod="blog-backend-54459fffdc-s5bpr", service="
kube_pod_container_status_restarts_total{container="mysql", endpoint="http", instance="172.31.0.108:8080", job="kube-state-metrics", namespace="default", pod="mysql-0", service="monitoring-kube-state-metric
kube_pod_container_status_restarts_total{container="backend", endpoint="http", instance="172.31.0.108:8080", job="kube-state-metrics", namespace="default", pod="blog-backend-54459fffdc-7zr2b", service="
kube_pod_container_status_restarts_total{container="frontend", endpoint="http", instance="172.31.0.108:8080", job="kube-state-metrics", namespace="default", pod="blog-frontend-55dc9d7d9b-57ggr", service="
kube_pod_container_status_restarts_total{container="frontend", endpoint="http", instance="172.31.0.108:8080", job="kube-state-metrics", namespace="default", pod="blog-frontend-55dc9d7d9b-z7w2c", service="


Prometheus
Query
Alerts
Status > Target health
Select scrape pool
Filter by target health
Filter by endpoint or labels
serviceMonitor/monitoring/blog-backend/0
2 / 2 up
Endpoint
Labels
Last scrape
State
http://172.31.39.120:5000/metrics
container="backend" endpoint="5000" instance="172.31.39.120:5000" job="blog-backend-service" namespace="default" pod="blog-backend-54459fffdc-7zr2b" service="blog-backend-service"
6.693s ago
4ms
UP
http://172.31.7.88:5000/metrics
container="backend" endpoint="5000" instance="172.31.7.88:5000" job="blog-backend-service" namespace="default" pod="blog-backend-54459fffdc-s5bpr" service="blog-backend-service"
11.83s ago
8ms
UP
serviceMonitor/monitoring/monitoring-grafana/0
1 / 1 up
Endpoint
Labels
Last scrape
State
http://172.31.41.153:3000/metrics
container="grafana" endpoint="http-web" instance="172.31.41.153:3000" job="monitoring-grafana" namespace="monitoring" pod="monitoring-grafana-7747cc75f8-hqrbz" service="monitoring-grafana"
22.409s ago
36ms
UP
serviceMonitor/monitoring/monitoring-kube-prometheus-alertmanager/0
1 / 1 up
Endpoint
Labels
Last scrape
State
http://172.31.2.71:9093/metrics
container="alertmanager" endpoint="http-web" instance="172.31.2.71:9093" job="monitoring-kube-prometheus-alertmanager" namespace="monitoring" pod="alertmanager-monitoring-kube-prometheus-alertmanager-0" service="monitoring-kube-prometheus-alertmanager"
25.249s ago
4ms
UP
serviceMonitor/monitoring/monitoring-kube-prometheus-alertmanager/1
1 / 1 up
Endpoint
Labels
Last scrape
State
http://172.31.2.71:8080/metrics
container="config-reloader" endpoint="reloader-web" instance="172.31.2.71:8080" job="monitoring-kube-prometheus-alertmanager" namespace="monitoring" pod="alertmanager-monitoring-kube-prometheus-alertmanager-0" service="monitoring-kube-prometheus-alertmanager"
18.822s ago
4ms
UP
serviceMonitor/monitoring/monitoring-kube-prometheus-apiserver/0
2 / 2 up
Endpoint
Labels
Last scrape
State
https://172.31.46.211/metrics
endpoint="https" instance="172.31.46.211:443" job="apiserver" namespace="default" service="kubernetes"
23.432s ago
277ms
UP

Task 13 : Logging

Overview

Kubernetes-based application logging was implemented using Kubernetes workload logs.

Application Logging

Logs were verified for the following components:

- Frontend
- Backend
- Database
- Kubernetes workloads

Frontend Logs

Frontend logs were retrieved using : kubectl logs deployment/blog-frontend --tail=30

The logs showed HTTP requests and successful 200 responses from the Frontend workload.

Backend Logs

Backend logs were retrieved using : kubectl logs deployment/blog-backend --tail=30

The logs showed application health and readiness requests with successful 200 responses.

Database Logs

MySQL logs were retrieved using : kubectl logs statefulset/mysql --tail=30

The logs showed MySQL initialization, InnoDB startup, and the database becoming ready for connections on port 3306.

Kubernetes Workload Logs

Logs were also retrieved directly from a running Backend pod using:

kubectl logs blog-backend-54459fffdc-7zr2b --tail=20

This demonstrated direct log inspection of an individual Kubernetes workload.

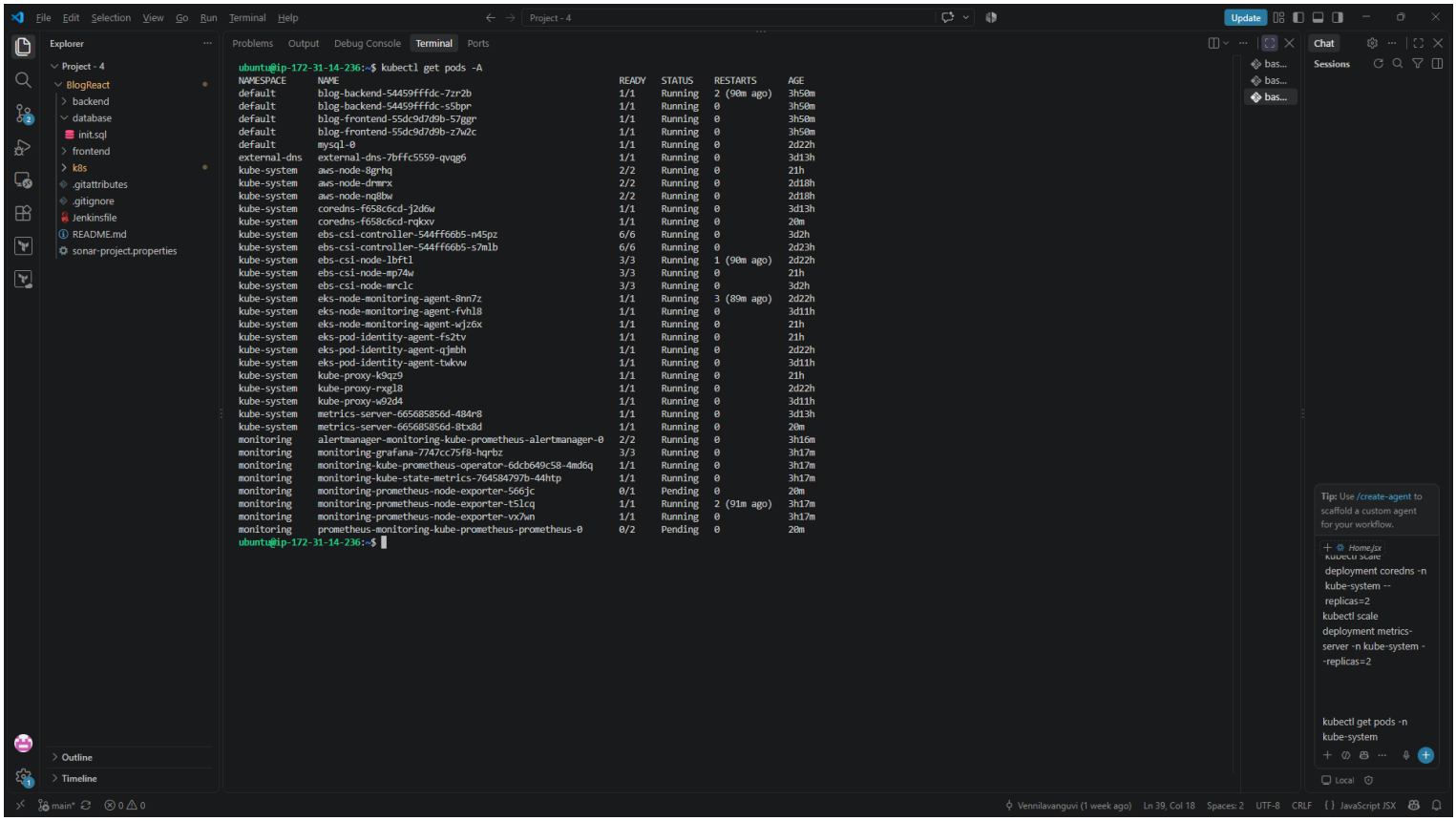
Verification

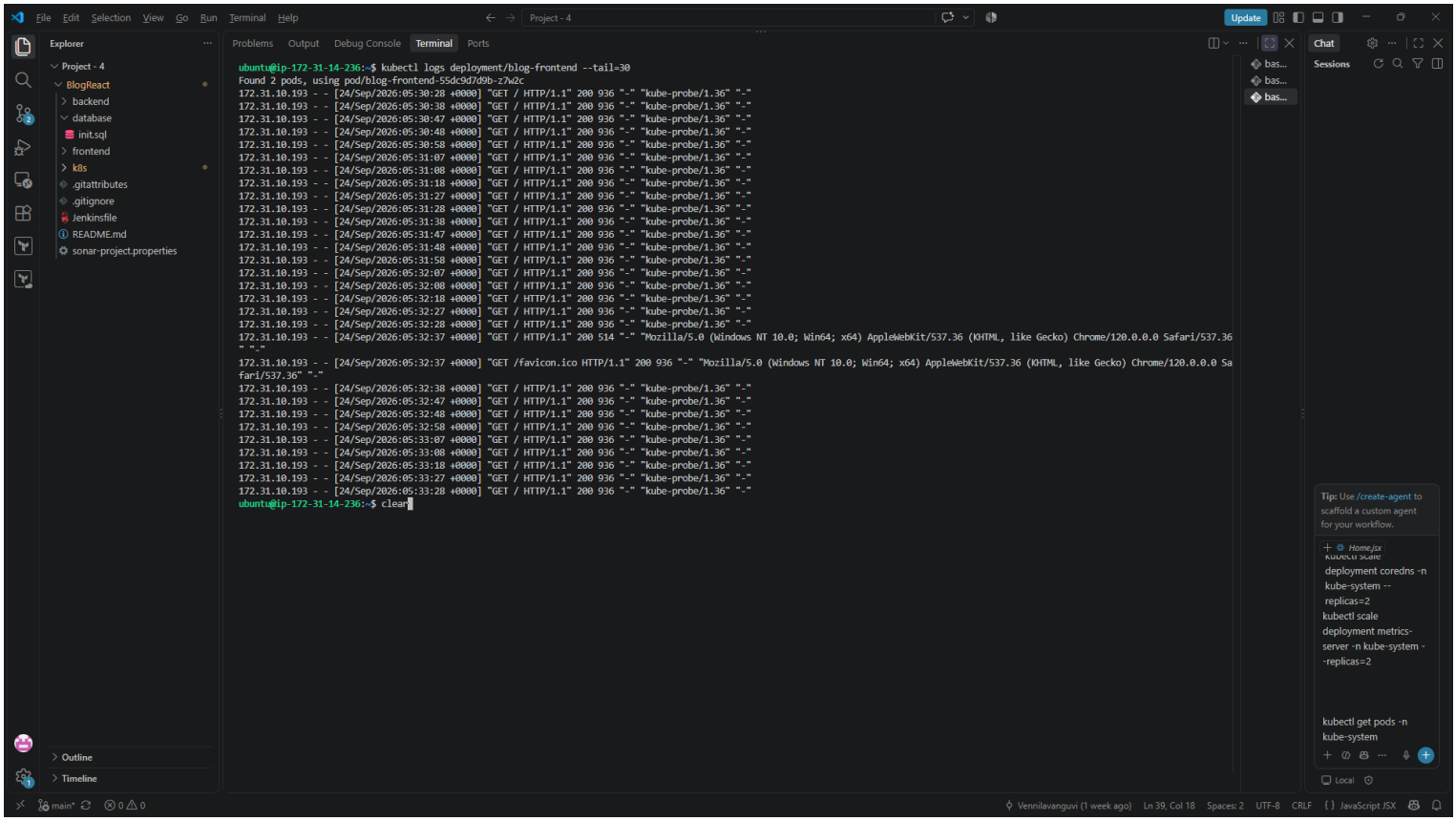
Kubernetes logs were successfully retrieved for the Frontend, Backend, Database, and application workloads.

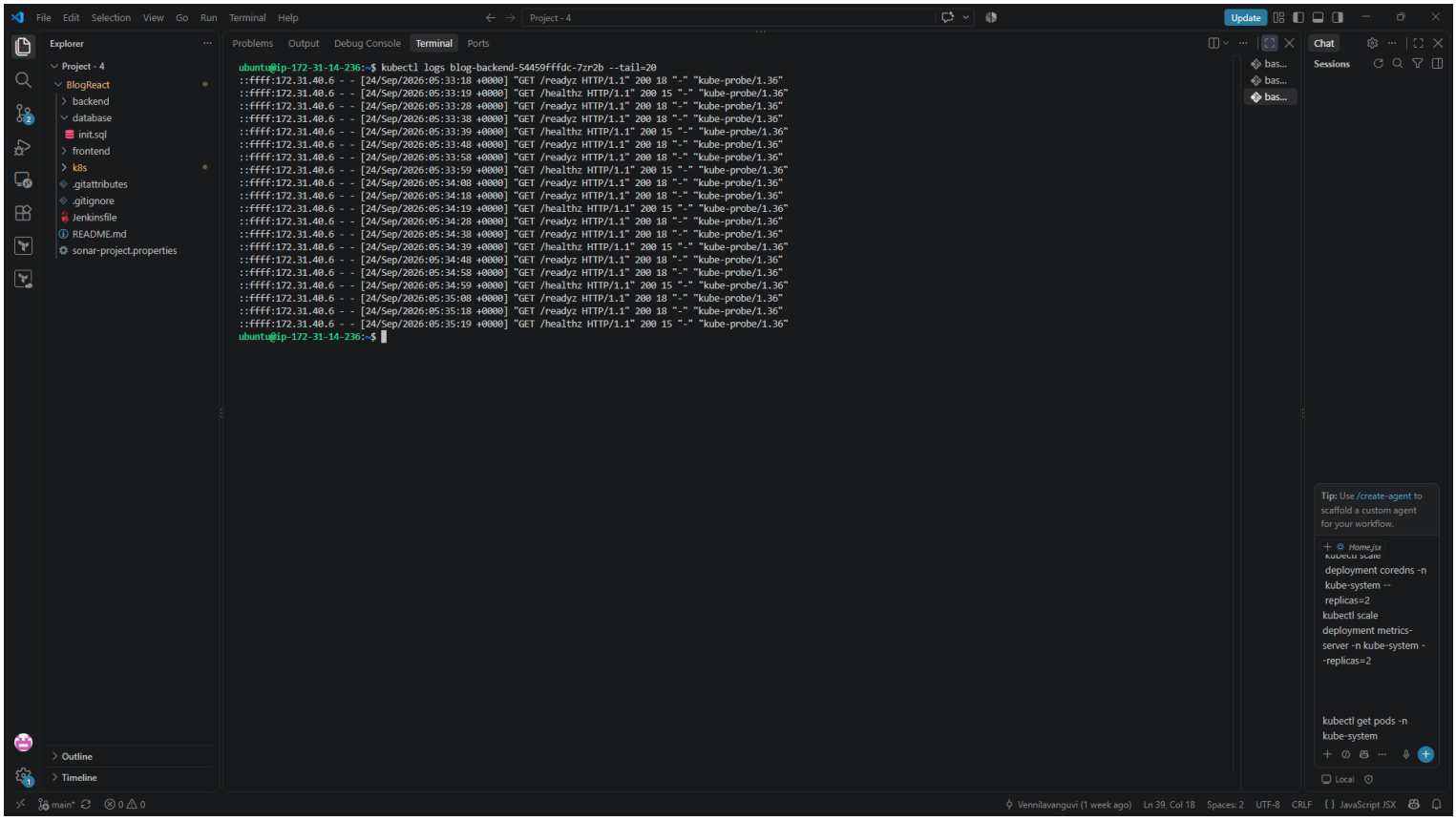
The available logs provide information such as HTTP request status, health/readiness checks, application activity, and database startup status, which can be used to identify application or deployment issues.

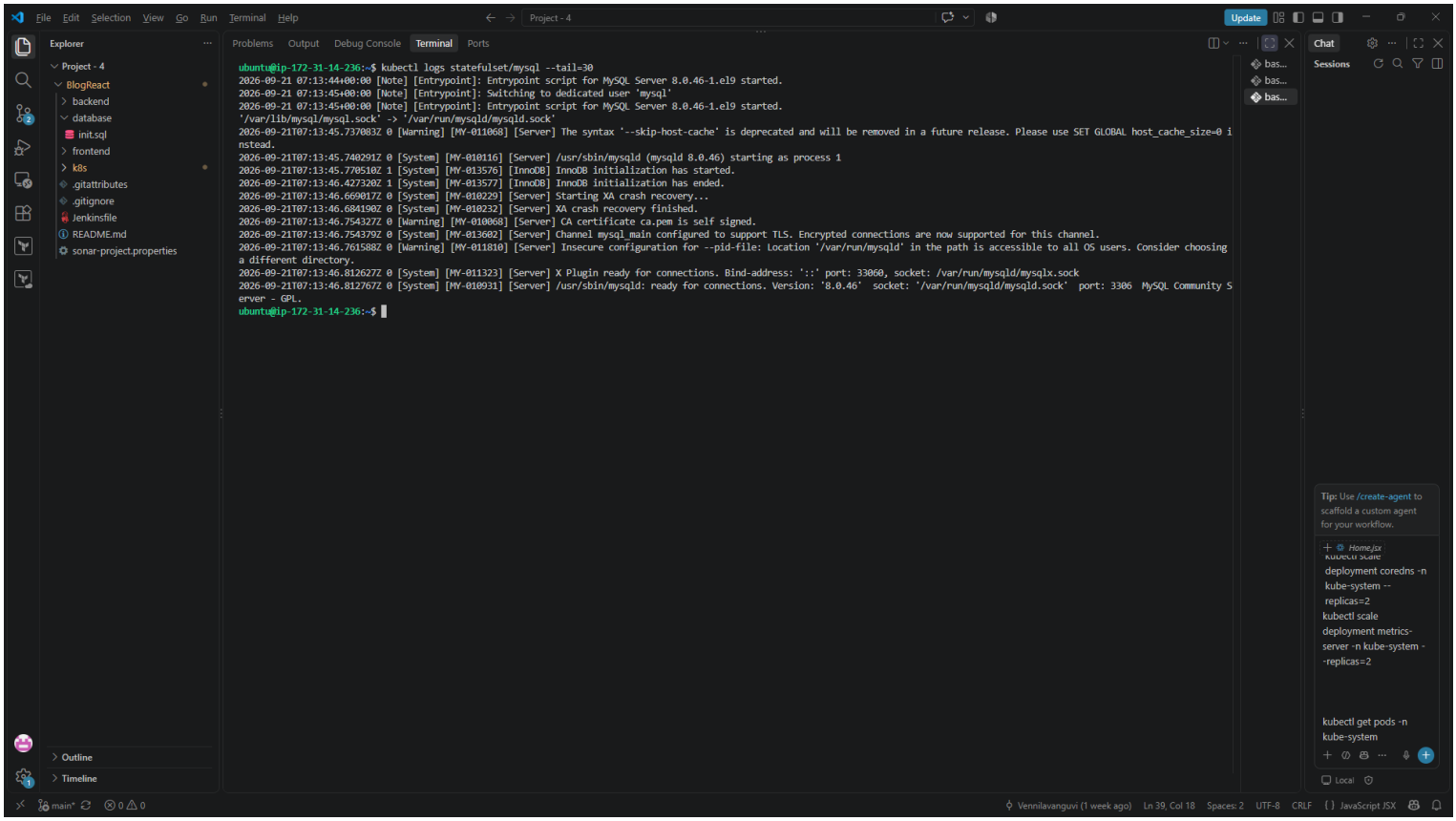
Result

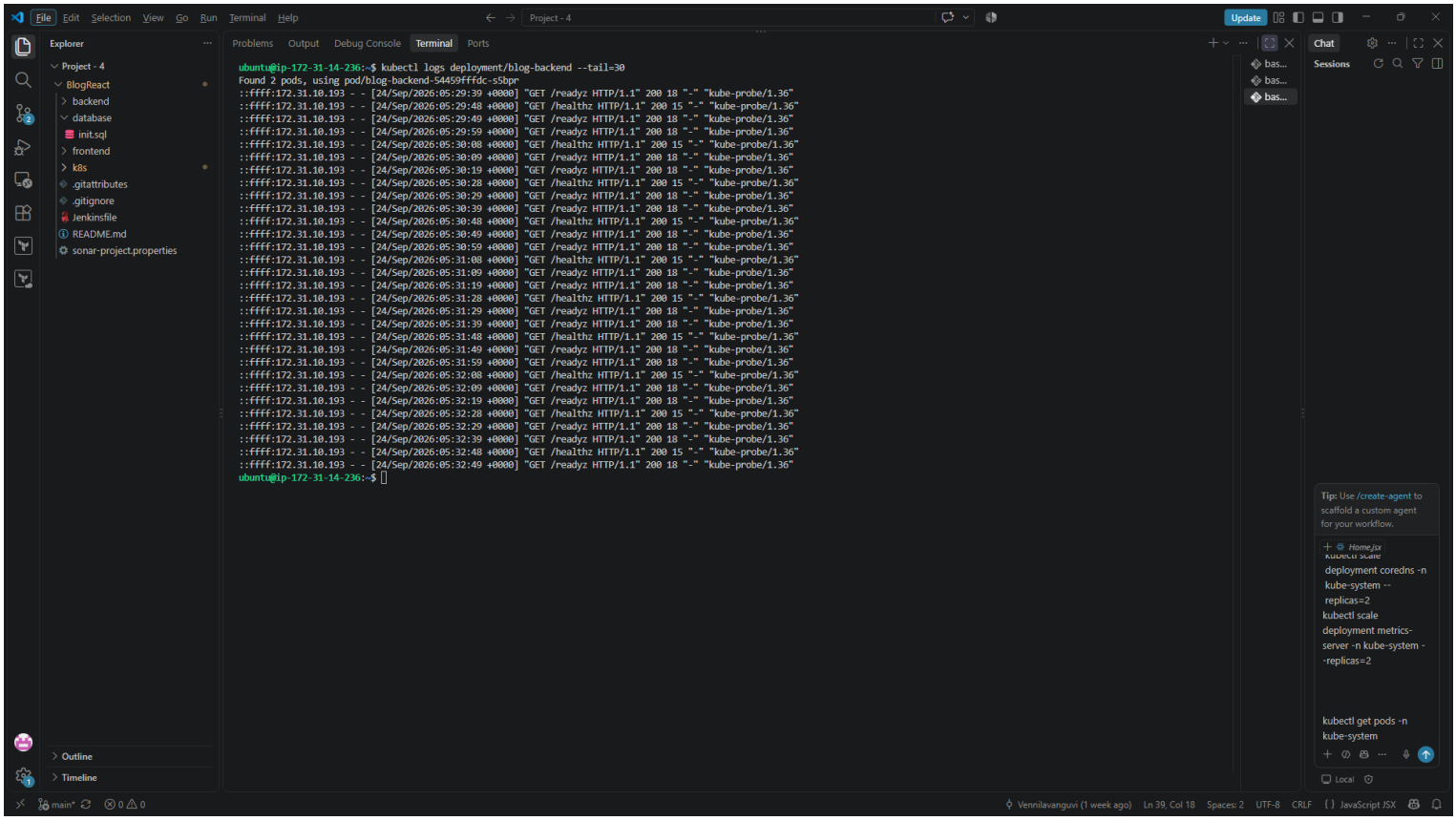
Kubernetes-based logging was successfully demonstrated for the deployed Blog Application. Logs from the Frontend, Backend, Database, and Kubernetes workloads can be inspected for application and deployment troubleshooting.




File Edit Selection View Go Run Terminal Help
Project - 4
Update
Explorer
Project - 4
BlogReact
backend
database
init.sql
frontend
k8s
.gitattributes
.gitignore
Jenkinsfile
README.md
sonar-project.properties
Outline
Timeline
Problems Output Debug Console Terminal Ports
ubuntu@ip-172-31-14-236:~$ kubectl logs deployment/blog-frontend --tail=30
Found 2 pods, using pod/blog-frontend-55dc9d7d9b-z7w2c
172.31.10.193 - - [24/Sep/2026:05:30:28 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:30:38 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:30:47 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:30:48 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:30:58 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:31:07 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:31:08 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:31:18 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:31:27 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:31:28 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:31:38 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:31:47 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:31:48 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:31:58 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:32:07 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:32:08 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:32:18 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:32:27 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:32:28 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:32:37 +0000] "GET / HTTP/1.1" 200 514 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36" "-"
172.31.10.193 - - [24/Sep/2026:05:32:37 +0000] "GET /favicon.ico HTTP/1.1" 200 936 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36" "-"
172.31.10.193 - - [24/Sep/2026:05:32:38 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:32:47 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:32:48 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:32:58 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:33:07 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:33:08 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:33:18 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:33:27 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
172.31.10.193 - - [24/Sep/2026:05:33:28 +0000] "GET / HTTP/1.1" 200 936 "-" "kube-probe/1.36" "-"
ubuntu@ip-172-31-14-236:~$ clear
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
deployment coredns -n kube-system --replicas=2
kubectl scale deployment metrics-server -n kube-system --replicas=2
kubectl get pods -n kube-system
Local
main*
Vennilavanguvi (1 week ago)
Ln 39, Col 18
Spaces: 2
UTF-8
CRLF
JavaScript JSX


File Edit Selection View Go Run Terminal Help
Project - 4
Update
Explorer
Project - 4
BlogReact
backend
database
init.sql
frontend
k8s
.gitattributes
.gitignore
Jenkinsfile
README.md
sonar-project.properties
Outline
Timeline
Problems Output Debug Console Terminal Ports
ubuntu@ip-172-31-14-236:~$ kubectl logs blog-backend-54459fffdc-7zr2b --tail=20
::ffff:172.31.40.6 - - [24/Sep/2026:05:33:18 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.40.6 - - [24/Sep/2026:05:33:19 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "kube-probe/1.36"
::ffff:172.31.40.6 - - [24/Sep/2026:05:33:28 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.40.6 - - [24/Sep/2026:05:33:38 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.40.6 - - [24/Sep/2026:05:33:39 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "kube-probe/1.36"
::ffff:172.31.40.6 - - [24/Sep/2026:05:33:48 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.40.6 - - [24/Sep/2026:05:33:58 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.40.6 - - [24/Sep/2026:05:33:59 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "kube-probe/1.36"
::ffff:172.31.40.6 - - [24/Sep/2026:05:34:08 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.40.6 - - [24/Sep/2026:05:34:18 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.40.6 - - [24/Sep/2026:05:34:19 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "kube-probe/1.36"
::ffff:172.31.40.6 - - [24/Sep/2026:05:34:28 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.40.6 - - [24/Sep/2026:05:34:38 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.40.6 - - [24/Sep/2026:05:34:39 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "kube-probe/1.36"
::ffff:172.31.40.6 - - [24/Sep/2026:05:34:48 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.40.6 - - [24/Sep/2026:05:34:58 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.40.6 - - [24/Sep/2026:05:34:59 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "kube-probe/1.36"
::ffff:172.31.40.6 - - [24/Sep/2026:05:35:08 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.40.6 - - [24/Sep/2026:05:35:18 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.40.6 - - [24/Sep/2026:05:35:19 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "kube-probe/1.36"
ubuntu@ip-172-31-14-236:~$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
deployment coredns -n kube-system --replicas=2
kubectl scale deployment metrics-server -n kube-system --replicas=2
kubectl get pods -n kube-system
Local
main*
Vennilavanguvi (1 week ago)
Ln 39, Col 18
Spaces: 2
UTF-8
CRLF
JavaScript JSX


ubuntu@ip-172-31-14-236:~$ kubectl logs statefulset/mysql --tail=30
2026-09-21 07:13:44+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.
2026-09-21 07:13:45+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
2026-09-21 07:13:45+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.
'/var/lib/mysql/mysql.sock' -> '/var/run/mysqld/mysqld.sock'
2026-09-21T07:13:45.737083Z 0 [Warning] [MY-011068] [Server] The syntax '--skip-host-cache' is deprecated and will be removed in a future release. Please use SET GLOBAL host_cache_size=0 instead.
2026-09-21T07:13:45.740291Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.0.46) starting as process 1
2026-09-21T07:13:45.770510Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2026-09-21T07:13:46.427320Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2026-09-21T07:13:46.669017Z 0 [System] [MY-010229] [Server] Starting XA crash recovery...
2026-09-21T07:13:46.684190Z 0 [System] [MY-010232] [Server] XA crash recovery finished.
2026-09-21T07:13:46.754327Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
2026-09-21T07:13:46.754379Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted connections are now supported for this channel.
2026-09-21T07:13:46.761588Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mysqld' in the path is accessible to all OS users. Consider choosing a different directory.
2026-09-21T07:13:46.812627Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Bind-address: '::' port: 33060, socket: /var/run/mysqld/mysqlx.sock
2026-09-21T07:13:46.812767Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.0.46' socket: '/var/run/mysqld/mysqld.sock' port: 3306 MySQL Community Server - GPL.
ubuntu@ip-172-31-14-236:~$


ubuntu@ip-172-31-14-236:~$ kubectl logs deployment/blog-backend --tail=30
Found 2 pods, using pod/blog-backend-54459fffdc-s5bpr
::ffff:172.31.10.193 - - [24/Sep/2026:05:29:39 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:29:48 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:29:49 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:29:59 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:30:08 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:30:09 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:30:19 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:30:28 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:30:29 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:30:39 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:30:48 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:30:49 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:30:59 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:31:08 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:31:09 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:31:19 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:31:28 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:31:29 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:31:39 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:31:48 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:31:49 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:31:59 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:32:08 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:32:09 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:32:19 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:32:28 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:32:29 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:32:39 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:32:48 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "kube-probe/1.36"
::ffff:172.31.10.193 - - [24/Sep/2026:05:32:49 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.36"
ubuntu@ip-172-31-14-236:~$

Task 14 : Security

Overview

Production-oriented security practices were implemented across AWS, Kubernetes, containers, database access, and application configuration.

Security Controls

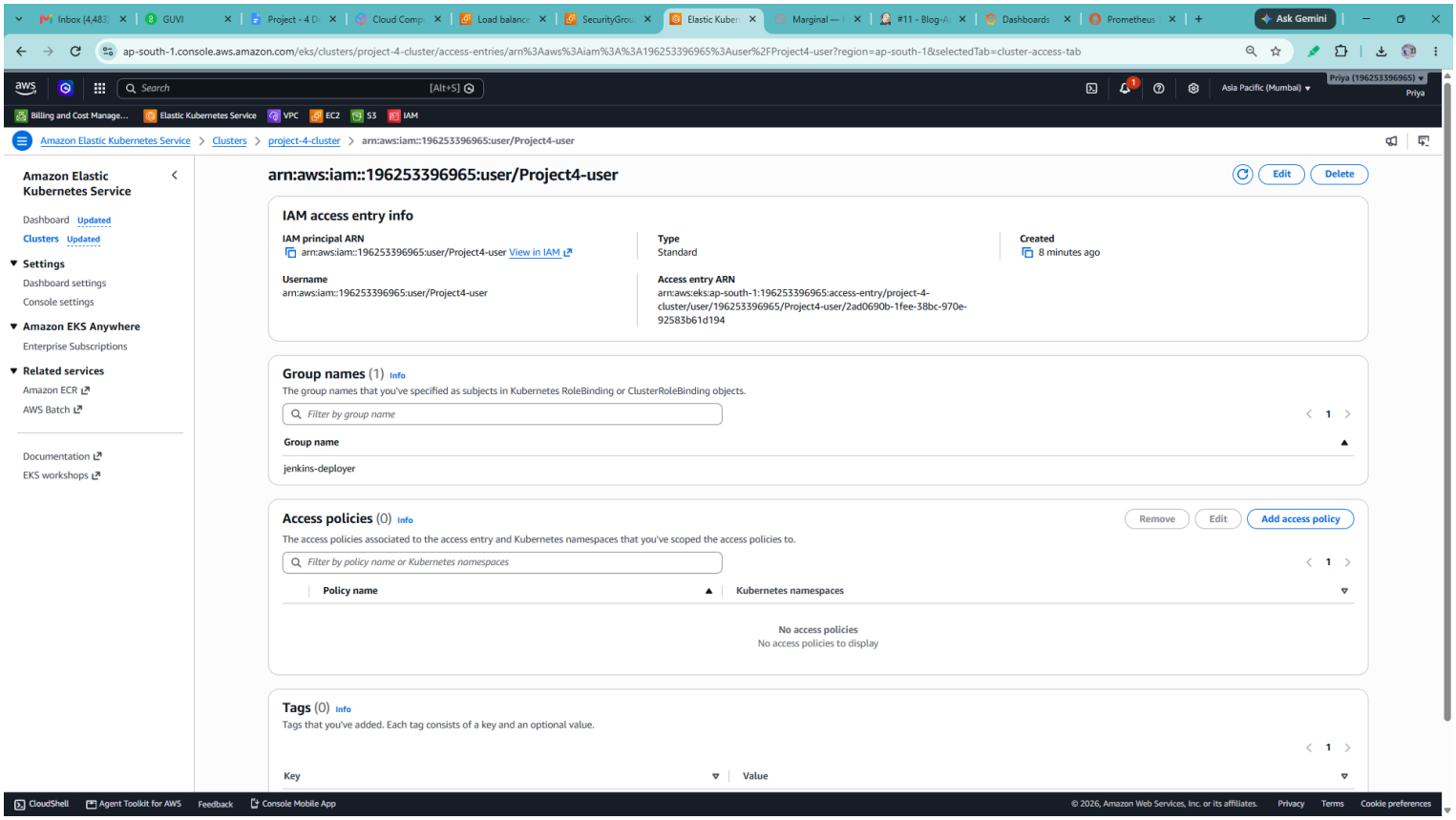
- IAM Least Privilege: A dedicated Project4-user was configured for Jenkins with the required permissions for ECR and EKS operations.
- Kubernetes RBAC: An EKS Access Entry, jenkins-deployer group, Role, and RoleBinding were configured to provide Jenkins with the required Kubernetes permissions.
- Restricted Network Access: The mysql-allow-backend-only NetworkPolicy restricts access to the MySQL database to the Backend workload.
- Secure Database Access: MySQL runs privately inside EKS using a ClusterIP Service and persistent storage.
- Container Security: Backend and Frontend containers run as non-root users (nodeuser and nginxuser).
- Vulnerability Scanning: Trivy scans the Frontend and Backend container images as part of the Jenkins pipeline before deployment.
- Secrets Management: Sensitive MySQL credentials are stored in the Kubernetes mysql-secret Secret and accessed using secretKeyRef.
- Git Protection: The actual Secret manifest is excluded from Git tracking using .gitignore.

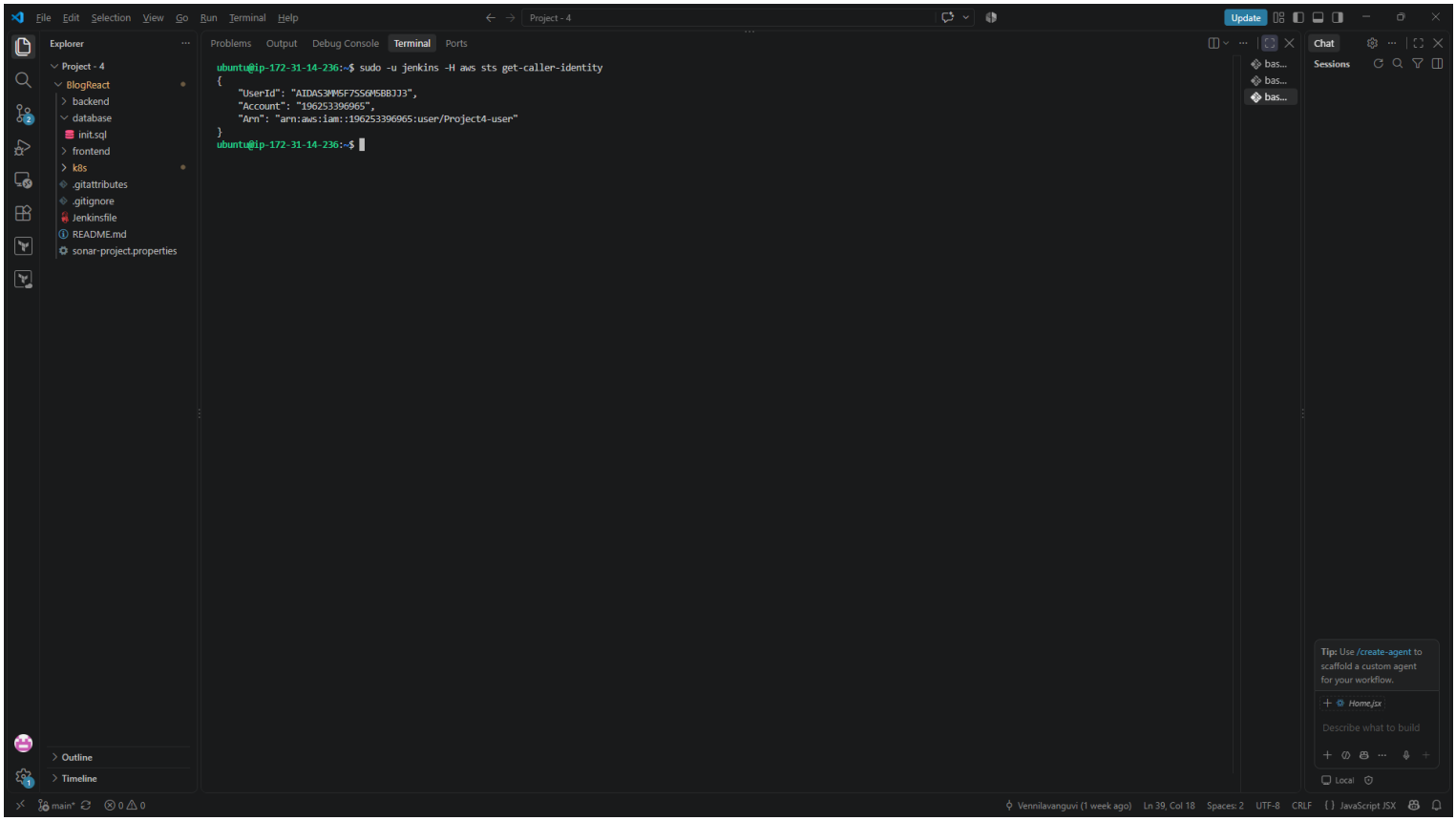
Verification

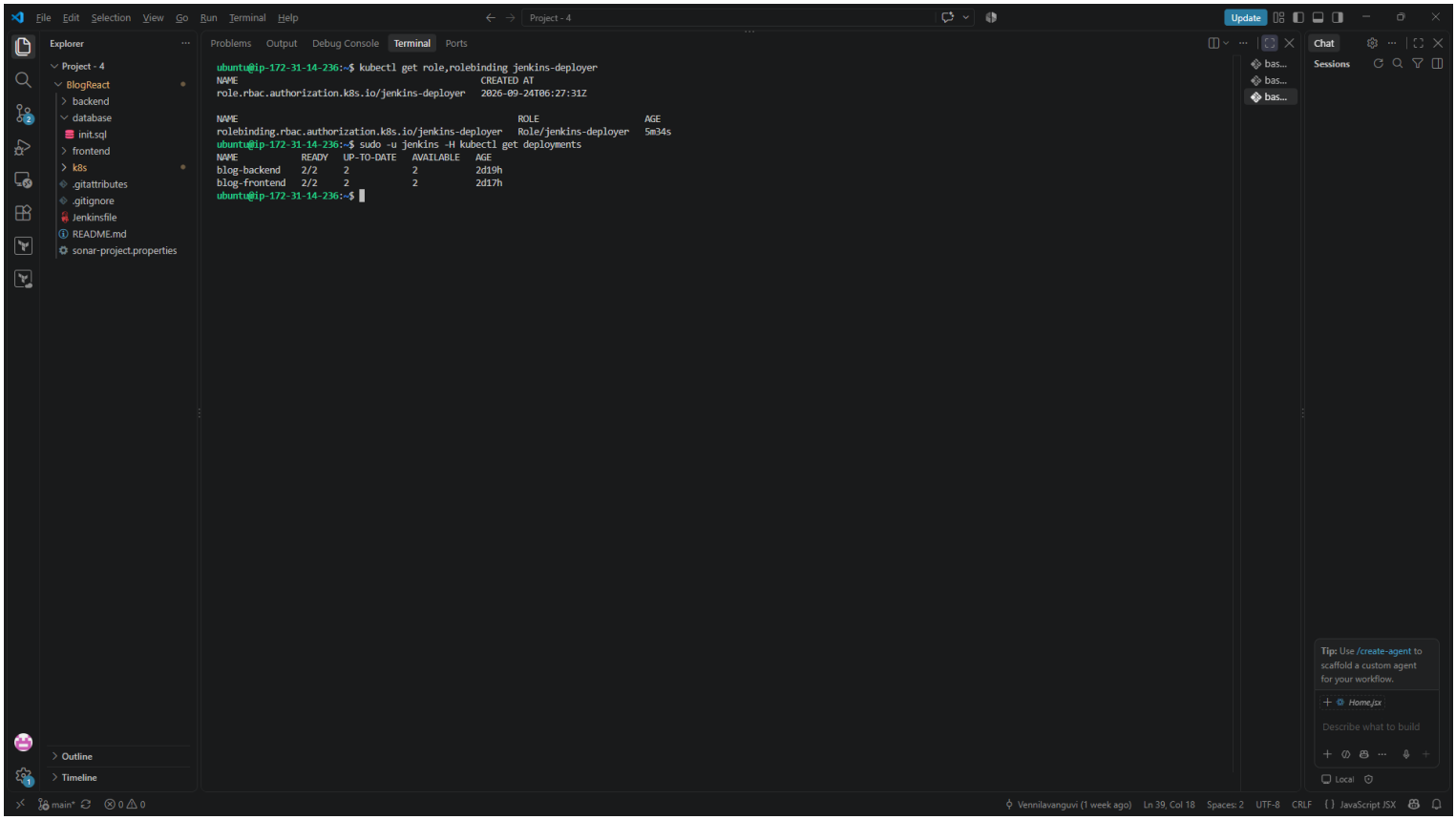
The security controls were verified through AWS IAM identity verification, EKS access configuration, Kubernetes RBAC, NetworkPolicy, non-root container configuration, Trivy scanning, Kubernetes Secret configuration, and Git tracking verification.

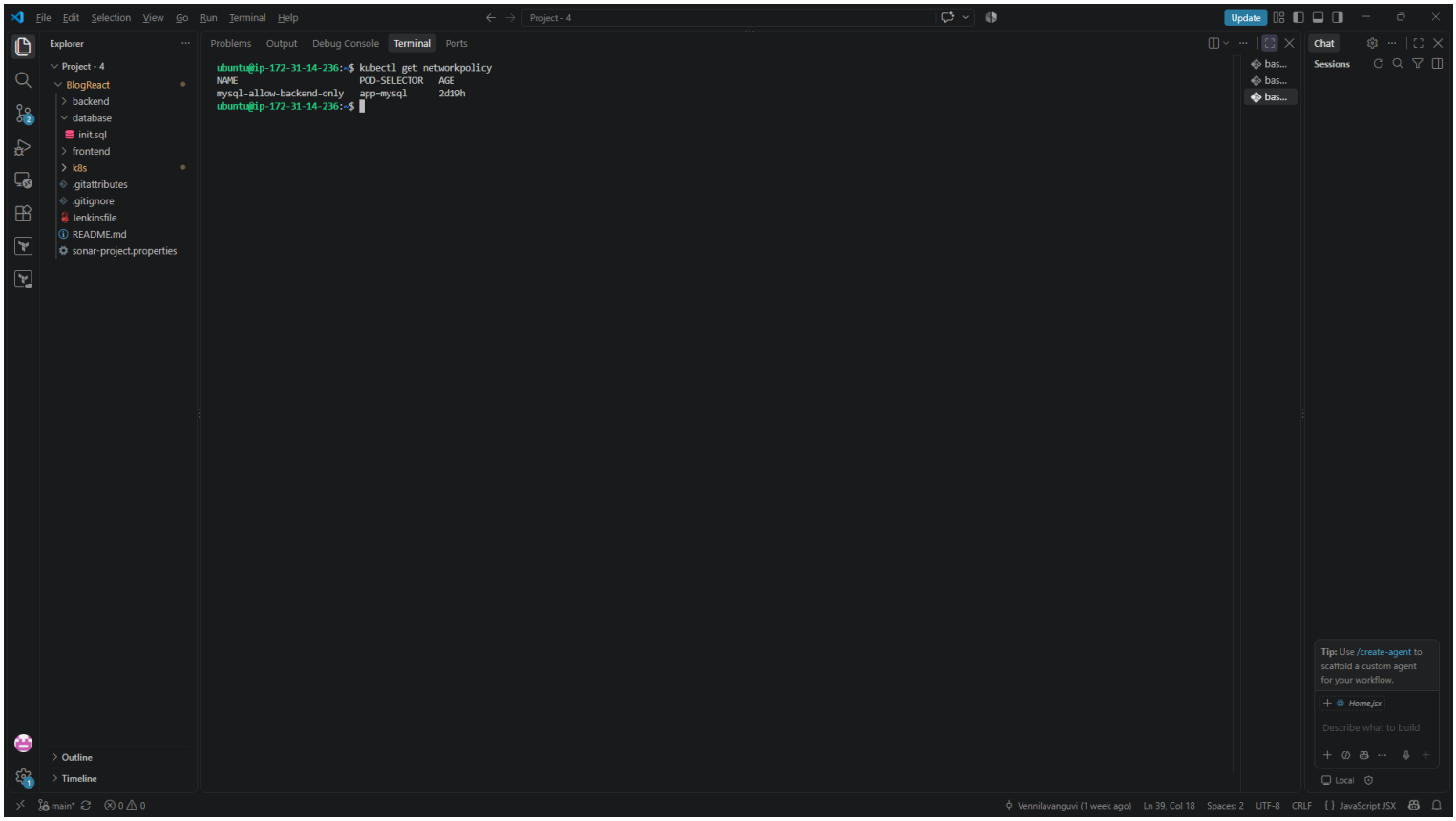
Result

The deployed Blog Application demonstrates the required AWS, Kubernetes, container, database, network, vulnerability scanning, and secrets-management security practices specified in the project.


ap-south-1.console.aws.amazon.com/eks/clusters/project-4-cluster/access-entries/arn%3Aaws%3Aiam%3A%3A196253396965%3Auser%2FProject4-user?region=ap-south-1&selectedTab=cluster-access-tab
Amazon Elastic Kubernetes Service > Clusters > project-4-cluster > arn:aws:iam::196253396965:user/Project4-user
Amazon Elastic Kubernetes Service
Dashboard Updated
Clusters Updated
Settings
Dashboard settings
Console settings
Amazon EKS Anywhere
Enterprise Subscriptions
Related services
Amazon ECR
AWS Batch
Documentation
EKS workshops
arn:aws:iam::196253396965:user/Project4-user
Edit
Delete
IAM access entry info
IAM principal ARN
arn:aws:iam::196253396965:user/Project4-user View in IAM
Type
Standard
Created
8 minutes ago
Username
arn:aws:iam::196253396965:user/Project4-user
Access entry ARN
arn:aws:eks:ap-south-1:196253396965:access-entry/project-4-cluster/user/196253396965/Project4-user/2ad0690b-1fee-38bc-970e-92583b61d194
Group names (1) Info
The group names that you've specified as subjects in Kubernetes RoleBinding or ClusterRoleBinding objects.
Filter by group name
Group name
jenkins-deployer
Access policies (0) Info
Remove
Edit
Add access policy
The access policies associated to the access entry and Kubernetes namespaces that you've scoped the access policies to.
Filter by policy name or Kubernetes namespaces
Policy name
Kubernetes namespaces
No access policies
No access policies to display
Tags (0) Info
Tags that you've added. Each tag consists of a key and an optional value.
Key
Value
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Explorer
Project - 4
BlogReact
backend
database
init.sql
frontend
k8s
.gitattributes
.gitignore
Jenkinsfile
README.md
sonar-project.properties
Problems
Output
Debug Console
Terminal
Ports
ubuntu@ip-172-31-14-236:~$ sudo -u jenkins -H aws sts get-caller-identity
{
"UserId": "AIDAS3MM5F7SS6M5BBJJ3",
"Account": "196253396965",
"Arn": "arn:aws:iam::196253396965:user/Project4-user"
}
ubuntu@ip-172-31-14-236:~$
Chat
Sessions
Outline
Timeline


File Edit Selection View Go Run Terminal Help
Project - 4
Update
Explorer
Project - 4
BlogReact
backend
database
init.sql
frontend
k8s
.gitattributes
.gitignore
Jenkinsfile
README.md
sonar-project.properties
Problems Output Debug Console Terminal Ports
ubuntu@ip-172-31-14-236:~$ kubectl get role,rolebinding jenkins-deployer
NAME CREATED AT
role.rbac.authorization.k8s.io/jenkins-deployer 2026-09-24T06:27:31Z
NAME ROLE AGE
rolebinding.rbac.authorization.k8s.io/jenkins-deployer Role/jenkins-deployer 5m34s
ubuntu@ip-172-31-14-236:~$ sudo -u jenkins -H kubectl get deployments
NAME READY UP-TO-DATE AVAILABLE AGE
blog-backend 2/2 2 2 2d19h
blog-frontend 2/2 2 2 2d17h
ubuntu@ip-172-31-14-236:~$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Home.jsx
Describe what to build
Local
Outline
Timeline
main*
Vennilavanguvi (1 week ago) Ln 39, Col 18 Spaces: 2 UTF-8 CRLF JavaScript JSX


File Edit Selection View Go Run Terminal Help
Project - 4
Update
Explorer
Project - 4
BlogReact
backend
database
init.sql
frontend
k8s
.gitattributes
.gitignore
Jenkinsfile
README.md
sonar-project.properties
Problems Output Debug Console Terminal Ports
ubuntu@ip-172-31-14-236:~$ kubectl get networkpolicy
NAME POD-SELECTOR AGE
mysql-allow-backend-only app=mysql 2d19h
ubuntu@ip-172-31-14-236:~$
Chat
Sessions
Tip: Use /create-agent to scaffold a custom agent for your workflow.
Home.jsx
Describe what to build
Local
Outline
Timeline
main*
Vennilavanguvi (1 week ago) Ln 39, Col 18 Spaces: 2 UTF-8 CRLF JavaScript JSX

Application URL

The deployed Blog Application is externally accessible through the AWS LoadBalancer :

URL : http://a6dfd43fc5f2c4d2e938d20c5ea16330-530452254.ap-south-1.elb.amazonaws.com

The URL opens the React Frontend and displays the blog posts successfully.